



黑马程序员
www.itheima.com

传智播客旗下
高端IT教育品牌

为莘莘学子改命运而讲课，为千万学生少走弯路而著书



黑马程序员™
www.itheima.com



博学谷

BIG DATA 狂野大数据 SQL面试宝典

V3.0 大厂高频面试题集锦

专治薪资上不去，技术转型问题

因为够牛，所以够狂！

北京市昌平区建材城西路金燕龙办公楼一层

电话：400-618-9090

课扫
程码
大获
全取





Python大数据V2.2SQL面试宝典

版本	增加模块日志	修改时间
V2.0	增加腾讯、华泰证券面试题	20220101
V2.1	增加Hive部分SQL经典10题	20220201
V2.2	增加广州网讯、字节跳动、平安产险、vivo、交通银行(第二套)面试题	20220303

适用范围：大数据各个阶段学习完之后提前准备阶段重点面试题

注意：如何讲解项目中的技术点？

- (1) 讲清楚知识点是什么
- (2) 讲清楚知识点原理是什么
- (3) 讲清楚知识点如何使用

讲清楚有没有优化及应用场景使用技巧：以计算组件为核心复习，主要复习Hive，Spark及Flink计算组件。



目录

Python大数据V2.0高频核心面试题.....	1
SQL提高秘籍.....	7
初级SQL-Hive10题.....	8
1. 需求列表.....	8
2. 第1题：访问量统计.....	8
3. 第2题：电商场景TopK统计.....	10
4. 第3题：订单量统计.....	13
5. 第4题：大数据排序统计.....	14
6. 第5题：活跃用户统计.....	16
7. 第6题：电商购买金额统计实战.....	18
8. 第7题：教育领域SQL实战.....	19
9. 第8题：服务日志SQL统计.....	24
10. 第9题：充值日志SQL实战.....	26
11. 第10题：电商分组TopK实战.....	28
微众银行.....	30
1. 需求列表.....	30
2. 源数据表：.....	30
3. 创建库和源数据表，并生成数据。.....	31
4. 第1题：授信金额统计.....	36
5. 第2题：贷款产品不良统计.....	41
6. 第3题：逾期率统计.....	46
7. 第4题：用户画像统计.....	49
8. 第5题：Vintage账龄分析.....	51
华泰证券.....	58
1. 需求列表.....	58
2. 第1题 查询每门课都大于80分的学生.....	58
3. 第2题 表转置.....	59



4. 第3题 业务需求实现.....	62
5. 第4题 查询每日买入金额前十和卖出金额前十的基金.....	64
6. 第5题 求每日每个客户每只股票的市值增长率.....	65
7. 第6题 计算基金净值回撤.....	66
招商信诺人寿.....	68
1. 需求列表.....	68
2. 第1题 排序取第8-16条记录.....	68
3. 第2题 SQL执行顺序.....	69
4. 第3题 班级分组内按成绩排名.....	69
5. 第4题 聚合窗口函数.....	71
6. 第5题 时间序列题型.....	72
交通银行.....	76
1. 需求列表.....	76
2. 第1题 金额环比增长率.....	76
3. 第2题 金额排名占比.....	77
4. 第3题 数据发散问题.....	79
5. 第4题 left/semi/anti/join区分.....	81
百丽.....	83
1. 需求列表.....	83
2. 第1题 Hive数据导入.....	83
3. 第2题 Grouping sets 维度分析.....	83
4. 第3题 百丽业务统计分析.....	84
跨越物流.....	87
1. 需求列表.....	87
2. 第1题 求出每个部门工资最高前三名员工，计算员工工资占所属部门总工资百分比.....	87
3. 第2题 统计每年入职总数以及截至本年累计入职总人数。.....	89
4. 第3题 每个品牌的总营销天数(时间序列题型).....	90
腾讯外包(微信相关).....	92
1. 需求列表.....	92



2. 第1题 Python数据清洗转换.....	92
3. 第2题 查询当天阅读最多的 100 篇文章.....	94
4. 第3题 查询当天阅读最多的 10 个公众号.....	94
小米电商.....	95
1. 需求列表.....	95
2. 第1题 用户订单量统计.....	95
3. 第2题 利用增量表得到全量表.....	97
4. 第3题 订单量中位数.....	97
5. 第3题 订单top10的城市.....	98
鄂州机场.....	99
1. 需求列表.....	99
2. 第1题 SQL基础考察.....	99
腾讯游戏数据开发A卷.....	103
1. 需求列表.....	103
2. 选择题.....	103
3. 解答题.....	104
4. 问答题.....	110
腾讯视频游戏直播.....	114
1. 需求列表.....	114
2. 第1题：活跃用户数、次日留存率.....	114
3. 第2题：用户留存率、用户活跃度得分排序.....	117
4. 第3题 考察点：row_number.....	119
5. 第4题 考察点：having子句的使用.....	119
帆软[MySQL方向].....	122
1. 需求列表.....	122
2. 第1题：销售环比增长率.....	122
3. 第2题 分组max-min-avg.....	123
4. 第3题 销售业绩综合统计.....	125
5. 第4题 LEFT JOIN、RIGHT JOIN、INNER JOIN、FULL JOIN的区别.....	132



PayPal	134
1. 需求列表	134
2. 第1题: Use Pyspark to generate plot line chart	134
3. 第2题: Write a SQL query to get the second highest salary from the Employee table	134
4. 第3题: Hive SQL 累积访问次数	135
Oppo	137
1. 需求列表	137
2. 第1题 举例一下Hive中的数据类型?	137
3. 第2题 最高成绩课程&每门课程大于80分的学生	137
4. 第3题 连续三天登录人员	138
附录	168
1. 微众银行	168
2. 华泰证券	171
3. 招商信诺人寿	174
4. 交通银行	177
5. 百丽	182
6. 跨越物流	183
7. 腾讯外包(微信相关)	184
8. 小米电商	184
9. 鄂州机场	185
10. 腾讯游戏数据开发A卷	185
11. 腾讯视频号游戏直播	185
12. 帆软[MySQL方向]	185
13. PayPal	186
14. Oppo	186





大数据必备SQL提高秘籍

提高SQL功底思路。

1、造数据。因为有数据支撑，会方便我们根据数据结果去不断调整SQL的写法。

造数据语法既可以create table再insert into，也可以用下面的create temporary view xx as values语句，更简单。

其中create temporary view xx as values语句，SparkSQL语法支持，hive不支持。

2、先将结果表画出来，包括结果字段名有哪些，数据量也画几条。这是分析他要什么。

从源表到结果表，一路可能要走多个步骤，其实就是可能需要多个子查询，过程多就用with as来重构提高可读性。

3、要由简单过度到复杂，不要一下子就写一个很复杂的。

先写简单的select * from table..每个中间步骤都执行打印结果，看是否符合预期，

根据中间结果，进一步调整修饰SQL语句，再执行，直到接近结果表。

4、数据量要小，工具要快，如果用hive，就设置set hive.exec.mode.local.auto=true;

如果是SparkSQL，就设置合适的shuffle并行度，set spark.sql.shuffle.partitions=4;



初级SQL-Hive10题

1. 需求列表

2. 第1题：访问量统计

2.1 需求

我们有如下的用户访问数据

userId	visitDate	visitCount
u01	2017/1/21	5
u02	2017/1/23	6
u03	2017/1/22	8
u04	2017/1/20	3
u01	2017/1/23	6
u01	2017/2/21	8
U02	2017/1/23	6
U01	2017/2/22	4

要求使用SQL统计出每个用户的累积访问次数，如下表所示：

用户id	月份	小计	累积
u01	2017-01	11	11
u01	2017-02	12	23
u02	2017-01	12	12
u03	2017-01	8	8





u04	2017-01	3	3
-----	---------	---	---

实现

2.2 数据准备

```
CREATE TABLE test_sql.test1 (  
    userId string,  
    visitDate string,  
    visitCount INT )  
  
ROW format delimited FIELDS TERMINATED BY "\t";  
  
INSERT INTO TABLE test_sql.test1  
VALUES  
  
    ('u01', '2017/1/21', 5),  
    ('u02', '2017/1/23', 6),  
    ('u03', '2017/1/22', 8),  
    ('u04', '2017/1/20', 3),  
    ('u01', '2017/1/23', 6),  
    ('u01', '2017/2/21', 8),  
    ('u02', '2017/1/23', 6),  
    ('u01', '2017/2/22', 4);
```

2.3 查询SQL

```
SELECT t2.userid,  
    t2.visitmonth,  
    subtotal_visit_cnt,  
    sum(subtotal_visit_cnt) over (partition BY userid  
                                ORDER BY visitmonth) AS total_visit_cnt  
  
FROM  
  
    (SELECT userid,  
        visitmonth,  
        sum(visitcount) AS subtotal_visit_cnt  
    FROM  
  
        (SELECT userid,  
            date_format(regexp_replace(visitdate, '/', '-'), 'yyyy-MM') AS visitmonth,
```





```
visitcount
FROM test_sql.test1) t1
GROUP BY userid,
visitmonth)t2
ORDER BY t2.userid,
t2.visitmonth
```

3. 第2题：电商场景TopK统计

3.1 需求

有50W个京东店铺，每个顾客访客访问任何一个店铺的任何一个商品时都会产生一条访问日志，访问日志存储的表名为Visit，访客的用户id为user_id，被访问的店铺名称为shop，数据如下：

user_id	shop
u1	a
u2	b
u1	b
u1	a
u3	c
u4	b
u1	a
u2	c
u5	b
u4	b
u6	c
u2	c
u1	b





u2	a
u2	a
u3	a
u5	a
u5	a
u5	a

请统计：

(1) 每个店铺的UV（访客数）

(2) 每个店铺访问次数top3的访客信息。输出店铺名称、访客id、访问次数

实现

3.2 数据准备

```
CREATE TABLE test_sql.test2 (
    user_id string,
    shop string )
ROW format delimited FIELDS TERMINATED BY '\t';
INSERT INTO TABLE test_sql.test2 VALUES
('u1', 'a' ),
('u2', 'b' ),
('u1', 'b' ),
('u1', 'a' ),
('u3', 'c' ),
('u4', 'b' ),
('u1', 'a' ),
('u2', 'c' ),
('u5', 'b' ),
('u4', 'b' ),
('u6', 'c' ),
('u2', 'c' ),
('u1', 'b' ),
('u2', 'a' ),
('u2', 'a' ),
```





```
('u3','a'),  
('u5','a'),  
('u5','a'),  
('u5','a');
```

3.3 查询SQL实现

(1) 方式1:

```
//每个店铺的UV (访客数)  
SELECT shop,count(DISTINCT user_id)  
FROM test_sql.test2  
GROUP BY shop
```

方式2:

```
//每个店铺的UV (访客数)  
SELECT t.shop,count(*)  
FROM  
(SELECT user_id,shop  
FROM test_sql.test2  
GROUP BY user_id,shop) t  
GROUP BY t.shop
```

//每个店铺访问次数top3的访客信息。输出店铺名称、访客id、访问次数

```
SELECT t2.shop,t2.user_id,t2.cnt  
FROM  
(SELECT t1.*,  
row_number() over(partition BY t1.shop ORDER BY t1.cnt DESC) rank  
FROM  
(SELECT user_id,shop,count(*) AS cnt  
FROM test_sql.test2  
GROUP BY user_id,shop) t1)t2  
WHERE rank <= 3
```





4. 第3题：订单量统计

4.1 需求

已知一个表STG.ORDER，有如下字段:Date, Order_id, User_id, amount。

数据样例:2017-01-01,10029028,1000003251,33.57。

请给出sql进行统计:

- (1)给出 2017年每个月的订单数、用户数、总成交金额。
- (2)给出2017年11月的新客数(指在11月才有第一笔订单)

实现

4.2 数据准备

```
CREATE TABLE test_sql.test3 (  
    dt string,  
    order_id string,  
    user_id string,  
    amount DECIMAL ( 10, 2 ) )  
  
ROW format delimited FIELDS TERMINATED BY '\t';  
  
INSERT INTO TABLE test_sql.test3 VALUES ('2017-01-01','10029028','1000003251',33.57);  
INSERT INTO TABLE test_sql.test3 VALUES ('2017-01-01','10029029','1000003251',33.57);  
INSERT INTO TABLE test_sql.test3 VALUES ('2017-01-01','100290288','1000003252',33.57);  
INSERT INTO TABLE test_sql.test3 VALUES ('2017-02-02','10029088','1000003251',33.57);  
INSERT INTO TABLE test_sql.test3 VALUES ('2017-02-02','100290281','1000003251',33.57);  
INSERT INTO TABLE test_sql.test3 VALUES ('2017-02-02','100290282','1000003253',33.57);  
INSERT INTO TABLE test_sql.test3 VALUES ('2017-11-02','10290282','100003253',234);  
INSERT INTO TABLE test_sql.test3 VALUES ('2018-11-02','10290284','100003243',234);
```

4.3 查询SQL

- (1)给出 2017年每个月的订单数、用户数、总成交金额。





```
SELECT t1.mon,
       count(t1.order_id) AS order_cnt,
       count(DISTINCT t1.user_id) AS user_cnt,
       sum(amount) AS total_amount
FROM
  (SELECT order_id,
          user_id,
          amount,
          date_format(dt,'yyy-MM') mon
   FROM test_sql.test3
   WHERE date_format(dt,'yyyy') = '2017') t1
GROUP BY t1.mon
```

(2)给出2017年11月的新客数(指在11月才有第一笔订单)

```
select count(user_id) cnt
from
(select user_id,
       min(date_format(dt,'yyy-MM')) min_month
 from test3 group by user_id
 having min(date_format(dt,'yyy-MM'))='2017-11');
```

5. 第4题：大数据排序统计

5.1 需求

有一个5000万的用户文件(user_id, name, age)，一个2亿记录的用户看电影的记录文件(user_id, url)，根据年龄段观看电影的次数进行排序？

实现

5.2 数据准备

```
CREATE TABLE test_sql.test4user
  (user_id string,name string,age int);

CREATE TABLE test_sql.test4log
  (user_id string,url string);

INSERT INTO TABLE test_sql.test4user VALUES('001','u1',10);
```





```
INSERT INTO TABLE test_sql.test4user VALUES('002','u2',15);
INSERT INTO TABLE test_sql.test4user VALUES('003','u3',15);
INSERT INTO TABLE test_sql.test4user VALUES('004','u4',20);
INSERT INTO TABLE test_sql.test4user VALUES('005','u5',25);
INSERT INTO TABLE test_sql.test4user VALUES('006','u6',35);
INSERT INTO TABLE test_sql.test4user VALUES('007','u7',40);
INSERT INTO TABLE test_sql.test4user VALUES('008','u8',45);
INSERT INTO TABLE test_sql.test4user VALUES('009','u9',50);
INSERT INTO TABLE test_sql.test4user VALUES('0010','u10',65);
INSERT INTO TABLE test_sql.test4log VALUES('001','url1');
INSERT INTO TABLE test_sql.test4log VALUES('002','url1');
INSERT INTO TABLE test_sql.test4log VALUES('003','url2');
INSERT INTO TABLE test_sql.test4log VALUES('004','url3');
INSERT INTO TABLE test_sql.test4log VALUES('005','url3');
INSERT INTO TABLE test_sql.test4log VALUES('006','url1');
INSERT INTO TABLE test_sql.test4log VALUES('007','url5');
INSERT INTO TABLE test_sql.test4log VALUES('008','url7');
INSERT INTO TABLE test_sql.test4log VALUES('009','url5');
INSERT INTO TABLE test_sql.test4log VALUES('0010','url1');
```

5.3 查询SQL

```
SELECT
t2.age_phase,
sum(t1.cnt) as view_cnt
FROM
(SELECT user_id,
count(*) cnt
FROM test_sql.test4log
GROUP BY user_id) t1
JOIN(SELECT user_id,
CASE WHEN age <= 10 AND age > 0 THEN '0-10'
WHEN age <= 20 AND age > 10 THEN '10-20'
WHEN age >20 AND age <=30 THEN '20-30'
WHEN age >30 AND age <=40 THEN '30-40'
WHEN age >40 AND age <=50 THEN '40-50'
```



```
WHEN age >50 AND age <=60 THEN '50-60'
WHEN age >60 AND age <=70 THEN '60-70'
ELSE '70以上' END as age_phase
FROM test_sql.test4user) t2 ON t1.user_id = t2.user_id
GROUP BY t2.age_phase

--方式2
select age_type,
       sum(cnt) sum1
from (select user_id, count(url) cnt from test4log group by user_id) t1
join (select *,
          concat(floor(age/10)*10,'-',(floor(age/10)+1)*10) age_type
      from test4user) t2
on t1.user_id = t2.user_id
group by age_type
order by sum(cnt) desc;
```

6. 第5题：活跃用户统计

6.1 需求

有日志如下，请写出代码求得所有用户和活跃用户的总数及平均年龄。（活跃用户指连续两天都有访问记录的用户）

```
日期 用户 年龄
2019-02-11,test_1,23
2019-02-11,test_2,19
2019-02-11,test_3,39
2019-02-11,test_1,23
2019-02-11,test_3,39
2019-02-11,test_1,23
2019-02-12,test_2,19
2019-02-13,test_1,23
2019-02-15,test_2,19
2019-02-16,test_2,19
```

实现

6.2 数据准备





```
CREATE TABLE test5(  
dt string,  
user_id string,  
age int)  
ROW format delimited fields terminated BY ',';  
INSERT INTO TABLE test_sql.test5 VALUES ('2019-02-11','test_1',23);  
INSERT INTO TABLE test_sql.test5 VALUES ('2019-02-11','test_2',19);  
INSERT INTO TABLE test_sql.test5 VALUES ('2019-02-11','test_3',39);  
INSERT INTO TABLE test_sql.test5 VALUES ('2019-02-11','test_1',23);  
INSERT INTO TABLE test_sql.test5 VALUES ('2019-02-11','test_3',39);  
INSERT INTO TABLE test_sql.test5 VALUES ('2019-02-11','test_1',23);  
INSERT INTO TABLE test_sql.test5 VALUES ('2019-02-12','test_2',19);  
INSERT INTO TABLE test_sql.test5 VALUES ('2019-02-13','test_1',23);  
INSERT INTO TABLE test_sql.test5 VALUES ('2019-02-15','test_2',19);  
INSERT INTO TABLE test_sql.test5 VALUES ('2019-02-16','test_2',19);
```

6.3 查询SQL

```
SELECT sum(total_user_cnt) total_user_cnt,  
       sum(total_user_avg_age) total_user_avg_age,  
       sum(two_days_cnt) two_days_cnt,  
       sum(avg_age) avg_age  
FROM  
(SELECT 0 total_user_cnt,  
        0 total_user_avg_age,  
        count(*) AS two_days_cnt,  
        cast(sum(age) / count(*) AS decimal(5,2)) AS avg_age  
FROM  
(SELECT user_id,max(age) age  
FROM  
(SELECT user_id,max(age) age  
FROM  
(SELECT user_id,age,date_sub(dt,rank) flag  
FROM  
(SELECT dt,user_id,max(age) age,  
row_number() over(PARTITION BY user_id  
ORDER BY dt) rank
```



```
FROM test_sql.test5
GROUP BY dt,user_id) t1) t2
GROUP BY user_id,flag
HAVING count(*) >=2) t3
GROUP BY user_id) t4
UNION ALL SELECT count(*) total_user_cnt,
cast(sum(age) /count(*) AS decimal(5,2)) total_user_avg_age,
0 two_days_cnt,
0 avg_age
FROM
(SELECT user_id,
max(age) age
FROM test_sql.test5
GROUP BY user_id) t5) t6
```

--方案2

--用with as来重构上述代码，使步骤清晰

with t0 as (select count(user_id) cnt, avg(age) avg_age from (select distinct user_id, age from test5)),

--下面t1到t4都是处理活跃用户

t1 as (select *, row_number() over (partition by user_id order by dt) rn

from (select distinct * from test5)

order by user_id, dt),

t2 as (select *, date_sub(dt, rn) date2 from t1),

t3 as (select user_id, date2, max(age) age, count(*) cnt from t2 group by user_id, date2 having count(*) >= 2),

t4 as (select distinct user_id, age from t3)

select '所有用户' type,cnt,avg_age from t0 union all

select '活跃用户', count(user_id) , avg(age) from t4;

7. 第6题：电商购买金额统计实战

7.1 需求

请用sql写出所有用户中在今年10月份第一次购买商品的金额，

表ordertable字段: (购买用户: userid, 金额: money, 购买时间: paymenttime(格式: 2017-10-01), 订单id: orderid)

实现





7.2 数据准备

```
CREATE TABLE test_sql.test6 (  
    userid string,  
    money decimal(10,2),  
    paymenttime string,  
    orderid string);  
  
INSERT INTO TABLE test_sql.test6 VALUES('001',100,'2017-10-01','123');  
INSERT INTO TABLE test_sql.test6 VALUES('001',200,'2017-10-02','124');  
INSERT INTO TABLE test_sql.test6 VALUES('002',500,'2017-10-01','125');  
INSERT INTO TABLE test_sql.test6 VALUES('001',100,'2017-11-01','126');
```

7.3 查询SQL

```
SELECT  
    userid,  
    paymenttime,  
    money,  
    orderid  
from(SELECT userid,  
    money,  
    paymenttime,  
    orderid,  
    row_number() over (PARTITION BY userid  
        ORDER BY paymenttime) rank  
FROM test_sql.test6  
WHERE date_format(paymenttime,'yyyy-MM') = '2017-10') t  
WHERE rank = 1
```

8. 第7题：教育领域SQL实战

8.1 需求

现有图书管理数据库的三个数据模型如下：

图书（数据表名：BOOK）





序号	字段名称	字段描述	字段类型
1	BOOK_ID	总编号	文本
2	SORT	分类号	文本
3	BOOK_NAME	书名	文本
4	WRITER	作者	文本
5	OUTPUT	出版单位	文本
6	PRICE	单价	数值（保留小数点后2位）

读者（数据表名：READER）

序号	字段名称	字段描述	字段类型
1	READER_ID	借书证号	文本
2	COMPANY	单位	文本
3	NAME	姓名	文本
4	SEX	性别	文本
5	GRADE	职称	文本
6	ADDR	地址	文本

借阅记录（数据表名：BORROW LOG）

序号	字段名称	字段描述	字段类型
1	READER_ID	借书证号	文本
2	BOOK_ID	总编号	文本
3	BORROW_DATE	借书日期	日期

(1) 创建图书管理库的图书、读者和借阅三个基本表的表结构。请写出建表语句。

(2) 找出姓李的读者姓名（NAME）和所在单位（COMPANY）。



- (3) 查找“高等教育出版社”的所有图书名称 (BOOK_NAME) 及单价 (PRICE)，结果按单价降序排序。
 - (4) 查找价格介于10元和20元之间的图书种类(SORT) 出版单位 (OUTPUT) 和单价 (PRICE)，结果按出版单位 (OUTPUT) 和单价 (PRICE) 升序排序。
 - (5) 查找所有借了书的读者的姓名 (NAME) 及所在单位 (COMPANY)。
 - (6) 求“科学出版社”图书的最高单价、最低单价、平均单价。
 - (7) 找出当前至少借阅了2本图书 (大于等于2本) 的读者姓名及其所在单位。
 - (8) 考虑到数据安全的需要，需定时将“借阅记录”中数据进行备份，请使用一条SQL语句，在备份用户bak下创建与“借阅记录”表结构完全一致的数据表BORROW_LOG_BAK并且将“借阅记录”中现有数据全部复制到BORROW_LOG_BAK中。
 - (9) 现在需要将原Oracle数据库中数据迁移至Hive仓库，请写出“图书”在Hive中的建表语句 (Hive实现，提示：列分隔符|；数据表数据需要外部导入：分区分别以month_part、day_part 命名)
 - (10) Hive中有表A，现在需要将表A的月分区 201505 中 user_id为20000的user_dinner字段更新为 bonc8920，其他用户user_dinner字段数据不变，请列出更新的方法步骤。(Hive实现，提示：Hive中无update语法，请通过其他办法进行数据更新)
- 实现

8.2 数据准备

(1)-- 创建图书表book

```
CREATE TABLE test_sql.book(book_id string,
                             `SORT` string,
                             book_name string,
                             writer string,
                             OUTPUT string,
                             price decimal(10,2));
INSERT INTO TABLE test_sql.book VALUES ('001','TP391','信息处理','author1','机械工业出版社','20');
INSERT INTO TABLE test_sql.book VALUES ('002','TP392','数据库','author12','科学出版社','15');
INSERT INTO TABLE test_sql.book VALUES ('003','TP393','计算机网络','author3','机械工业出版社','29');
INSERT INTO TABLE test_sql.book VALUES ('004','TP399','微机原理','author4','科学出版社','39');
INSERT INTO TABLE test_sql.book VALUES ('005','C931','管理信息系统','author5','机械工业出版社','40');
INSERT INTO TABLE test_sql.book VALUES ('006','C932','运筹学','author6','科学出版社','55');
```

-- 创建读者表reader

```
CREATE TABLE test_sql.reader (reader_id string,
                                company string,
                                name string,
```





```
sex string,  
grade string,  
addr string);  
  
INSERT INTO TABLE test_sql.reader VALUES ('0001','阿里巴巴','jack','男','vp','addr1');  
INSERT INTO TABLE test_sql.reader VALUES ('0002','百度','robin','男','vp','addr2');  
INSERT INTO TABLE test_sql.reader VALUES ('0003','腾讯','tony','男','vp','addr3');  
INSERT INTO TABLE test_sql.reader VALUES ('0004','京东','jasper','男','cfo','addr4');  
INSERT INTO TABLE test_sql.reader VALUES ('0005','网易','zhangsan','女','ceo','addr5');  
INSERT INTO TABLE test_sql.reader VALUES ('0006','搜狐','lisi','女','ceo','addr6');
```

-- 创建借阅记录表borrow_log

```
CREATE TABLE test_sql.borrow_log(reader_id string,  
book_id string,  
borrow_date string);  
  
INSERT INTO TABLE test_sql.borrow_log VALUES ('0001','002','2019-10-14');  
INSERT INTO TABLE test_sql.borrow_log VALUES ('0002','001','2019-10-13');  
INSERT INTO TABLE test_sql.borrow_log VALUES ('0003','005','2019-09-14');  
INSERT INTO TABLE test_sql.borrow_log VALUES ('0004','006','2019-08-15');  
INSERT INTO TABLE test_sql.borrow_log VALUES ('0005','003','2019-10-10');  
INSERT INTO TABLE test_sql.borrow_log VALUES ('0006','004','2019-17-13');
```

8.3 查询SQL

(2)找出姓李的读者姓名 (NAME) 和所在单位 (COMPANY) 。

```
SELECT name,  
company  
FROM test_sql.reader  
WHERE name LIKE '李%';
```

(3)查找“高等教育出版社”的所有图书名称 (BOOK_NAME) 及单价 (PRICE) ， 结果按单价降序排序。

```
SELECT book_name,  
price  
FROM test_sql.book  
WHERE OUTPUT = "高等教育出版社"  
ORDER BY price DESC;
```





(4)查找价格介于10元和20元之间的图书种类(SORT) 出版单位 (OUTPUT) 和单价 (PRICE) , 结果按出版单位 (OUTPUT) 和单价 (PRICE) 升序排序。

```
SELECT sort,  
        output,  
        price  
FROM test_sql.book  
WHERE price >= 10 and price <= 20  
ORDER BY output,price ;
```

(5)查找所有借了书的读者的姓名 (NAME) 及所在单位 (COMPANY) 。

```
SELECT b.name,  
        b.company  
FROM test_sql.borrow_log a  
JOIN test_sql.reader b ON a.reader_id = b.reader_id;
```

(6)求“科学出版社”图书的最高单价、最低单价、平均单价。

```
SELECT max(price),  
        min(price),  
        avg(price)  
FROM test_sql.book  
WHERE OUTPUT = '科学出版社';
```

(7)找出当前至少借阅了2本图书 (大于等于2本) 的读者姓名及其所在单位。

```
SELECT b.name,  
        b.company  
FROM  
        (SELECT reader_id  
         FROM test_sql.borrow_log  
         GROUP BY reader_id  
         HAVING count(*) >= 2) a  
JOIN test_sql.reader b ON a.reader_id = b.reader_id;
```

(8)考虑到数据安全的需要,需定时将“借阅记录”中数据进行备份,请使用一条SQL语句,在备份用户bak下创建与“借阅记录”表结构完全一致的数据表BORROW_LOG_BAK.并且将“借阅记录”中现有数据全部复制到BORROW_LOG_BAK中。

```
CREATE TABLE test_sql.borrow_log_bak AS  
SELECT *  
FROM test_sql.borrow_log;
```





(9)现在需要将原Oracle数据库中数据迁移至Hive仓库，请写出“图书”在Hive中的建表语句（Hive实现，提示：列分隔符|；数据表数据需要外部导入：分区分别以month_part、day_part命名）

```
CREATE TABLE book_hive (  
    book_id string,  
    SORT string,  
    book_name string,  
    writer string,  
    OUTPUT string,  
    price DECIMAL ( 10, 2 )  
    partitioned BY ( month_part string, day_part string )  
    ROW format delimited FIELDS TERMINATED BY '|' stored AS textfile;
```

(10)Hive中有表A，现在需要将表A的月分区 201505 中 user_id为20000的user_dinner字段更新为bonc8920，其他用户user_dinner字段数据不变，请列出更新的方法步骤。（Hive实现，提示：Hive中无update语法，请通过其他办法进行数据更新）

方式1：配置hive支持事务操作，分桶表，orc存储格式

方式2：第一步找到要更新的数据，将要更改的字段替换为新的值，第二步找到不需要更新的数据，第三步将上两步的数据插入一张新表中。

--临时视图Spark支持，hive不支持。

create or replace temporary view A1 (user_id,user_dinner) as values

(20000,'bonc8888'),

(10000,'bonc7777'),

(30000,'bonc9999');

select * from A1;

create table A as select * from A1;

select * from A;

insert overwrite table A

select user_id,'bonc8920' as user_dinner from A where user_id=20000

union all

select * from A where user_id!=20000

9. 第8题：服务日志SQL统计

9.1 需求

有一个线上服务器访问日志格式如下（用sql答题）





时间	接口	ip地址
2016/11/9 14:22	/api/user/login	110.23.5.33
2016/11/9 14:23	/api/user/detail	57.3.2.16
2016/11/9 15:59	/api/user/login	200.6.5.166
...	...	...

求11月9号下午14点（14-15点），访问/api/user/login接口的top10的ip地址实现

9.2 数据准备

```
CREATE TABLE test_sql.test8(`date` string,  
                             `interface` string,  
                             `ip` string);  
  
INSERT INTO TABLE test_sql.test8 VALUES ('2016-11-09 11:22:05','/api/user/login','110.23.5.23');  
INSERT INTO TABLE test_sql.test8 VALUES ('2016-11-09 11:23:10','/api/user/detail','57.3.2.16');  
INSERT INTO TABLE test_sql.test8 VALUES ('2016-11-09 23:59:40','/api/user/login','200.6.5.166');  
INSERT INTO TABLE test_sql.test8 VALUES ('2016-11-09 11:14:23','/api/user/login','136.79.47.70');  
INSERT INTO TABLE test_sql.test8 VALUES ('2016-11-09 11:15:23','/api/user/detail','94.144.143.141');  
INSERT INTO TABLE test_sql.test8 VALUES ('2016-11-09 11:16:23','/api/user/login','197.161.8.206');  
INSERT INTO TABLE test_sql.test8 VALUES ('2016-11-09 12:14:23','/api/user/detail','240.227.107.145');  
INSERT INTO TABLE test_sql.test8 VALUES ('2016-11-09 13:14:23','/api/user/login','79.130.122.205');  
INSERT INTO TABLE test_sql.test8 VALUES ('2016-11-09 14:14:23','/api/user/detail','65.228.251.189');  
INSERT INTO TABLE test_sql.test8 VALUES ('2016-11-09 14:15:23','/api/user/detail','245.23.122.44');  
INSERT INTO TABLE test_sql.test8 VALUES ('2016-11-09 14:17:23','/api/user/detail','22.74.142.137');  
INSERT INTO TABLE test_sql.test8 VALUES ('2016-11-09 14:19:23','/api/user/detail','54.93.212.87');  
INSERT INTO TABLE test_sql.test8 VALUES ('2016-11-09 14:20:23','/api/user/detail','218.15.167.248');  
INSERT INTO TABLE test_sql.test8 VALUES ('2016-11-09 14:24:23','/api/user/detail','20.117.19.75');  
INSERT INTO TABLE test_sql.test8 VALUES ('2016-11-09 15:14:23','/api/user/login','183.162.66.97');  
INSERT INTO TABLE test_sql.test8 VALUES ('2016-11-09 16:14:23','/api/user/login','108.181.245.147');  
INSERT INTO TABLE test_sql.test8 VALUES ('2016-11-09 14:17:23','/api/user/login','22.74.142.137');  
INSERT INTO TABLE test_sql.test8 VALUES ('2016-11-09 14:19:23','/api/user/login','22.74.142.137');
```





9.3 查询SQL

求11月9号下午14点（14-15点），访问/api/user/login接口的top10的ip地址

```
SELECT ip,count(*) AS cnt
FROM test_sql.test8
WHERE date_format(date,'yyyy-MM-dd HH') >= '2016-11-09 14'
      AND date_format(date,'yyyy-MM-dd HH') < '2016-11-09 15'
      AND interface='/api/user/login'
GROUP BY ip
ORDER BY cnt desc
LIMIT 10;
```

10. 第9题：充值日志SQL实战

10.1 需求

有一个充值日志表credit_log，字段如下：

```
`dist_id` int    '区组id',
`account` string '账号',
`money` int      '充值金额',
`create_time` string '订单时间'
```

请写出SQL语句，查询充值日志表2019年01月02号每个区组下充值额最大的账号，要求结果：

区组id，账号，金额，充值时间

实现：

10.2 数据准备

```
CREATE TABLE test_sql.test9(
    dist_id string COMMENT '区组id',
    account string COMMENT '账号',
    `money` decimal(10,2) COMMENT '充值金额',
    create_time string COMMENT '订单时间');
```





```
INSERT INTO TABLE test_sql.test9 VALUES ('1','11',100006,'2019-01-02 13:00:01');
INSERT INTO TABLE test_sql.test9 VALUES ('1','22',110000,'2019-01-02 13:00:02');
INSERT INTO TABLE test_sql.test9 VALUES ('1','33',102000,'2019-01-02 13:00:03');
INSERT INTO TABLE test_sql.test9 VALUES ('1','44',100300,'2019-01-02 13:00:04');
INSERT INTO TABLE test_sql.test9 VALUES ('1','55',100040,'2019-01-02 13:00:05');
INSERT INTO TABLE test_sql.test9 VALUES ('1','66',100005,'2019-01-02 13:00:06');
INSERT INTO TABLE test_sql.test9 VALUES ('1','77',180000,'2019-01-03 13:00:07');
INSERT INTO TABLE test_sql.test9 VALUES ('1','88',106000,'2019-01-02 13:00:08');
INSERT INTO TABLE test_sql.test9 VALUES ('1','99',100400,'2019-01-02 13:00:09');
INSERT INTO TABLE test_sql.test9 VALUES ('1','12',100030,'2019-01-02 13:00:10');
INSERT INTO TABLE test_sql.test9 VALUES ('1','13',100003,'2019-01-02 13:00:20');
INSERT INTO TABLE test_sql.test9 VALUES ('1','14',100020,'2019-01-02 13:00:30');
INSERT INTO TABLE test_sql.test9 VALUES ('1','15',100500,'2019-01-02 13:00:40');
INSERT INTO TABLE test_sql.test9 VALUES ('1','16',106000,'2019-01-02 13:00:50');
INSERT INTO TABLE test_sql.test9 VALUES ('1','17',100800,'2019-01-02 13:00:59');
INSERT INTO TABLE test_sql.test9 VALUES ('2','18',100800,'2019-01-02 13:00:11');
INSERT INTO TABLE test_sql.test9 VALUES ('2','19',100030,'2019-01-02 13:00:12');
INSERT INTO TABLE test_sql.test9 VALUES ('2','10',100000,'2019-01-02 13:00:13');
INSERT INTO TABLE test_sql.test9 VALUES ('2','45',100010,'2019-01-02 13:00:14');
INSERT INTO TABLE test_sql.test9 VALUES ('2','78',100070,'2019-01-02 13:00:15');
```

10.3 查询SQL

```
WITH TEMP AS
(SELECT dist_id,account,sum('money') sum_money
FROM test_sql.test9
WHERE date_format(create_time,'yyyy-MM-dd') = '2019-01-02'
GROUP BY dist_id,account)

SELECT t1.dist_id,t1.account,t1.sum_money
FROM
(SELECT temp.dist_id,
temp.account,
temp.sum_money,
rank() over(partition BY temp.dist_id
ORDER BY temp.sum_money DESC) ranks
FROM TEMP) t1
```





```
WHERE ranks = 1
```

11. 第10题：电商分组TopK实战

11.1 需求

有一个账号表如下，请写出SQL语句，查询各自区组的money排名前十的账号（分组取前10）

dist_id string '区组id',

account string '账号',

gold int '金币'

实现

11.2 数据准备

```
CREATE TABLE test_sql.test10(  
    'dist_id' string COMMENT '区组id',  
    'account' string COMMENT '账号',  
    'gold' int COMMENT '金币');  
  
INSERT INTO TABLE test_sql.test10 VALUES ('1','77',18);  
INSERT INTO TABLE test_sql.test10 VALUES ('1','88',106);  
INSERT INTO TABLE test_sql.test10 VALUES ('1','99',10);  
INSERT INTO TABLE test_sql.test10 VALUES ('1','12',13);  
INSERT INTO TABLE test_sql.test10 VALUES ('1','13',14);  
INSERT INTO TABLE test_sql.test10 VALUES ('1','14',25);  
INSERT INTO TABLE test_sql.test10 VALUES ('1','15',36);  
INSERT INTO TABLE test_sql.test10 VALUES ('1','16',12);  
INSERT INTO TABLE test_sql.test10 VALUES ('1','17',158);  
INSERT INTO TABLE test_sql.test10 VALUES ('2','18',12);  
INSERT INTO TABLE test_sql.test10 VALUES ('2','19',44);  
INSERT INTO TABLE test_sql.test10 VALUES ('2','10',66);  
INSERT INTO TABLE test_sql.test10 VALUES ('2','45',80);  
INSERT INTO TABLE test_sql.test10 VALUES ('2','78',98);
```





11.3 查询SQL

```
SELECT dist_id,  
       account,  
       gold  
FROM (SELECT dist_id,  
            account,  
            gold,  
            row_number () over (PARTITION BY dist_id  
                                ORDER BY gold DESC) rank  
FROM test_sql.test10) t  
WHERE rank <= 10
```



微众银行

1. 需求列表

需求1：授信金额统计

需求2：贷款产品不良统计

需求3：逾期率统计

需求4：用户画像统计

需求5：Vintage账龄分析

2. 源数据表：

1.核额流水表

字段名	字段意义	字段类型
ds	日期分区，样例格式为20200101，每个分区有全量流水	string
sno	每个ds内主键，流水号	string
uid	户id	string
is_risk_apply	是否核额申请（核额漏斗第一步）取值0和1	bigint
is_pass_rule	是否通过规则（核额漏斗第二步）取值0和1	bigint
is_obtain_quota	是否授信成功（核额漏斗第三步）取值0和1	bigint
quota	授信金额	decimal(30,6)
update_time	更新时间样例格式为2020-11-14 08:12:12	string

2.借据表

字段名	字段意义	字段类型
ds	日期分区，样例格式为20200101每个分区有全量借据	string
duebilid	借据号（每个日期分区内的主键）	string



uid	用户id	string
prod_type	产品名称仅3个枚举值XX贷YY贷ZZ贷	string
putout_date	发放日期样例格式为2020-10-10 00:10:30	bigint
putout_amt	发放金额	decimal (30,6)
balance	借据余额	decimal (30,6)
is_buliang	状态-是否不良取值0和1	bigint
overduedays	逾期天数	bigint

3. 模型输出表

字段名	字段意义	字段类型
ds	日期分区，样例格式为20200101增量表部分流水记录可能有更新	strng
sno	流水号，主键	strng
create_time	创建日期样例格式为2020-10-10 00:10:30与sno唯一绑定，不会变更	strng
uid	用户id	strng
content	son格式key值名称为V01~V06，value值取值为0和1	strng
create_time	更新日期样例格式为2020-10-1000:10:30	strng

3. 创建库和源数据表，并生成数据。

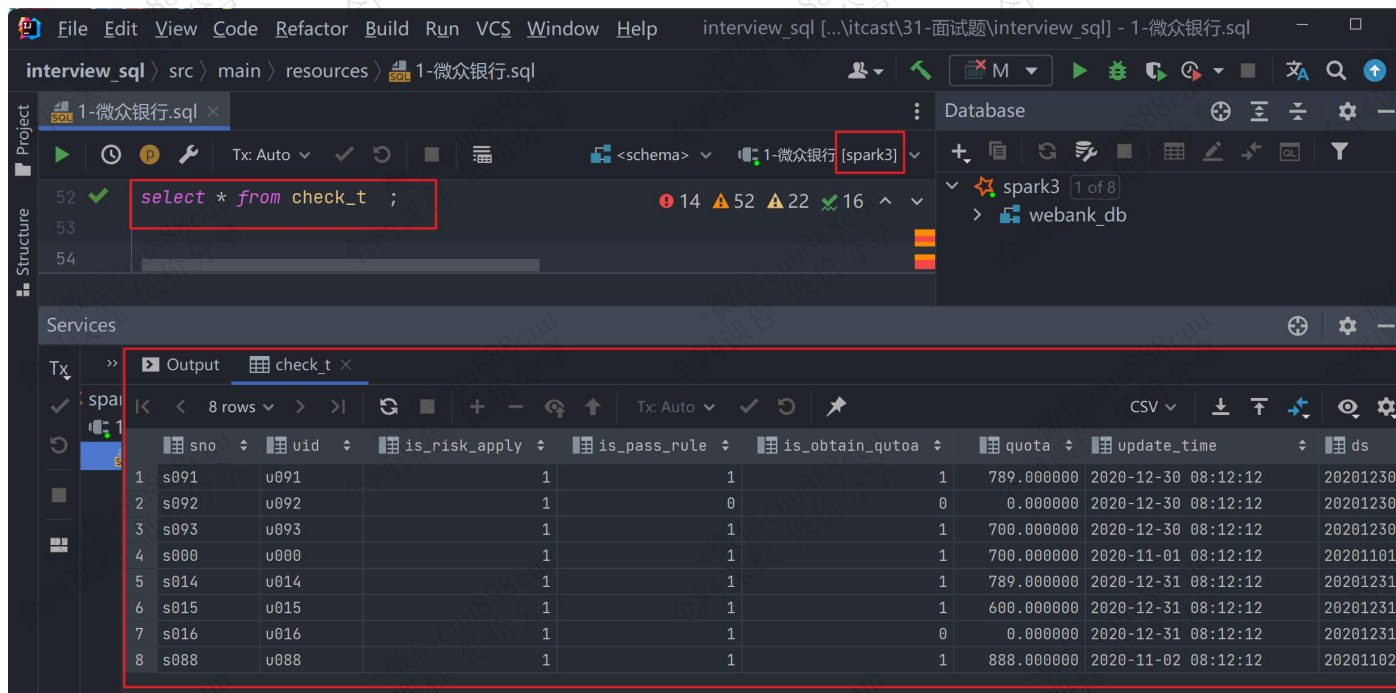
```
set spark.sql.shuffle.partitions=4;
create database webank_db;
use webank_db;
create or replace temporary view check_view
(ds comment '日期分区',
 sno comment '流水号',
 uid comment '用户id',
 is_risk_apply comment '是否核额申请',
 is_pass_rule comment '是否通过规则',
 is_obtain_qutoa comment '是否授信成功',
```





```
        quota comment '授信金额',
        update_time comment '更新时间')
as
values ('20201101', 's000', 'u000', 1, 1, 1, 700, '2020-11-01 08:12:12'),
        ('20201102', 's088', 'u088', 1, 1, 1, 888, '2020-11-02 08:12:12'),
        ('20201230', 's091', 'u091', 1, 1, 1, 789, '2020-12-30 08:12:12'),
        ('20201230', 's092', 'u092', 1, 0, 0, 0, '2020-12-30 08:12:12'),
        ('20201230', 's093', 'u093', 1, 1, 1, 700, '2020-12-30 08:12:12'),
        ('20201231', 's094', 'u094', 1, 1, 1, 789, '2020-12-31 08:12:12'),
        ('20201231', 's095', 'u095', 1, 1, 1, 600, '2020-12-31 08:12:12'),
        ('20201231', 's096', 'u096', 1, 1, 0, 0, '2020-12-31 08:12:12')
;
--创建核额流水表
drop table if exists check_t;
create table check_t (
        sno string comment '流水号',
        uid string,
        is_risk_apply bigint,
        is_pass_rule bigint,
        is_obtain_quota bigint,
        quota decimal(30,6),
        update_time string
) partitioned by (ds string comment '日期分区');
--动态分区需要设置
set hive.exec.dynamic.partition=true;
set hive.exec.dynamic.partition.mode=nonstrict;
insert overwrite table check_t partition (ds)
select sno,
        uid,
        is_risk_apply,
        is_pass_rule,
        is_obtain_quota,
        quota,
        update_time,
        ds
from check_view;
```





--创建借据表

```
create table debt(
    duebill_id string comment '借据号',
    uid string,
    prod_type string,
    putout_date string,
    putout_amt decimal(30, 6),
    balance decimal(30, 6),
    is_buliang int,
    overduedays int
)
```

)partitioned by (ds string comment '日期分区');

--资料提供了一个34899条借据数据的文件

--下面补充如何将文件的数据导入到分区表中。需要一个中间普通表过渡。

```
drop table if exists webank_db.debt_temp;
```

```
create table webank_db.debt_temp(
    duebill_id string comment '借据号',
    uid string,
    prod_type string,
    putout_date string,
    putout_amt decimal(30, 6),
```



```
balance    decimal(30, 6),
is_buliang int,
overduedays int,
ds string comment '日期分区'
) row format delimited fields terminated by '\t';
load data local inpath '/root/debt.txt' overwrite into table webank_db.debt_temp;

--动态分区需要设置
set hive.exec.dynamic.partition=true;
set hive.exec.dynamic.partition.mode=nonstrict;
insert overwrite table webank_db.debt partition (ds)
select * from webank_db.debt_temp;
```

```
--技巧：如果查询debt表，由于分区数太多，导致查询很慢。
-- 开发阶段，我们可以事先将表缓存起来，并且降低分区数比如为6，那么查缓存表
大大提升了开发效率。
-- 上线阶段，再用实际表替换缓存表。
-- 首次缓存会耗时慢
cache table cache_debt as select /*+ coalesce(6) */ * from debt;
-- 第二次使用缓存会很快
select count(*) from cache_debt;
select ds,count(1) from cache_debt group by ds;
```

先了解表数据的分布情况，有2年多，每天都有分区，共760多个分区。





```
show partitions debt;
```

partition
1 ds=2019-10-01
2 ds=2019-10-02
3 ds=2019-10-03
4 ds=2019-10-04
5 ds=2019-10-05
6 ds=2019-10-06
7 ds=2019-10-07
8 ds=2019-10-08
9 ds=2019-10-09
10 ds=2019-10-10
11 ds=2019-10-11
12 ds=2019-10-12

```
> cache table cache_debt as select /* coalesce(6) */ * from debt;
[2021-12-24 14:03:47] 0 rows retrieved in 21 s 211 ms (execution: 21 s 180 ms, fetching: 31 ms)
> select count(*) from cache_debt
[2021-12-24 14:04:28] 1 row retrieved starting from 1 in 725 ms (execution: 691 ms, fetching: 34 ms)
```

随机观察2个借据的情况

```
select * from cache_debt where duebill_id='u001-1' order by ds;
select * from cache_debt where duebill_id='u005-2' order by ds;
```



-- 随机观察2个借据的情况

```
select * from cache_debt where duebill_id='u001-1' order by ds;
```

Output cache_debt ×

366 rows

	duebill_id	uid	prod_type	putout_date	putout_amt	balance	is_buliang	overduedays	ds
1	u001-1	u001	xx	2020-09-01	1200.000000	1200.000000	0	0	2020-09-01
2	u001-1	u001	xx	2020-09-01	1200.000000	1200.000000	0	0	2020-09-02
3	u001-1	u001	xx	2020-09-01	1200.000000	1200.000000	0	0	2020-09-03
4	u001-1	u001	xx	2020-09-01	1200.000000	1200.000000	0	0	2020-09-04
5	u001-1	u001	xx	2020-09-01	1200.000000	1200.000000	0	0	2020-09-05
同一笔借据 中间省略每天的余额信息 日期分区不同									
361	u001-1	u001	xx	2020-09-01	1200.000000	100.000000	0	0	2021-08-27
362	u001-1	u001	xx	2020-09-01	1200.000000	100.000000	0	0	2021-08-28
363	u001-1	u001	xx	2020-09-01	1200.000000	100.000000	0	0	2021-08-29
364	u001-1	u001	xx	2020-09-01	1200.000000	100.000000	0	0	2021-08-30
365	u001-1	u001	xx	2020-09-01	1200.000000	100.000000	0	0	2021-08-31
366	u001-1	u001	xx	2020-09-01	1200.000000	0.000000	0	0	2021-09-01

```
select * from cache_debt where duebill_id='u005-2' order by ds;
```

Output cache_debt ×

363 rows

	duebill_id	uid	prod_type	putout_date	putout_amt	balance	is_buliang	overduedays	ds
1	u005-2	u005	yy	2020-11-01	1200.000000	1200.000000	0	0	2020-11-01
2	u005-2	u005	yy	2020-11-01	1200.000000	1200.000000	0	0	2020-11-02
3	u005-2	u005	yy	2020-11-01	1200.000000	1200.000000	0	0	2020-11-03
4	u005-2	u005	yy	2020-11-01	1200.000000	1200.000000	0	0	2020-11-04
不良 逾期96天									
359	u005-2	u005	yy	2020-11-01	1200.000000	100.000000	1	96	2021-10-25
360	u005-2	u005	yy	2020-11-01	1200.000000	100.000000	1	97	2021-10-26
361	u005-2	u005	yy	2020-11-01	1200.000000	100.000000	1	98	2021-10-27
362	u005-2	u005	yy	2020-11-01	1200.000000	100.000000	1	99	2021-10-28
363	u005-2	u005	yy	2020-11-01	1200.000000	100.000000	1	100	2021-10-29

4. 第1题：授信金额统计

笔试题目

说明：SCQL语法请使用HiveSQL/SparksQL

1.基于录1《核额流水表》和附录2《借据表》统计下述指标，请提供统计SQL

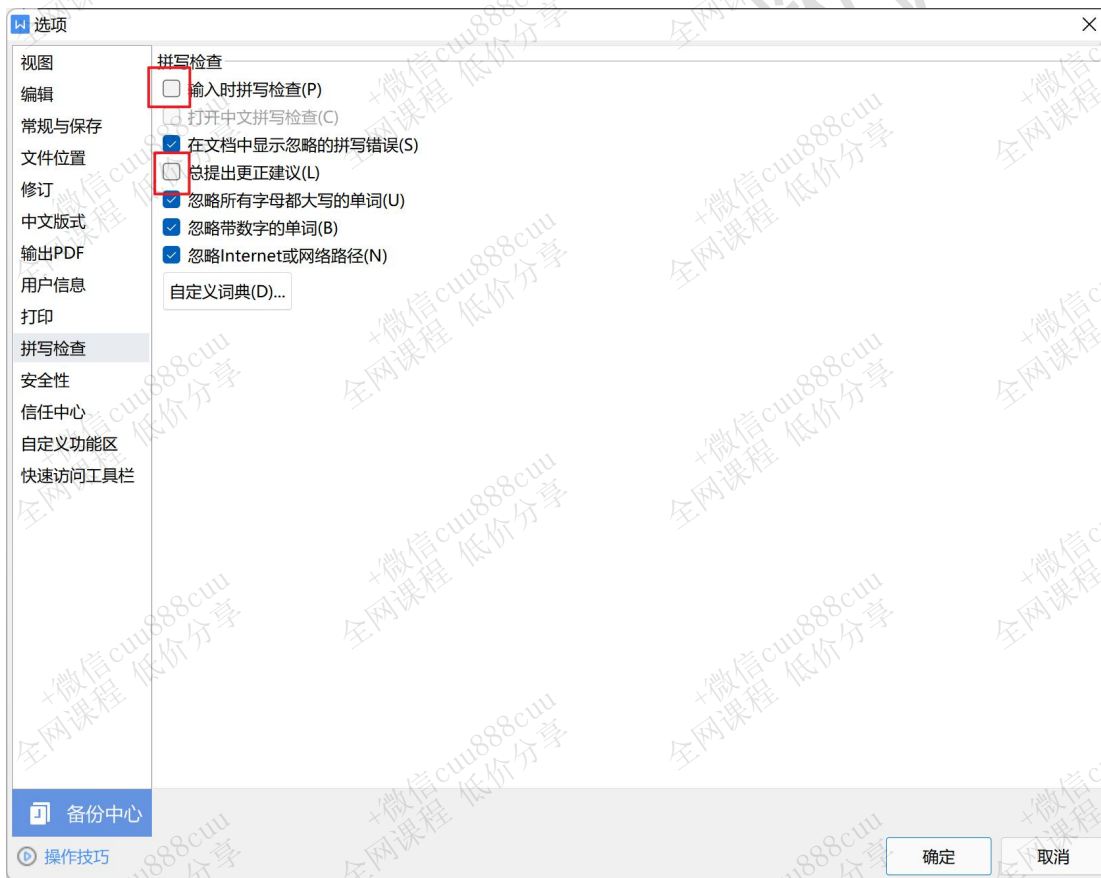
指标	当日新增	昨日新增	历史累计
申请户数			
规则通过户数			



核额成功户数			
授信金额			
平均核额			
发放金额			
户均发放金额			

建议用【Word-视图-Web版式视图】查看代码。

并且观看时不要开启单词拼写检查，否则大量下划线，如下在WPS中【文件-选线-拼写检查】关闭。



-- 第一问 方案1

-- 假设当天是20201231，昨日是20201230

-- 预先将复用分数数据集缓存起来，只用加载一次源表。后面多次union all起来。cache table仅Spark支持，hive不支持。



```
drop table if exists t1;
cache table t1 as (
    select count_if(ds = '20201231') as today_apply_cnt,
    count_if(ds = '20201231' and is_pass_rule = 1) as today_pass_cnt,
    count_if(ds = '20201231' and is_obtain_qutoa = 1) as today_obtain_cnt,
    sum(if(ds = '20201231', quota, 0)) as today_sum_quota,
    sum(if(ds = '20201231', quota, 0)) / count_if(ds = '20201231' and is_obtain_qutoa = 1) as today_avg_quota,

    count_if(ds = '20201230') as yesterday_apply_cnt,
    count_if(ds = '20201230' and is_pass_rule = 1) as yesterday_pass_cnt,
    count_if(ds = '20201230' and is_obtain_qutoa = 1) as yesterday_obtain_cnt,
    sum(if(ds = '20201230', quota, 0)) as yesterday_sum_quota,
    sum(if(ds = '20201230', quota, 0)) / count_if(ds = '20201230' and is_obtain_qutoa = 1) as yesterday_avg_quota,

    count(uid) as total_apply_cnt,
    count_if(is_pass_rule = 1) as total_pass_cnt,
    count_if(is_obtain_qutoa = 1) as total_obtain_cnt,
    sum(quota) as total_sum_quota,
    sum(quota) / count_if(is_obtain_qutoa = 1) as total_avg_quota
    from check_t);

--去重有3种方式
--1、distinct
--2、group by
--3、row_number+排序取第一，此处用这一种
drop table if exists t2;
cache table t2 as
with t1 as (
    select *
    from (select *,
        to_date(putout_date) as putout_date2,
        row_number() over (partition by duebill_id order by ds ) rn
        from cache_debt) t
    where t.rn = 1
)
select sum(if(putout_date2='2020-12-31', putout_amt,0)) as today_sum_putout_amt,
    sum(if(putout_date2='2020-12-31',putout_amt,0))/count(distinct if(putout_date2='2020-12-31',uid,null)) as today_avg_putout_amt,
    sum(if(putout_date2='2020-12-30', putout_amt,0)) as yesterday_sum_putout_amt,
    sum(if(putout_date2='2020-12-30',putout_amt,0))/count(distinct if(putout_date2='2020-12-30',uid,null))
as yesterday_avg_putout_amt,
    sum(if(putout_date2<='2020-12-31', putout_amt, 0 )) as all_sum_putout_amt,
    sum(if(putout_date2<='2020-12-31', putout_amt, 0 ))/count(distinct if(putout_date2<='2020-12-31' , uid,null) )
as all_avg_putout_amt
from t1;

select '申请户数' as `指标`, today_apply_cnt as `当日新增`, yesterday_apply_cnt as `昨日新增`,all_apply_cnt as `历史累计` from t1
union all
select '规则通过户数'as item, today_pass_cnt,yesterday_pass_cnt,all_pass_cnt from t1 union all
select '核额成功户数'as item,today_obtain_cnt,yesterday_obtain_cnt,all_obtain_cnt from t1 union all
select '授信金额'as item,today_quota_sum,yesterday_quota_sum,all_quota_sum from t1 union all
select '平均核额'as item,today_quota_avg,yesterday_quota_avg,all_quota_avg from t1 union all
select '发放金额', today_sum_putout_amt,yesterday_sum_putout_amt,all_sum_putout_amt from t2 union all
select '户均发放', today_avg_putout_amt,yesterday_avg_putout_amt,all_avg_putout_amt from t2 ;
```





结果：

```
select '申请户数' as `指标`, today_apply_cnt as `当日新增`, yestoday_apply_cnt as `昨日新增`, all_apply_cnt as `历史累计` from t1 union all
select '规则通过户数' as item, today_pass_cnt, yestoday_pass_cnt, all_pass_cnt from t1 union all
select '核额成功户数' as item, today_obtain_cnt, yestoday_obtain_cnt, all_obtain_cnt from t1 union all
select '授信金额' as item, today_quota_sum, yestoday_quota_sum, all_quota_sum from t1 union all
select '平均核额' as item, today_quota_avg, yestoday_quota_avg, all_quota_avg from t1 union all
select '发放金额', today_sum_putout_amt, yestoday_sum_putout_amt, all_sum_putout_amt from t2 union all
select '户均发放金额', today_avg_putout_amt, yestoday_avg_putout_amt, all_avg_putout_amt from t2 ;
```

指标	当日新增	昨日新增	历史累计
1 申请户数	3.000000	3.000000	8.000000
2 规则通过户数	3.000000	2.000000	7.000000
3 核额成功户数	2.000000	2.000000	6.000000
4 授信金额	1389.000000	1489.000000	4466.000000
5 平均核额	694.500000	744.500000	744.333333
6 发放金额	1389.000000	1489.000000	111266.000000
7 户均发放金额	463.000000	496.333333	2318.041667

第一问 方案2，借用stack函数，性能与方案1一样，都只加载一次表。

```
with t1 as (
    select count_if(ds = '20201231') as today_apply_cnt,
    count_if(ds = '20201231' and is_pass_rule = 1) as today_pass_cnt,
    count_if(ds = '20201231' and is_obtain_quota = 1) as today_obtain_cnt,
    sum(if(ds = '20201231', quota, 0)) as today_sum_quota,
    sum(if(ds = '20201231', quota, 0)) / count_if(ds = '20201231' and is_obtain_quota = 1) as today_avg_quota,

    count_if(ds = '20201230') as yestoday_apply_cnt,
    count_if(ds = '20201230' and is_pass_rule = 1) as yestoday_pass_cnt,
    count_if(ds = '20201230' and is_obtain_quota = 1) as yestoday_obtain_cnt,
    sum(if(ds = '20201230', quota, 0)) as yestoday_sum_quota,
    sum(if(ds = '20201230', quota, 0)) / count_if(ds = '20201230' and is_obtain_quota = 1) as yestoday_avg_quota,

    count(uid) as total_apply_cnt,
    count_if(is_pass_rule = 1) as total_pass_cnt,
    count_if(is_obtain_quota = 1) as total_obtain_cnt,
    sum(quota) as total_sum_quota,
    sum(quota) / count_if(is_obtain_quota = 1) as total_avg_quota
from check t),
```





```
t2 as (
    select uid, putout_amt, to_date(putout_date) as putout_date
    from cache_debt
    group by uid, putout_amt, to_date(putout_date)
),
t3 as (
    select sum(if(putout_date = '2020-12-31', putout_amt, 0)) as today_sum_putout_amt,
           avg(if(putout_date = '2020-12-31', putout_amt, null)) as today_avg_putout_amt,

           sum(if(putout_date = '2020-12-30', putout_amt, 0)) as yestoday_sum_putout_amt,
           avg(if(putout_date = '2020-12-30', putout_amt, null)) as yestoday_avg_putout_amt,

           sum(putout_amt) as total_putout_amt,
           sum(putout_amt) / count(distinct uid) as avg_putout_amt
    from t2
)
--一行转多行的函数有 stack inline explode。stack比inline更简洁。
--因为下面`当日新增`列既有count结果int类型，又有sum结果double类型，所以统一转换成double。
select stack(5,
    '申请户数',
    double(today_apply_cnt),
    double(yestoday_apply_cnt),
    double(total_apply_cnt),
    '规则通过户数',
    double(today_pass_cnt),
    double(yestoday_pass_cnt),
    double(total_pass_cnt),
    '核额通过户数',
    double(today_obtain_cnt),
    double(yestoday_obtain_cnt),
    double(total_obtain_cnt),
    '授信金额',
    double(today_sum_quota),
    double(yestoday_sum_quota),
    double(total_sum_quota),
    '平均核额',
    double(today_avg_quota),
    double(yestoday_avg_quota),
    double(total_avg_quota)) as (`指标`, `当日新增`, `昨日新增`, `历史累计`)
from t1
union all
select stack(2, '发放金额',
    double(today_sum_putout_amt),
    double(yestoday_sum_putout_amt),
    double(total_putout_amt),
    '户均发放金额',
    double(today_avg_putout_amt),
    double(yestoday_avg_putout_amt),
    double(avg_putout_amt))
from t3;
```

结果:



指标	当日新增	昨日新增	历史累计
1 申请户数	3	3	8
2 规则通过户数	3	2	7
3 核额通过户数	2	2	6
4 授信金额	1389	1489	4466
5 平均核额	694.5	744.5	744.333333
6 发放金额	1389	1489	111266
7 户均发放金额	463	496.333333	2318.041667

5. 第2题：贷款产品不良统计

2.基于附录2《借据表》统计下述指标，请提供计SQL

产品类型	在贷客户数	在贷余额	不良余额	余额不良率	不良客户数	客户不良率
XX贷						
YY贷						
ZZ贷						
汇总						

--第二问 假设是今天(2021-10-29)的统计，并考虑对用户去重

```
select * from debt where ds='2021-10-29' ;
drop table if exists prod_type_agg;
create table prod_type_agg(
    prod_type string comment '产品',
    cnt int,
    sum_balance decimal(30, 6),
    bad_balance decimal(30, 6),
    bad_balance_rate decimal(7, 6),
    bad_cnt int,
    bad_cnt_rate decimal(7, 6)
) partitioned by (ds string comment '结果分区');
```





```
--方式一 union all
select prod_type,
       count(distinct uid)          as all_cnt,
       sum(balance)                  as sum_balance,
       sum(if(is_buliang = 1, balance, 0)) as sum_bad_balance,
       sum(if(is_buliang = 1, balance, 0)) / sum(balance) as bad_balance_rate,
       count(distinct if(is_buliang = 1, uid, null)) as bad_cnt,
       count(distinct if(is_buliang = 1, uid, null)) / count(distinct uid) bad_user_rate
from cache_debt
where ds = '2021-10-29'
group by prod_type
union all
select '汇总' prod_type,
       count(distinct uid)          as all_cnt,
       sum(balance)                  as sum_balance,
       sum(if(is_buliang = 1, balance, 0)) as sum_bad_balance,
       sum(if(is_buliang = 1, balance, 0)) / sum(balance) bad_balance_rate,
       count(distinct if(is_buliang = 1, uid, null)) as bad_cnt,
       count(distinct if(is_buliang = 1, uid, null)) / count(distinct uid) bad_user_rate
from cache_debt
where ds = '2021-10-29';
```

结果:

prod_type	all_cnt	sum_balance	sum_bad_balance	bad_balance_rate	bad_cnt	bad_user_rate
xx	7	2500.000000	1000.000000	0.400000	2	0.2857142857142857
zz	5	2300.000000	1500.000000	0.652174	3	0.6
yy	8	1500.000000	300.000000	0.200000	3	0.375
汇总	18	6300.000000	2800.000000	0.444444	8	0.4444444444444444

方式2 使用grouping sets

下面单独用一个简单例子介绍grouping sets 用法

需要计算性别，城市2个维度的所有组合的统计指标。



```
create or replace temporary view bbb(a, b, num) as
values ('M', 'gd', 25),
       ('F', 'bj', 26),
       ('F', 'gd', 25),
       ('M', 'bj', 21),
       ('M', 'gd', 25),
       ('F', 'bj', 28);

set spark.sql.shuffle.partitions=8;
select * from bbb;
```

```
-- 方式2 使用grouping sets
-- 下面单独用一个简单例子介绍grouping sets 用法
-- 需要计算性别，城市2个维度的所有组合的统计指标。
create or replace temporary view bbb(a, b, num) as
values ('M', 'gd', 25),
       ('F', 'bj', 26),
       ('F', 'gd', 25),
       ('M', 'bj', 21),
       ('M', 'gd', 25),
       ('F', 'bj', 28);

set spark.sql.shuffle.partitions=8;
select * from bbb;
```

vices

	a	b	num
1	M	gd	25
2	F	bj	26
3	F	gd	25
4	M	bj	21
5	M	gd	25
6	F	bj	28

--统计语法1

```
select a, b, sum(num) as total_num from bbb group by a, b union all
select a, null as b, sum(num) as total_num from bbb group by a union all
select null as a, b, sum(num) as total_num from bbb group by b union all
select null as a, null as b, sum(num) as total_num from bbb;
```



Output Result 25

9 rows

	a	b	total_num
1	F	gd	25
2	M	bj	21
3	F	bj	54
4	M	gd	50
5	M	<null>	71
6	F	<null>	79
7	<null>	bj	75
8	<null>	gd	75
9	<null>	<null>	150

上面简化写法1，用grouping sets

```
SELECT a,b,sum(num)
FROM bbb
GROUP BY a,b
grouping sets ((a,b),(a),(b),());
```

	a	b	`sum(num)`
1	F	<null>	79
2	M	bj	21
3	F	gd	25
4	M	gd	50
5	F	bj	54
6	M	<null>	71
7	<null>	<null>	150
8	<null>	gd	75
9	<null>	bj	75

上面简化写法2，用with cube





```
SELECT a, b, sum(num) AS total_num
FROM BBB
GROUP BY a, b
with cube;
```

	a	b	total_num
1	F	<null>	79
2	M	bj	21
3	F	gd	25
4	M	gd	50
5	F	bj	54
6	M	<null>	71
7	<null>	<null>	150
8	<null>	gd	75
9	<null>	bj	75

需求方式2 使用grouping sets

```
select if(grouping__id=1,'汇总',prod_type) prod_type,
       count(distinct uid) as all_cnt,
       sum(balance) as sum_balance,
       sum(if(is_buliang = 1, balance, 0)) as sum_bad_balance,
       sum(if(is_buliang = 1, balance, 0)) / sum(balance) bad_balance_rate,
       count(distinct if(is_buliang = 1, uid, null)) as bad_cnt,
       count(distinct if(is_buliang = 1, uid, null)) / count(distinct uid) bad_user_rate
from cache_debt
where ds = '2021-10-29'
group by prod_type
grouping sets (prod_type , ());
```





6. 第3题：逾期率统计

3. 基于附录2《借据表》统计下述指标，请提供统计SQL

产品类型	户数	余额	逾期率/ 不良率
逾期1-30天			
逾期30-90天			
逾期90天以上			
逾期合计			
不良合计			

假设计算当天2021-10-29日的报表

方案一

```
with t1 as (
    select *,
        case
            when overduedays >= 1 and overduedays <= 30 then '逾期1-30天'
            when overduedays > 30 and overduedays <= 90 then '逾期30-90天'
            when overduedays > 90 then '逾期90天以上'
            else '无逾期'
        end type,
        --所有的人数
        count( uid ) over () as all_cnt
    from debt
    where ds = '2021-10-29'
),
t2 as (
    select type,
        count(distinct uid)          as cnt, --户数
        sum(balance)                as balance, --余额
        count(distinct uid) / all_cnt as delay_rate --逾期率
    from t1
    where type != '无逾期'
    group by type, all_cnt
),
t3 as (
    select '逾期合计',
        count(distinct if(overduedays > 0, uid, null)) as cnt, --户数
```



```
sum(if(overduedays > 0, balance, 0))          as balance,--余额
count(distinct if(overduedays > 0,uid,null)) / all_cnt as rate, --逾期率
'不良合计',
count(distinct if(is_buliang=1, uid, null)) as cnt,--不良户数
sum(if(is_buliang=1, balance, 0))            as balance,--不良余额
count(distinct if(is_buliang=1, uid, null)) / all_cnt as rate--不良率

from t1
group by all_cnt
)

select * from t2 union all
select stack(2,*) from t3
```

```
--方案二
with t1 as (
    select *,
        case
            when overduedays >= 1 and overduedays <= 30 then '1-30'
            when overduedays > 30 and overduedays <= 90 then '30-90'
            when overduedays > 90 then '90-'
            else '0'
        end          as overdue_type,
        count(uid) over () as cnt_all
    from debt
    where ds = '2021-10-29'
)
select if(overdue_type is null , '逾期合计',overdue_type ) as `产品类型`,
    count(distinct uid)          as cnt,
    sum(balance)                as sum_balance,
    count(distinct uid) / cnt_all as bad_rate
from t1 where overduedays>0
group by overdue_type, cnt_all
    grouping sets ((overdue_type,cnt_all),(cnt_all))
union all
select '不良合计'              as `产品类型`,
    count(distinct uid)          as cnt,
```





```
sum(balance) as sum_balance,  
count(distinct uid) / cnt_all as bad_rate  
from t1 where is_buliang=1  
group by cnt_all;
```

--方案三

```
with t1 as (  
    select *,  
        case  
            when overduedays >= 1 and overduedays <= 30 then '1-30'  
            when overduedays > 30 and overduedays <= 90 then '30-90'  
            when overduedays > 90 then '90-'  
            else '0'  
        end as overdue_type,  
        count(uid) over () as cnt_all  
    from debt  
    where ds = '2021-10-29'  
) ,  
t2 as (  
    select  
        --逾期1-30  
        '逾期1-30天' as prod_type,  
        count(distinct if(overdue_type = '1-30', uid, null)) as cnt1,  
        sum(if(overdue_type = '1-30', balance, 0)) as sum_balance1,  
        count(distinct if(overdue_type = '1-30', uid, null)) / count(distinct uid) as bad_rate1,  
        --逾期30-90  
        '逾期30-90天' as prod_type,  
        count(distinct if(overdue_type = '30-90', uid, null)) as cnt2,  
        sum(if(overdue_type = '30-90', balance, 0)) as sum_balance2,  
        count(distinct if(overdue_type = '30-90', uid, null)) / count(distinct uid) as bad_rate2,  
        --逾期90以上  
        '逾期90天以上' as prod_type,  
        count(distinct if(overdue_type = '90-', uid, null)) as cnt3,  
        sum(if(overdue_type = '90-', balance, 0)) as sum_balance3,  
        count(distinct if(overdue_type = '90-', uid, null)) / count(distinct uid) as bad_rate3,  
        --逾期合计  
        '逾期合计' as prod_type,  
        count(distinct if(overdue_type != '0', uid, null)) as cnt4,  
        sum(if(overdue_type != '0', balance, 0)) as sum_balance4,  
        count(distinct if(overdue_type != '0', uid, null)) / count(distinct uid) as bad_rate4,  
        --不良合计  
        '不良合计' as prod_type,  
        count(distinct if(is_buliang = 1, uid, null)) as cnt5,  
        sum(if(is_buliang = 1, balance, 0)) as sum_balance5,  
        count(distinct if(is_buliang = 1, uid, null)) / count(distinct uid) as bad_rate5  
    from t1  
)  
select stack(5,t2.* ) as ('产品类型','户数','余额','逾期率/不良率') from t2  
;
```



结果:

	产品类型	户数	余额	逾期率/不良率
1	逾期1-30天	1	700.000000	0.0555555555555555
2	逾期30-90天	5	1900.000000	0.2777777777777778
3	逾期90天以上	2	200.000000	0.1111111111111111
4	逾期合计	8	2800.000000	0.4444444444444444
5	不良合计	8	2800.000000	0.4444444444444444

7. 第4题：用户画像统计

4. 基于附录3《横型输出表》统计下述指标，请提供统计SQL（备注: value值为1时即命中）

统计日期	统计指标	命中户数	命中率
20201010/累计	V01		
	V02		
	V03		
	V04		
	V05		
	V06		

```
--创建模型输出表数据集
create or replace temporary view output_model(ds,sno,create_time,uid,content,update_time) as
values
('20200101','s001','20210101','u001',{'V01':0,"V02":1,"V03":1,"V04":0,"V05":1,"V06":0},'20210101'),
('20200101','s002','20210101','u002',{'V01':1,"V02":1,"V03":0,"V04":1,"V05":0,"V06":0},'20210101'),
('20200101','s003','20210101','u003',{'V01':0,"V02":1,"V03":0,"V04":1,"V05":0,"V06":1},'20210101'),
('20200101','s004','20210101','u004',{'V01':0,"V02":1,"V03":1,"V04":1,"V05":1,"V06":1},'20210101'),
('20200101','s005','20210101','u005',{'V01':0,"V02":0,"V03":0,"V04":1,"V05":1,"V06":1},'20210101');
```





为莘莘学子改变命运而讲课，为千万学生少走弯路而著书

```
select * from output_model;

select stack(6,

        'V01', sum(v01) , sum(v01)/count(1) ,
        'V02', sum(v02) , sum(v02)/count(1) ,
        'V03', sum(v03) , sum(v03)/count(1) ,
        'V04', sum(v04) , sum(v04)/count(1) ,
        'V05', sum(v05) , sum(v05)/count(1) ,
        'V06', sum(v06) , sum(v06)/count(1)

) as (item,cnt,rate)

from

(select *,

    get_json_object( content , '$.V01' ) as v01,
    get_json_object( content , '$.V02' ) as v02,
    get_json_object( content , '$.V03' ) as v03,
    get_json_object( content , '$.V04' ) as v04,
    get_json_object( content , '$.V05' ) as v05,
    get_json_object( content , '$.V06' ) as v06

from output_model where ds<='20201010' ) t ;
```

```
with t1 as (  
    select ds,  
           uid,  
           inline(  
               map_entries(  
                   str_to_map(  
                       --去掉json字符串中的开头的左花括号(、结尾的右花括号)、双引号、单引号  
                       regexp_replace(content, '^\\{\\|\\}|$"|\'|\'|\'', '')  
                   )  
               )  
           ) as (key, value)  
    from output_model  
    where ds <= '20201010'
```

课程大全





```
),  
t2 as (  
    select ds,  
           key,  
           sum(value) as cnt,  
           sum(value) / count(uid) rate  
    from t1  
    group by ds, key  
)  
select * from t2;
```

8. 第5题：Vintage账龄分析

基于附录2《借据表》统计下述指标，请提供Vintage统计SQL（mobX指的是发放后第X月末的不良余额/发放月金额）

发放月份	发放金额	MOB1	MOB2	MOB3	MOB4	MOB5	MOB6	MOB7	MOB8	MOB9	MOB10	MOB11	MOB12
2019-10													
2019-11													
2019-12													
2020-01													
2020-02													
2020-03													
2020-04													
2020-05													
2020-06													
2020-07													
2020-08													
2020-09													
2020-10													

再次回顾借据表：



```
show partitions debt;
```

partition
1 ds=2019-10-01
2 ds=2019-10-02
3 ds=2019-10-03
4 ds=2019-10-04
5 ds=2019-10-05
6 ds=2019-10-06
7 ds=2019-10-07
8 ds=2019-10-08
9 ds=2019-10-09
10 ds=2019-10-10
11 ds=2019-10-11
12 ds=2019-10-12

```
-- 首次缓存会耗时慢  
cache table cache_debt as select /*+ coalesce(6) */ * from debt;  
-- 第二次使用缓存会很快  
select count(*) from cache_debt;
```

count(*)
34893

随机观察2个借据的情况

```
select * from cache_debt where duebill_id='u001-1' order by ds;  
select * from cache_debt where duebill_id='u005-2' order by ds;
```



-- 随机观察2个借据的情况

```
select * from cache_debt where duebill_id='u001-1' order by ds;
```

Output cache_debt

	duebill_id	uid	prod_type	putout_date	putout_amt	balance	is_buliang	overduedays	ds
1	u001-1	u001	xx	2020-09-01	1200.000000	1200.000000	0	0	2020-09-01
2	u001-1	u001	xx	2020-09-01	1200.000000	1200.000000	0	0	2020-09-02
3	u001-1	u001	xx	2020-09-01	1200.000000	1200.000000	0	0	2020-09-03
4	u001-1	u001	xx	2020-09-01	1200.000000	1200.000000	0	0	2020-09-04
5	u001-1	u001	xx	2020-09-01	1200.000000	1200.000000	0	0	2020-09-05
同一笔借据 中间省略每天的余额信息 日期分区不同									
361	u001-1	u001	xx	2020-09-01	1200.000000	100.000000	0	0	2021-08-27
362	u001-1	u001	xx	2020-09-01	1200.000000	100.000000	0	0	2021-08-28
363	u001-1	u001	xx	2020-09-01	1200.000000	100.000000	0	0	2021-08-29
364	u001-1	u001	xx	2020-09-01	1200.000000	100.000000	0	0	2021-08-30
365	u001-1	u001	xx	2020-09-01	1200.000000	100.000000	0	0	2021-08-31
366	u001-1	u001	xx	2020-09-01	1200.000000	0.000000	0	0	2021-09-01

```
select * from cache_debt where duebill_id='u005-2' order by ds;
```

Output cache_debt

	duebill_id	uid	prod_type	putout_date	putout_amt	balance	is_buliang	overduedays	ds
1	u005-2	u005	yy	2020-11-01	1200.000000	1200.000000	0	0	2020-11-01
2	u005-2	u005	yy	2020-11-01	1200.000000	1200.000000	0	0	2020-11-02
3	u005-2	u005	yy	2020-11-01	1200.000000	1200.000000	0	0	2020-11-03
4	u005-2	u005	yy	2020-11-01	1200.000000	1200.000000	0	0	2020-11-04
不良 逾期96天									
359	u005-2	u005	yy	2020-11-01	1200.000000	100.000000	1	96	2021-10-25
360	u005-2	u005	yy	2020-11-01	1200.000000	100.000000	1	97	2021-10-26
361	u005-2	u005	yy	2020-11-01	1200.000000	100.000000	1	98	2021-10-27
362	u005-2	u005	yy	2020-11-01	1200.000000	100.000000	1	99	2021-10-28
363	u005-2	u005	yy	2020-11-01	1200.000000	100.000000	1	100	2021-10-29

分3个大步骤：

步骤1：形成临时表1_每月的总发放金额

发放月份	发放金额
2019-10	aa
2019-11	bb
2019-12	cc
2020-01	dd
2020-02	ee



2020-03	ff
2020-04	gg
2020-05	hh

步骤2：形成临时表2_发放后第几个月末时的不良余额

	发放后第几个月末时的不良余额(元)							
发放月份	1月后	2月后	3月后	4月后	5月后	6月后	7月后	8月后
2019-10	a1	a2	a3	a4	a5	a6	a7	a8
2019-11	b1	b2	b3	b4	b5	b6	b7	b8
2019-12	c1	c2	c3	c4	c5	c6	c7	c8
2020-01	d1	d2	d3	d4	d5	d6	d7	d8
2020-02	e1	e2	e3	e4	e5	e6	e7	e8
2020-03	f1	f2	f3	f4	f5	f6	f7	f8
2020-04	g1	g2	g3	g4	g5	g6	g7	g8
2020-05	h1	h2	h3	h4	h5	h6	h7	h8

步骤3：用上面的临时表1关联临时表2，用临时表2的每个值除以临时表的总金额。

		发放后第几个月末时的不良余额占发放金额的比例							
发放月份	发放金额	1月后	2月后	3月后	4月后	5月后	6月后	7月后	8月后
2019-10	aa	a1/aa	a2/aa	a3/aa	a4/aa	a5/aa	a6/aa	a7/aa	a8/aa
2019-11	bb	b1/bb	b2/bb	b3/bb	b4/bb	b5/bb	b6/bb	b7/bb	b8/bb
2019-12	cc	c1/cc	c2/cc	c3/cc	c4/cc	c5/cc	c6/cc	c7/cc	c8/cc
2020-01	dd	d1/dd	d2/dd	d3/dd	d4/dd	d5/dd	d6/dd	d7/dd	d8/dd
2020-02	ee	e1/ee	e2/ee	e3/ee	e4/ee	e5/ee	e6/ee	e7/ee	e8/ee
2020-03	ff	f1/ff	f2/ff	f3/ff	f4/ff	f5/ff	f6/ff	f7/ff	f8/ff
2020-04	gg	g1/gg	g2/gg	g3/gg	g4/gg	g5/gg	g6/gg	g7/gg	g8/gg





2020-05	hh	h1/hh	h2/hh	h3/hh	h4/hh	h5/hh	h6/hh	h7/hh	h8/hh
---------	----	-------	-------	-------	-------	-------	-------	-------	-------

```
--因为每天都有全量的借据表，
-- 1、下面的t1-t3，是为了统计每个发放月份的发放金额。
-- 每笔借款发放后，可能有365条状态数据，只取发放的第一条数据，获取发放金额。
-- 因为一个借据的日期变但是发放金额不会变。
-- 2、下面的t4-t6，是为了统计在发放月后，第几月的不良余额。
--3、最后t3和t6左关联。
--去重有3种方式
--1、distinct 字段1，字段2
--2、group by 字段1，字段2
--3、row_number() over(partition by 字段1，字段2 order by 字段3) rn
--下面用row_number去重
with t1 as (
    select *,
        row_number() over (partition by duebill_id order by ds) rn
    from cache_debt
),
t2 as (
    select *
    from t1
    where rn = 1
),
t3 as (
    select substr(putout_date, 0, 7) as putout_month,
        sum(putout_amt) as sum_putout_amt
    from t2
    group by substr(putout_date, 0, 7)
),
t4 as (
    select *,
        row_number() over (partition by uid,duebill_id,substr(ds, 0, 7) order by ds desc) as rn2
    from cache_debt
),
t5 as (
    select *,
        ceil(months_between(ds, putout_date)) as month_diff,
        substr(putout_date, 0, 7) as putout_month
    from t4
    where rn2 = 1
),
t6 as (
    select putout_month,
        sum(if(month_diff = 1 and is_buliang = 1, balance, 0)) as sum_bad1,
        sum(if(month_diff = 2 and is_buliang = 1, balance, 0)) as sum_bad2,
        sum(if(month_diff = 3 and is_buliang = 1, balance, 0)) as sum_bad3,
        sum(if(month_diff = 4 and is_buliang = 1, balance, 0)) as sum_bad4,
        sum(if(month_diff = 5 and is_buliang = 1, balance, 0)) as sum_bad5,
        sum(if(month_diff = 6 and is_buliang = 1, balance, 0)) as sum_bad6,
        sum(if(month_diff = 7 and is_buliang = 1, balance, 0)) as sum_bad7,
        sum(if(month_diff = 8 and is_buliang = 1, balance, 0)) as sum_bad8,
        sum(if(month_diff = 9 and is_buliang = 1, balance, 0)) as sum_bad9,
```





```
sum(if(month_diff = 10 and is_buliang = 1, balance, 0)) as sum_bad10,
sum(if(month_diff = 11 and is_buliang = 1, balance, 0)) as sum_bad11,
sum(if(month_diff = 12 and is_buliang = 1, balance, 0)) as sum_bad12

from t5
group by putout_month
),
t7 as (
select t3.putout_month,
sum_putout_amt,
sum_bad1 / sum_putout_amt as mob1,
sum_bad2 / sum_putout_amt as mob2,
sum_bad3 / sum_putout_amt as mob3,
sum_bad4 / sum_putout_amt as mob4,
sum_bad5 / sum_putout_amt as mob5,
sum_bad6 / sum_putout_amt as mob6,
sum_bad7 / sum_putout_amt as mob7,
sum_bad8 / sum_putout_amt as mob8,
sum_bad9 / sum_putout_amt as mob9,
sum_bad10 / sum_putout_amt as mob10,
sum_bad11 / sum_putout_amt as mob11,
sum_bad12 / sum_putout_amt as mob12
from t3
left join t6 on t3.putout_month = t6.putout_month
)
select * from t7;
```

经分析上面加载了2次cache_debt表，可以合并成只加载一次。

优化：

```
with t1 as (
select *,
row_number() over (partition by duebill_id order by ds) rn,
row_number() over (partition by uid,duebill_id,substr(ds, 0, 7) order by ds desc) as rn2,
ceil(months_between(ds, putout_date)) as month_diff,
substr(putout_date, 0, 7) as putout_month
from cache_debt
),
t2 as (
select putout_month,
sum(if(rn=1, putout_amt,0)) as sum_putout_amt,
sum(if(rn2=1 and month_diff = 1 and is_buliang = 1, balance, 0)) as sum_bad1,
sum(if(rn2=1 and month_diff = 2 and is_buliang = 1, balance, 0)) as sum_bad2,
sum(if(rn2=1 and month_diff = 3 and is_buliang = 1, balance, 0)) as sum_bad3,
sum(if(rn2=1 and month_diff = 4 and is_buliang = 1, balance, 0)) as sum_bad4,
sum(if(rn2=1 and month_diff = 5 and is_buliang = 1, balance, 0)) as sum_bad5,
sum(if(rn2=1 and month_diff = 6 and is_buliang = 1, balance, 0)) as sum_bad6,
sum(if(rn2=1 and month_diff = 7 and is_buliang = 1, balance, 0)) as sum_bad7,
```





```
sum(if(rn2=1 and month_diff = 8 and is_buliang = 1, balance, 0)) as sum_bad8,
sum(if(rn2=1 and month_diff = 9 and is_buliang = 1, balance, 0)) as sum_bad9,
sum(if(rn2=1 and month_diff = 10 and is_buliang = 1, balance, 0)) as sum_bad10,
sum(if(rn2=1 and month_diff = 11 and is_buliang = 1, balance, 0)) as sum_bad11,
sum(if(rn2=1 and month_diff = 12 and is_buliang = 1, balance, 0)) as sum_bad12
from t1 where rn=1 or rn2=1
group by putout_month
),
t3 as (
select putout_month,
sum_putout_amt,
sum_bad1 / sum_putout_amt as mob1,
sum_bad2 / sum_putout_amt as mob2,
sum_bad3 / sum_putout_amt as mob3,
sum_bad4 / sum_putout_amt as mob4,
sum_bad5 / sum_putout_amt as mob5,
sum_bad6 / sum_putout_amt as mob6,
sum_bad7 / sum_putout_amt as mob7,
sum_bad8 / sum_putout_amt as mob8,
sum_bad9 / sum_putout_amt as mob9,
sum_bad10 / sum_putout_amt as mob10,
sum_bad11 / sum_putout_amt as mob11,
sum_bad12 / sum_putout_amt as mob12
from t2
)
select * from t3;
```

结果:

	putout_month	sum_putout_amt	mob1	mob2	mob3	mob4	mob5	mob6	mob7	mob8	mob9	mob10	mob11	mob12
1	2019-10	12000.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
2	2019-11	9600.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
3	2019-12	8400.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
4	2020-01	10800.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
5	2020-02	9600.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
6	2020-03	6000.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
7	2020-04	2400.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
8	2020-05	2400.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
9	2020-06	9600.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
10	2020-07	6000.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
11	2020-08	6000.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
12	2020-09	6000.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
13	2020-10	6000.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
14	2020-11	11180.000000	0.062567	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.071505	0.080443	0.071505	0.035753
15	2020-12	5278.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
16	2021-01	2400.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
17	2021-02	1200.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
18	2021-03	3600.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.388889	0.333333	0.277778	0.000000	0.000000	0.000000
19	2021-04	2400.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
20	2021-05	2400.000000	0.000000	0.000000	0.000000	0.375000	0.333333	0.583333	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000





华泰证券

1. 需求列表

需求1：查询每门课都大于80分的学生

需求2：表转置

需求3：查询查询没有学全所有课的学生

需求4：查询每日买入金额前十和卖出金额前十的基金

需求5：求每日每个客户每只股票的市值增长率

需求6：计算基金净值回撤

2. 第1题 查询每门课都大于80分的学生

1.用一条SQL语句查询出每门课都大于80分的学生姓名

name	kecheng	fenshu
张三	语文	81
张三	数学	75
李四	语文	76
李四	数学	90
王五	语文	81
王五	数学	100
王五	英语	90

```
--查看shuffle并行度，默认是200
set spark.sql.shuffle.partitions;
--手动指定shuffle并行度为4
set spark.sql.shuffle.partitions=4;
--题目一
--造数据
```





```
create or replace temporary view table0(name,kecheng,fenshu) as
values
    ('zs','语文',81),
    ('zs','数学',75),
    ('ls','语文',76),
    ('zs','数学',90),
    ('ww','语文',81),
    ('ww','数学',100),
    ('ww','英语',90);
select * from table0;
--写SQL
--解法一、窗口函数
select distinct name from
(select *,
    --取最小，语法一
    min(fenshu) over(partition by name) min1,
    --取最小，语法二
    first_value(fenshu) over(partition by name order by fenshu) min2
from table0) a
where a.min1>80;

--解法二、group by,语法1
select name from
(select name,
    min(fenshu) min_score
from table0 group by name) a
where min_score>80;
--解法二、group by,语法2
select name
from table0 group by name
having min(fenshu)>80;
```

3. 第2题 表转置

2.面试题：表转置





year	month	amount
1991	1	1.1
1991	2	1.2
1991	3	1.3
1991	4	1.4
1992	1	2.1
1992	2	2.2
1992	3	2.3
1992	4	2.4

查成这样一个结果

year	m1	m2	m3	m4
1991	1.1	1.2	1.3	1.4
1992	2.1	2.2	2.3	2.4

```
create or replace temporary view table2(year,month,amount) as
values
    (1991,1,1.1),
    (1991,2,1.2),
    (1991,3,1.3),
    (1991,4,1.4),
    (1992,1,2.1),
    (1992,2,2.2),
    (1992,3,2.3),
    (1992,4,2.4);
select * from table2;
```

```
--行转列
--常规做法是，group by+sum(if())
--SQLserver中有pivot专门用来行转列
--原始写法
```





```
select year,
       sum(a) as m1,
       sum(b) as m2,
       sum(c) as m3,
       sum(d) as m4
from
  (select *,
         if(month=1,amount,0) a,
         if(month=2,amount,0) b,
         if(month=3,amount,0) c,
         if(month=4,amount,0) d
   from table2) t
group by t.year
;
--简化写法
select year,
       sum(if(month=1,amount,0)) m1,
       sum(if(month=2,amount,0)) m2,
       sum(if(month=3,amount,0)) m3,
       sum(if(month=4,amount,0)) m4
from table2
group by year;
--方法二
select year,
       arr[0] m1,
       arr[1] m2,
       arr[2] m3,
       arr[3] m4
from
  (select year,
         collect_list(amount) arr
   from table2 group by year) a;
```





4. 第3题 业务需求实现

2.有表结构及数据如下，请完成相应查询

班级表: class	
cid	caption
1	三年二班
2	一年三班
3	三年一班

学生表: student			
sid	sname	gender	class_id
1	张三	女	1
2	李四	女	1
3	王五	男	2

老师表: teacher	
tid	tname
1	马云
2	马化腾
3	俞敏洪

课程表: course			
cid	cname	teacher_id	
1	生物	1	
2	体育	1	
3	物理	2	

成绩表: score			
sid	student_id	course_id	number
1	1	1	58
2	1	2	68
3	2	2	89

4.1 3.1查询课程2比课程1成绩低的学号

查询课程编号“2”的乘积比课程编号“1”低的所有同学的学号、姓名。





4.2 3.2查询没有学全所有课的学生

查询没有学全所有课的同学的学号、姓名

```
create or replace temporary view student
    (sid, sname, gender, class_id) as
values (1, '张三', '女', 1),
    (2, '李四', '女', 1),
    (3, '王五', '男', 2);
select * from student;
create or replace temporary view course
    (cid, cname, teacher_id) as
values (1, '生物', 1),
    (2, '体育', 1),
    (3, '物理', 2);
select * from course;

create or replace temporary view score
    (sid, student_id, course_id, number) as
values (1, 1, 1, 58),
    (2, 1, 2, 68),
    (3, 2, 2, 89);
select * from score;
--分题1
with t1 as(
    select student_id,
        sum(if(course_id=2,number,0)) as pe, --体育
        sum(if(course_id=1,number,0)) as bio --生物
    from score
    group by student_id
    having pe<bio)
select sid, sname
from t1
join student
on t1.student_id = sid
;

--分题2
--如果一个语句中子查询多，可以用with as语法来提高可读性。
with t1 as (
```





```
select a.sid,a.sname,
       count(course_id) as cnt1
from student a
left join score b on a.sid=b.student_id
group by a.sid,a.sname
),
t2 as (
    select count(cid) as cnt_all from course
)
select
    sid,sname
from t1 join t2
where cnt1<cnt_all;
```

5. 第4题 查询每日买入金额前十和卖出金额前十的基金

基金产品交易流水表t_trade,描述如下:

```
create table t_trade(
    init_date integer comment '日期',
    client_id varchar(20) comment '客户号',
    fund_code varchar(6) comment '基金产品代码',
    business_flag integer comment '1卖出 2买入',
    business_balance numeric(38,10) comment '交易金额'
);
```

求每日买入成交金额前10名和卖出成交金额的前十名的基金产品。

```
create or replace view
t_trade(init_date,client_id,fund_code,business_flag,business_balance) as
values
(1,1,1,1,1000) ,
(1,88,1,1,2000) ,
```





```
(1,2,2,1,900) ,
(1,3,3,1,800) ,
(1,4,4,1,700) ,
(1,8,9,2,10000) ,
(1,7,8,2,9000) ,
(1,6,7,2,8000)
;

with t1 as (
    select init_date,fund_code,business_flag,
           sum(business_balance) money
    from t_trade
    group by init_date,fund_code,business_flag
),
t2 as (
    select *,
           row_number() over (partition by init_date,business_flag order by money) rn
    from t1
)
select * from t2 where rn<=10;
```

6. 第5题 求每日每个客户每只股票的市值增长率

5.客户购买理财hs_asset.stock,描述如下:

```
create table hs_asset.stock(
    init_date integer comment '日期',
    client_id varchar(20) comment '客户号',
    stock_code varchar(6) comment '股票代码',
    market_value numeric(38,20) comment '持仓市值'
);
```

定义: 市值增长率=(当日市值-上日市值)/上日市值,

求每日每个客户每只股票的市值增长率





--假设都是连续日期，没有中断。休市也有持仓记录。

```
create or replace temporary view
```

```
stock(init_date, client_id, stock_code, market_value)
```

```
as values
```

```
('2021-09-01', 1, '001', 100),
```

```
('2021-09-02', 1, '001', 200),
```

```
('2021-09-03', 1, '001', 300),
```

```
('2021-09-01', 1, '002', 500),
```

```
('2021-09-02', 1, '002', 400),
```

```
('2021-09-03', 1, '002', 300),
```

```
('2021-09-01', 2, '003', 100),
```

```
('2021-09-02', 2, '003', 200),
```

```
('2021-09-03', 2, '003', 300)
```

```
;
```

```
select *,
```

```
if(last_market_value!=0,(market_value-last_market_value)/last_market_value,null) as incre_rate
```

```
from
```

```
(select *,
```

```
lag(market_value) over (partition by client_id,stock_code order by init_date) as last_market_value--
```

```
上日市值
```

```
from stock) a;
```

7. 第6题 计算基金净值回撤

编写一个脚本，代码可用python或pyspark或scala（40分）

需求：cust_pft是客户(cust_id)每天(date)的资产净值(pft)，现在需要获得每个客户近1年的最大回撤：

最大回撤定义：在该客户的净值曲线中，当出现最大的净值的时点记为m1，这之后出现的净值比m1那天净值相差最大的净值记为m2，最大回撤就是(m2-m1)/m1.(注意是出现最大净值之后的最小净值，两者的差)





```
create or replace temporary view cust_pft (cust_id,date,pft)
as values
(1,'2021-01-01',10000),
(1,'2021-01-02',10010),
(1,'2021-01-03',10020),
(1,'2021-01-04',9999),
(1,'2021-01-05',9998),
(1,'2021-01-06',10020),
(1,'2021-12-27',6000),
(1,'2021-12-28',6001),
(1,'2021-12-29',6002);

with t1 as (
    select *,
        min(pft) over (partition by cust_id) m1
    from cust_pft
    where date >= date_sub(`current_date`, 365)
),
t2 as (
    select *,
        min(if(m1=pft,date,null)) over (partition by cust_id) as date1
    from t1
),
t3 as (
    select cust_id,
        max(pft) m1,
        min(pft) m2,
        (max(pft)-min(pft))/max(pft) as max_withdraw
    from t2
    where date>=date1
    group by cust_id
)
select * from t3
;
```





招商信诺人寿

1. 需求列表

需求1：排序取第8-16条记录

需求2：SQL执行顺序

需求3：班级分组内按成绩排名

需求4：聚合窗口函数

需求5：时间序列题型

2. 第1题 排序取第8-16条记录

有一张表T(F1,F2,F3,F4),要根据字段F2排序后取第8-16条记录显示，请写出SQL。

```
create temporary view t(f1, f2, f3, f4) as
values (1, 1, 1, 1),
(1, 2, 1, 1),
(1, 3, 1, 1),
(1, 4, 1, 1),
(1, 5, 1, 1),
(1, 6, 1, 1),
(1, 7, 1, 1),
(1, 8, 1, 1),
(1, 9, 1, 1);

--limit Y: 从头取 Y 个数据
--limit X, Y: 跳过 X 个数据，读取 Y 个数据
--limit Y offset X: 跳过 X 个数据，选取 Y 个数据
--如果跳过的数量大于总数量则返回为空，如果选取的数量不够，则有多少返回多少。
--limit Y语句hive和SparkSQL都支持。
--但是limit X, Y语句hive支持，SparkSQL不支持，需要用row_number来解决。
--mysql/hive版
select * from t order by f2 limit 7,9;
--oracle版
select * from (select * from t order by f2) where rownum>=8 and rownum<=16 ;
```




```
--spark-sql版  
select * from (select *,row_number() over (order by f2) rn from t ) where rn>=8 and  
rn<=16 ;
```

3. 第2题 SQL执行顺序

写出下面SQL语句的执行顺序：

select a.column1,count(*) as countvalue,sum(b.column3) as sumvalue from tablea a inner join
tableb b on a.column1=b.column1 and a.column2=1 where b.column3=10 group by a.column1
having count(*) >10 order by a.column1

1 __from__ 2 __inner join__ 3 __where__ 4 __group by__
5 __select__ 6 __count/ sum__ 7 __having__ 8 __order by__

4. 第3题 班级分组内按成绩排名

成绩表Score:SNO、SCLASS、CHINESE、ENGLISH、ARITH分别代表学生号、学生所属班级、语文成绩、英语成绩、数学成绩。

数据如下：

SNO	SCLASS	CHINESE	ENGLISH	ARITH
1	1	83	57	73
2	1	58	76	90
3	3	89	84	96
4	2	79	86	82
5	1	58	81	93
6	1	56	90	59





7	3	77	66	77
8	2	63	56	55
9	1	91	78	91
10	3	95	81	50

根据以班级分组，按照语文成绩、英语成绩、数学成绩进行排名。

SNO	SCLASS	CHINESE	ENGLISH	ARITH	CHINESE_RANK	ENGLISH_RANK	ARITH_RANK
5	1	58	81	93	3	2	1
9	1	91	78	91	1	3	2
2	1	58	76	90	3	4	3
1	1	83	57	73	2	5	4
6	1	56	90	59	5	1	5
4	2	79	86	82	1	1	1
8	2	63	56	55	2	2	2
3	3	89	84	96	2	1	1
7	3	77	66	77	3	3	2
10	3	95	81	50	1	2	3

```
--3.成绩表, Score:SNO、SCLASS、CHINESE、ENGLISH、ARITH
-- 分别代表学生号、学生所属班级、语文成绩、英语成绩、数学成绩。
create or replace temporary view
score(SNO,SCLASS,CHINESE,ENGLISH,ARITH) as values
(1,1,83,57,73),
(2,1,58,76,90),
(3,3,89,84,96),
(4,2,79,86,82),
(5,1,58,81,93),
```





```
(6,1, 56, 90, 59),
(7,3, 77, 66, 77),
(8,2, 63, 56, 55),
(9,1, 91, 78, 91),
(10,3, 95, 81, 50);

select * from score;

select *,
       rank() over (partition by SCLASS order by CHINESE desc) chinese_rank,
       row_number() over (partition by SCLASS order by ENGLISH desc) english_rank,
       row_number() over (partition by SCLASS order by ARITH desc) ARITH_rank
from score;
```

5. 第4题 聚合窗口函数

根据下图1的数据编写SQL，实现图2的结果

图1

ID	NAME
1	/
2	A
3	B
4	C
5	/
6	D
7	E
8	/
9	F
10	G
11	H

图2

ID	NAME	GROUP ID
2	A	1
3	B	1
4	C	1
6	D	2
7	E	2
9	F	3
10	G	3
11	H	3



```
create or replace temporary view tab3(id, name) as
values (1, '/'),
       (2, 'A'),
       (3, 'B'),
       (4, 'C'),
       (5, '/'),
       (6, 'D'),
       (7, 'E'),
       (8, '/'),
       (9, 'F'),
       (10, 'G'),
       (11, 'H');

select * from tab3;

select *
from (select *, count_if(name = '/') over ( order by id) cnt from tab3) t1
where name != '/';
```

6. 第5题 时间序列题型

5根据下图中的数据，请编写SQL找出申请日期到受理日期间超过10个工作日（包含10个工作日）的ID有哪些？

日期 dt	是否工作日 isworkday
2020-06-1	Y
2020-06-2	Y
2020-06-3	Y
2020-06-4	Y
2020-06-5	Y
2020-06-6	N
2020-06-7	N
2020-06-8	Y
2020-06-9	Y

id	申请日期 startdate	受理日期 enddate
1	2020-06-02	2020-06-08
2	2020-06-03	2020-06-26
3	2020-06-05	2020-06-15
4	2020-06-15	2020-06-30
5	2020-06-18	2020-06-27





2020-06-10	Y
2020-06-11	Y
2020-06-12	Y
2020-06-13	N
2020-06-14	N
2020-06-15	Y
2020-06-16	Y
2020-06-17	Y
2020-06-18	Y
2020-06-19	Y
2020-06-20	N
2020-06-21	N
2020-06-22	Y
2020-06-23	Y
2020-06-24	Y
2020-06-25	N
2020-06-26	N
2020-06-27	N
2020-06-28	Y
2020-06-29	Y
2020-06-30	Y

```
create or replace temporary view tab1 (dt, isworkday) as
values ('2020-06-1', 'Y'),
       ('2020-06-2', 'Y'),
       ('2020-06-3', 'Y'),
       ('2020-06-4', 'Y'),
       ('2020-06-5', 'Y'),
```





```
('2020-06-6', 'N'),
('2020-06-7', 'N'),
('2020-06-8', 'Y'),
('2020-06-9', 'Y'),
('2020-06-10', 'Y'),
('2020-06-11', 'Y'),
('2020-06-12', 'Y'),
('2020-06-13', 'N'),
('2020-06-14', 'N'),
('2020-06-15', 'Y'),
('2020-06-16', 'Y'),
('2020-06-17', 'Y'),
('2020-06-18', 'Y'),
('2020-06-19', 'Y'),
('2020-06-20', 'N'),
('2020-06-21', 'N'),
('2020-06-22', 'Y'),
('2020-06-23', 'Y'),
('2020-06-24', 'Y'),
('2020-06-25', 'N'),
('2020-06-26', 'N'),
('2020-06-27', 'N'),
('2020-06-28', 'Y'),
('2020-06-29', 'Y'),
('2020-06-30', 'Y');

create temporary view tab2 (id, startdate, enddate) as
values (1, '2020-06-02', '2020-06-08'),
(2, '2020-06-03', '2020-06-26'),
(3, '2020-06-05', '2020-06-15'),
(4, '2020-06-15', '2020-06-30'),
(5, '2020-06-18', '2020-06-27');

select * from tab1;
select * from tab2;

with a as (select id, explode(sequence(to_date(startdate),
to_date(enddate), interval 1 day)) dat from tab2)
select id, count(dat) days
from a
join tab1 b on a.dat = to_date(b.dt)
where isworkday = 'Y'
group by id
having days >= 10
```





```
order by id;
```





交通银行

1. 需求列表

需求1：金额环比增长率

需求2：金额排名占比

需求3：数据发散问题

需求4：left/semi/anti/join区分

2. 第1题 金额环比增长率

1: SALES表的表数据如下：

year	month	amt
2021	04	100
2021	05	200
2020	04	400
2020	05	150
2019	04	300
2019	05	
2020	04	50

问题：请写出可以得到以下的结果SQL

year	4月份金额	5月份金额	总金额	环比 (%)
2020	150	150	600	
2021	100	200	300	

```
create temporary view sales(year,month,amt)
```





```
as values
(2021, '04', 100),
(2021, '05', 200),
(2020, '04', 400),
(2020, '05', 150),
(2019, '04', 300),
(2019, '05', null),
(2020, '04', 50);
--简答题第一题
--行转列的套路，就是用sum(case when)
with t1 as (
    select year,
           sum(if(month = '04', amt, 0)) as amt_4,
           sum(if(month = '05', amt, 0)) as amt_5,
           sum(amt) as amt_all
    from sales
    group by year
),
    t2 as (
        select *,
               round(if(amt_4 != 0, (amt_5 - amt_4) / amt_4, null), 2) as rate
        from t1
    )
select year, amt_4, amt_5, amt_all,
       rate*100 as rate
from t2 ;
```

3. 第2题 金额排名占比

Emp表的表数据如下：

NAME	MONTH	AMT
张三	01	100
李四	02	120
王五	03	150
赵六	04	500
张三	05	400



李四	06	350
王五	07	180
赵六	08	400

问题：请写出可以得到以下的结果SQL

NAME	总金额	排名
赵六	900	1
张三	500	2
李四	470	3
王五	330	4

```
create temporary view emp(name, month, amt)
as values ('张三', '01', 100),
        ('李四', '02', 120),
        ('王五', '03', 150),
        ('赵六', '04', 500),
        ('张三', '05', 400),
        ('李四', '06', 350),
        ('王五', '07', 180),
        ('赵六', '08', 400);

--rank 1224
--dense_rank 1223
with t1 as (select name,
                  sum(amt) as sum_amt
            from emp
            group by name),
    t2 as (
        select name,
               sum_amt,
               row_number() over (order by sum_amt desc) rn,
```



```
sum_amt/sum(sum_amt) over () as rate
from t1
)
select name, sum_amt, rn, concat(round(rate*100,2),'%') rate from t2;
```

4. 第3题 数据发散问题

EMP表的数据如下，其中CUS_NO为主键

CUS_NO	CUS_NAME	ORG_CODE	BALANCE
1	张三	01330999999	100000
2	李四	01331999999	200000
3	王五	01332999999	300000
4	赵六	01331999999	400000
5	胜七	01331999999	500000

EMP_ORG表的数据如下，其中CUS_NO+ORG_CODE为主键

CUS_NO	ORG_CODE	ORG_NAME	HIRE_NO
1	01330999999	深圳分行	G0000001
1	01331999999	广州分行	G0000002
1	01332999999	湛江分行	G0000003

```
SELECT CUS_NO
,A.CUS_NAME
,COALESCE(A.ORG_CODE,B.ORG_CODE) AS ORG_CODE
,B.ORG_NAME
,A.BALANCE
,B.HIRE_NO
FROM EMP A
JOIN EMP_ORG B
ON A.CUS_NO = B.CUS_NO
```





以上SQL是否存在问题？如果存在问题，请指出问题，并修改正确。

问题2：问题1中修改正确后的SQL可以得到几条数据？

```
create or replace temporary view emp(cus_no, cus_name, org_code, balance)
as values
(1, '张三', 0133099999, 100000),
(2, '李四', 0133199999, 2000),
(3, '五', 0133299999, 300000),
(4, '趣六', 0133199999, 400000),
(5, '胜七', 0133199999, 500000);

create or replace temporary view emp_org(cus_no,
org_code, org_name, hier_no)
as
values
(1, 0133099999, '深圳分行', 'G0000001'),
(1, 0133199999, '广州分行', 'G0000001'),
(1, 0133299999, '湛江分行', 'G0000001');
--left join时，条件写在on和where后的效果是不一样的。
--下面是符合题目的写法，emp有几条结果就有几条
select *
from emp a
left join emp_org b
on a.cus_no = b.cus_no
and a.org_code=b.org_code;

select *
from emp a
left join emp_org b
on a.cus_no = b.cus_no
where a.org_code=b.org_code;

--如果用inner join 再做一次，条件写在on和where后的效果是一样的。
select *
from emp a
join emp_org b
on a.cus_no = b.cus_no
and a.org_code=b.org_code;

select *
from emp a
join emp_org b
on a.cus_no = b.cus_no
where a.org_code=b.org_code;
```





5. 第4题 left/semi/anti/join区分

HQL中left semi join、left anti join 和left join有什么区别？

```
--第九题，下面3者的区别
--left join
--left semi join
--left anti join
--所有的银行客户
create or replace temporary view tab1(id,name) as
  values
    (1,'zs'),
    (2,'ls'),
    (3,'ww'),
    (4,'zl'),
    (5,'sq');

--逾期的银行客户
create or replace temporary view tab2(id,name) as
  values
    (1,'zs'),
    (2,'ls'),
    (6,'xx');

select * from tab1;
select * from tab2;
--left join一般用来打标记，每个客户是否是逾期客户
select a.*,
       if(b.id is not null,true,false) is_delay
from tab1 a
left join tab2 b on a.id=b.id;

--用来从a中筛选出在b中的数据，结果不会展现任何b的数据。
--用来从a中筛选出逾期客户
select *
from tab1 a
left semi join tab2 b on a.id=b.id;
--写法2
select a.*
from tab1 a
left join tab2 b on a.id=b.id
```





```
where b.id is not null
;
--写法3
select a.*
from tab1 a
join tab2 b on a.id=b.id;

--用来从a中筛选出不在b中的数据，不会展现任何b的数据。
--用来从a中筛选出非逾期客户
select *
from tab1 a
left anti join tab2 b on a.id=b.id;

--写法2
select a.*
from tab1 a
left join tab2 b on a.id=b.id
where b.id is null;
```





百丽

1. 需求列表

需求1: Hive数据导入

需求2: Grouping sets 维度分析

需求3: 百丽业务统计分析

2. 第1题 Hive数据导入

笔试时间为30分钟，做完后请告知前台

一、请写出将/home/work/20191020.csv数据上传到hive表belle_test.dm_belle_csv_data(内部表)的脚本

```
load data local inpath '/home/work/20191020.csv' into table
belle_test.dm_belle_csv_data;
```

3. 第2题 Grouping sets 维度分析

二、请将如下SQL修改为Hive的窗口分析函数来实现（不使用union all）

```
select invoice_type,
       delivery_type,
       sum(quantity) quantity,
       sum(pay_total_amt) pay_total_amt
from bi_analysis.lark_ec_sale_order_parq
where pay_time = '2019-10-20'
group by invoice_type, delivery_type
grouping sets
((invoice_type, delivery_type), (invoice_type));
```



4. 第3题 百丽业务统计分析

4.1 第3.1题 平均购买件数

4.1.1 第3.2题 年龄段购买件数

4.1.2 第3.3题 查询喜好品牌

4.1.3 第3.4题 带条件的客户数统计

三、数据表如下

Date	username	user_age	brand	sales_number
2019-10-1	依娜	28	百丽	12
2019-10-1	依娜	28	天美意	30
2019-10-2	丽美	26	百丽	6
2019-10-2	筱雪	20	他她	9
2019-10-2	丽美	26	思加图	1
2019-10-3	被雪	20	拨佳	2
2019-10-3	诗凝	23	百思图	5

数据字典：

Date：日期

Username：用户名称

User_age：用户年龄

Brand：购买品牌

Sale_number：购买件数

问题：

- 1、统计：用户总量、用户平均年龄、用户平均购买件数
- 2、统计：每5岁一个年龄段，统计每个分段用户总量、用户平均购买件数





3、统计：每个用户最喜欢购买的品牌

4、统计：每个品牌购买件数大于等于3的用户总量，如果某个用户有一个品牌购买件数小于3就不算，例如：用户丽美，因为思加图品牌购买数小于3，所以不应该出现在结果中。

```
create or replace temporary view
    table0(date,username,user_age,brand,sales_number) as values
('2019/10/1','伊娜',28,'百丽',12),
('2019/10/1','伊娜',28,'天美意',30),
('2019/10/2','丽美',26,'百丽',6),
('2019/10/2','薇雪',20,'他她',9),
('2019/10/2','丽美',26,'思加图',1),
('2019/10/3','薇雪',20,'拔佳',2),
('2019/10/3','诗凝',23,'思加图',5);
-- 三.1
select count(username) cnt,
       avg(user_age)    avg_age,
       avg(sum1)        avg_num
from
(select username ,
       user_age,
       sum(sales_number) sum1
from table0 group by username,user_age);
-- 三.2
with t1 as (
    select *,
           concat( floor(user_age/5)*5,
                  '- ',
                  floor(user_age/5)*5+5 ) as stage
    from table0
)
select stage,
       count(distinct username) cnt1,
       sum(sales_number)/count(distinct username) avg1
from t1 group by stage;
-- 三.3
with t1 as (
    select username,brand,sum(sales_number) sum1
    from table0 group by username,brand
),t2 as (
    select *, row_number() over (partition by username order by sum1 desc) rn
    from t1
)
select username,brand from t2 where rn=1;
```





```
--三.4
select brand,
       count(distinct username) cnt1
from table0 where sales_number>=3
group by brand;

select brand,
       count(distinct if(sales_number>=3,username,null) ) cnt
from table0
group by brand;
```





跨越物流

1. 需求列表

需求1：求出每个部门工资最高的前三名员工

需求2：统计每年入职总数

需求3：每个品牌的总营销天数

温馨提示

- 可使用任何数据库SQL实现
- 需使用单条SQL实现功能，如果有多种实现方式可加分
- 如不理解题目含义，可以联系HR
- 要求在30分钟内完成题目

2. 第1题 求出每个部门工资最高的前三名员工，并计算这些员工的工资占所属部门总工资的百分比。

员工表结构

emp			
empno	员工工号	integer	
ename	员工姓名	string	
hiredate	入职日期	string	
sal	员工薪水	integer	
deptno	部门编号	integer	

员工表数据





empno	ename	hiredate	sal	deptno
7521	WARD	22/2/1981	1250	30
7566	JONES	2/4/1981	2975	20
7876	ADAMS	13/7/1987	1100	20
7369	SMITH	17/12/1980	800	20
7934	MILLER	23/1/1982	1300	10
7844	TURNER	8/9/1981	1500	30
7782	CLARK	9/6/1981	2450	10
7839	KING	17/11/1981	5000	10
7902	FORD	3/12/1981	3000	20
7499	ALLEN	20/2/1981	1600	30
7654	MARTIN	28/9/1981	1250	30
7900	JAMES	3/12/1981	950	30
7788	SCOTT	13/7/1987	3000	20
7698	BLAKE	1/5/1981	2850	30

题目描述

求出每个部门工资最高的前三名员工，并计算这些员工的工资占所属部门总工资的百分比。

结果

员工工号	员工工资	部门编号	部门薪资排名	部门总工资	工资占部分比例
7698	2850	30	1	9400	0.30
7499	1600	30	2	9400	0.17
7844	1500	30	3	9400	0.16
7839	5000	10	1	8750	0.57
7782	2450	10	2	8750	0.28
7934	1300	10	3	8750	0.15
7902	3000	20	1	10875	0.28
7788	3000	20	2	10875	0.28
7566	2975	20	3	10875	0.27

```
create or replace temporary view emp(empno,ename,hiredate,sal,deptno) as values
( 7521 , 'ward' , '1981-02-22' , 1250 , 30 ),
( 7566 , 'jones' , '1981-04-02' , 2975 , 20 ),
( 7876 , 'adams' , '1987-07-13' , 1100 , 20 ) ;

select * from emp;

-- 求出每个部门工资最高的前三名员工，并计算这些员工的工资占所属部门总工资的百分比。
select a.empno,
       a.sal,
       a.deptno,
       a.rn,
       a.sum_sal,
       round(a.sal/a.sum_sal,2) as rate
from
(select *,

```





```
--每个部门工资排名
row_number() over (partition by deptno order by sal desc) as rn,
--每个部门的总工资
sum(sal) over(partition by deptno ) as sum_sal
from emp) a
where rn<=3;
```

3. 第2题 统计每年入职总数以及截至本年累计入职总人数。

题目描述

在第一题员工表的基础上，统计每年入职总数以及截至本年累计入职总人数。

截至本年累计入职总人数=本年总入职人数 + 本年之前所有年的总入职人数之和

结果

入职年	本年总入职人数	截止本年累计入职总人数
1980	1	1
1981	10	11
1982	1	12
1987	2	14

```
select *,
       sum(cnt) over (order by year1) cnt2
from
(select year(hiredate) as year1,
       count(1) as cnt
 from emp
 group by year(hiredate)) a;
```





4. 第3题 每个品牌的总营销天数(时间序列题型)

营销活动表结构

marketing	
brand 品牌	string
startdate 营销活动开始日期	date
enddate 营销活动结束日期	date

营销活动表数据

brand	startdate	enddate
华为	2018-08-04	2018-08-05
华为	2018-08-04	2020-12-25
小米	2018-08-15	2018-08-20
小米	2020-01-01	2020-01-05
苹果	2018-09-01	2018-09-05
苹果	2018-09-03	2018-09-06
苹果	2018-09-09	2018-09-15

题目描述

该表记录了每个品牌的营销活动开始日期以及结束日期，现需要统计出每个品牌的总营销天数。

注意

- 1: 苹果第一行数据的营销结束日期比第二行数据的营销开始日期要晚，这部分有重叠的日期的要去重计算。
- 2: 苹果第二行数据的营销结束日期和第三行的开始日期不连续，2019-09-07以及2019-09-08 不统计到营销天数中。

结果

品牌	总营销天数
小米	11
苹果	13
华为	875

```
create or replace temporary view marketing (brand, startdate, enddate) as values
( 'hw' , '2018-08-04' , '2018-08-05' ),
( 'hw' , '2018-08-04' , '2020-12-25' ),
( 'xm' , '2018-08-15' , '2018-08-20' ),
( 'xm' , '2020-01-01' , '2020-01-05' ),
( 'pg' , '2018-09-01' , '2018-09-05' ),
( 'pg' , '2018-09-03' , '2018-09-06' ),
( 'pg' , '2018-09-09' , '2018-09-15' );
```





```
select brand,
       count(distinct day1) cnt
from
(select *,
     explode(sequence(to_date(startdate), to_date(enddate), interval 1 day)) as day1
 from marketing) a
group by brand;
```



腾讯外包(微信相关)

1. 需求列表

需求1: Python数据清洗转换

需求2: 查询当天阅读最多的 100 篇文章

需求3: 查询当天阅读最多的 10 个公众号

2. 第1题 Python数据清洗转换

数据分析师小 Q 收到一份微信用户行为 log 文件，文件中每一行包含三列
(ftime, event_code, event_value)

数据样例如下：

2020-01-18 19:02:48	view_article	A1=wxid123&A3=post123&A4=1282&A5=
2020-01-18 19:02:48	comment_article	A1=wxid123&A3=post123&A4=1282&A5=

说明：

第一列 ftime 记录了用户行行为时间例如 2020-01-18 19:02:48

第二列 event_code 记录了用户事件id， 例如 'view_article' = 阅读文章，
'comment_article' = 评论文章

第三列 event_value 记录了事件详情， 例如 'A1=wxid123&A3=post123&A4=1&A5='

A1 表示用户 ID

A3 表示文章 ID

A4 表示文章类型

请帮助小 Q 实现一个函数，从文件中提取出用户阅读文章的行为字段 (时间, 用户 ID, 文章 ID)

```
# -*- coding:utf-8 -*-  
# Desc:This is Code Desc  
from pyspark import SparkConf, SparkContext  
  
import os
```





```
# 这里可以选择本地PySpark环境执行Spark代码，也可以使用虚拟机中PySpark环境，通过os可以配置
os.environ['SPARK_HOME'] = '/export/server/spark'
PYSPARK_PYTHON = "/root/anaconda3/bin/python"
# 当存在多个版本时，不指定很可能会导致出错
os.environ["PYSPARK_PYTHON"] = PYSPARK_PYTHON
os.environ["PYSPARK_DRIVER_PYTHON"] = PYSPARK_PYTHON

if __name__ == '__main__':
    #1-创建上下文对象
    conf=SparkConf().setAppName('wordcount').setMaster('local[*]')
    sc=SparkContext(conf=conf)
    #这个文件会自动从Pycharm中同步到Linux目录中
    #上下文对象加载源数据，得到RDD，每个元素是每行文字比如【2020-01-18 19:02:48 | view_article |
    A1=wxid123&A3=post123&A4=1282&A5=】
    rdd1=sc.textFile('file:///export/pyworkspace/pyspark_sz27/pyspark_base/data/wx.log')
    #接收每行文字，比如【2020-01-18 19:02:48 | view_article | A1=wxid123&A3=post123&A4=1282&A5=】
    def data_wash(line):
        #arr变成[2020-01-18 19:02:48 , view_article ,A1=wxid123&A3=post123&A4=1282&A5=]
        arr=line.split("|")
        ftime=arr[0]
        #arr2=[' A1=wxid123' , 'A3=post123' , 'A4=1282' , 'A5=']
        arr2=arr[2].split('&')
        arr3_userid=filter(lambda x:x.strip().startswith('A1'), arr2)
        #arr3_userid2=[' A1=wxid123']
        #filter后需要再用list包起来，变成list
        arr3_userid2=list(arr3_userid)
        #user_id_temp= 'A1=wxid123'
        user_id_temp=arr3_userid2[0].strip()
        #uid=wxid123
        uid=user_id_temp.split('=')[1]

        arr3_postid=filter(lambda x:x.strip().startswith('A3'), arr2)
        arr3_postid2=list(arr3_postid)
        postid_temp=arr3_postid2[0].strip()
        postid=postid_temp.split('=')[1]
        #下面的分隔符，要跟后期Hive表的列分隔符相对应，以'\t'为例。
        return ftime+'\t'+uid+'\t'+postid
    #将每个元素，通过data_wash函数进行转换清洗，得到结果RDD2
    rdd2=rdd1.map(data_wash)
    rdd2.saveAsTextFile('hdfs://node1:8020/pydata/wx')
    sc.stop()
```





3. 第2题 查询当天阅读最多的 100 篇文章

上述结果上传 HIVE 表 t_log_clean , 字段分别为 ftime uid postid , 请计算 2020 年 1 月 1 日当天阅读最多的 100 篇文章

```
create table t_log_clean(  
    ftime string,  
    uid string,  
    postid string  
) row format delimited fields terminated by '\t';  
load data inpath 'hdfs:///root/log.log' overwrite into table t_log_clean;  
  
select postid  
from (select postid, row_number() over (order by cnt desc) as rn  
     from (select postid, count() as cnt from t_log_clean where to_date(ftime) = '2020-01-01'  
          group by postid) t) t  
where rn <= 100;
```

4. 第3题 查询当天阅读最多的 10 个公众号

提供关联表 t_post_account 字段 postid accountid , 记录了每篇文章记录的公众号, 请计算2020年 1 月 1 日 当天阅读最多的 10 个公众号

```
select accountid  
from t_log_clean a  
join t_post_account b on a.postid=b.postid  
where to_date(ftime) = '2020-01-01' group by accountid  
order by count(1) desc  
limit 10
```





小米电商

1. 需求列表

需求1：用户订单量统计

需求2：利用增量表得到全量表

需求3：订单量中位数

需求4：订单top10的城市

2. 第1题 用户订单量统计

SQL题目：

要求：编写SQL能运行，数据正确且符合规范，如遇到自定义函数或不记得的函数可以用XX代替

1.已知有如下两个表sale： 字段如下

```
Create table sale_order(
```

```
Order_id bigint comment '订单ID',
```

```
User_id bigint comment '用户ID',
```

```
Order_status int,
```

```
Create_time string,
```

```
Last_update_time string,
```

```
Product_id bigint,
```

```
Product_num bigint
```

```
);
```

用户注册表：

```
Create table user_info(
```

```
user_id bigint comment '用户ID，唯一主键',
```

```
sex string.
```

```
age int);
```





问题：用一条SQL生成完整的用户画像表，包含如下字段：

User id,sex,age,d7order_num,d14_order_num，后面两个字段分别为近7天订单数量，近14天订单数量。

```
create table sale_order(  
    order_id bigint comment '订单ID',  
    user_id bigint comment '用户ID',  
    order_status int ,  
    create_time string,  
    last_update_time string,  
    product_id bigint,  
    product_num bigint  
);  
  
create table user_info(  
    user_id bigint comment '用户ID,唯一主键',  
    sex string,  
    age int  
);  
  
select u.user_id,  
       s.d7order_num,  
       s.d14order_num  
from user_info u  
left join (select user_id,  
                  count(if(create_time >= '7天前' and create_time <= '今天', order_id,null)) as d7order_num,  
                  count(if(create_time >= '14天前' and create_time <= '今天', order_id,null)) as d14order_num  
            from sale_order  
            where create_time >= '14天前'  
            group by user_id) s on u.user_id = s.user_id;
```





3. 第2题 利用增量表得到全量表

2. 有如下两张结构完全相同的表，sale_order（结构同第一题）.sale_order_inc.

其中sale_order 是截止到昨天凌晨的订单全量，sale_order_inc是今天凌晨根据最后修改记录拉取的昨天有更新或新增的订单记录。

问题：

现需要获取一个最新的订单全量，所有订单均是最新状态。

```
create table sale_order_inc like sale_order;
insert overwrite table sale_order
select order_id,
       user_id,
       order_status,
       create_time,
       last_update_time,
       product_id,
       product_num
from (select *, row_number() over (partition by order_id order by last_update_time desc) rn
      from (select * from sale_order union all select * from sale_order_inc) t) t
where t.rn = 1;
```

4. 第3题 订单量中位数

3. 基于以上sale_order表计算每个用户的订单量，并计算该订单量的中位数。

```
with t1 as (
  select user_id, count(order_id) cnt
  from sale_order group by user_id
),
t2 as (
  select user_id,
         cnt,
         row_number() over (order by cnt) rn,
         count(1) over () cnt2
  from t1
)
select user_id,
       cnt,
```





```
sum(if(rn=round(cnt2/2),cnt,null) )over() middle  
from t2;
```

5. 第3题 订单top10的城市

订单表, torder. 字段, user_id, order_id, ctime(10位时间戳), city_id, sale_num, sku_id(商品)

问题: 20201201至今每日订单量top10的城市及其订单量(订单量对order id去重)(在线写)

```
create or replace temporary view t_order (user_id, order_id, ctime, city_id, sale_num, sku_id) as  
values (null);  
with t1 as (select to_date(ctime) cdate, city_id, count(distinct order_id) cnt  
from t_order  
where to_date(ctime) >= '2020-12-01'  
and to_date(ctime) <= `current_date`()  
group by to_date(ctime), city_id),  
t2 as (select *, row_number() over (partition by cdate order by cnt desc) rn from t1)  
select cdate, city_id, cnt  
from t2  
where rn <= 10;
```





鄂州机场

1. 需求列表

需求1: SQL基础考察

2. 第1题 SQL基础考察

现在有三张数据表，一张为学生信息表students，一张为学生分数表score，还有一张为年级课程表class，表结构如下：

students

学号	姓名	年龄	年级
001	张三	6	1
002	李四	7	2
003	王五	8	3

score

学号	课程	分数
001	语文	88
001	数学	79
002	数学	85
003	语文	95
003	数学	80
003	英语	60

class



年级	课程
1	语文
1	数学
2	语文
2	数学
3	语文
3	数学
3	英语

2.1 查询出每个年级参加每门科目考试的总人数及平均成绩

用一条sql查询出每个年级参加每门科目考试的总人数及平均成绩。

```
select count(1) from insurance_ods.area;

create or replace temporary view
students(sid,name,age,cid) as values
  ('001','zs',6,1),
  ('002','ls',6,1);
create or replace temporary view
score(sid,cname,score) as values
  ('001','语文',88),
  ('002','语文',88),
  ('002','数学',88);

create or replace temporary view class(cid,cname)
as values (1,'语文'),(1,'数学');

--1.用一条sql查询出每个年级参加每门科目考试的总人数及平均成绩。
select c.cid,
       c.cname,
       count(distinct t.sid) as cnt,
```





```
avg(s.score) avg_score
from class      c
left join students t on c.cid = t.cid
left join score   s on s.sid = t.sid and s.cname = c.cname
group by c.cid, c.cname;
```

2.2 查询出每个年级参加了所有科目考试的学生的总人数

用一条sql查询出每个年级参加了所有科目考试的学生的总人数。

--2. 用一条sql查询出每个年级参加了所有科目考试的学生的总人数。

```
with t1 as(
    select t.sid, t.name, t.age, t.cid,
           c.cname, score,
           count_if(s.sid is null) over (partition by c.cid,t.sid ) cnt
    from class      c
    left join students t on c.cid = t.cid
    left join score   s on s.sid = t.sid and s.cname = c.cname
)
select cid,count(distinct sid) as cnt2 from t1 where cnt =0 group by cid;
```

2.3 查询2年级参加了所有科目考试并且每门课程的成绩都在80分以上的学生信息

用一条sql查询2年级参加了所有科目考试并且每门课程的成绩都在80分以上的学生信息。

--3. 用一条sql查询2年级参加了所有科目考试并且每门课程的成绩都在80分以上的学生信息。

```
with t1 as(
    select t.sid, t.name, t.age, t.cid,
           c.cname, score,
           count if(s.sid is null) over (partition by c.cid,t.sid ) cnt
    from class      c
    left join students t on c.cid = t.cid
    left join score   s on s.sid = t.sid and s.cname = c.cname
)
```





```
from class      c
left join students t on c.cid = t.cid
left join score   s on s.sid = t.sid and s.cname = c.cname

select distinct sid, name, age, cid from t1 where cnt =0 and score>80;
```



腾讯游戏数据开发A卷

1. 需求列表

需求1：带条件的聚合统计

需求2：请对题目2中写出的两条语句的效率进行说明

需求3：累计统计

需求4：查询每个玩家的最后一条战力值

需求5：like与范围的性能比较

需求6：带条件的聚合统计

需求7：长度length+替换replace函数

2. 选择题

1. Select 语句中用来连接字符串的符号是_C_____.

A. "+" B. "&" C. "|" D. "|"

2. 通过 SQL，您如何在表 Persons 中选择 FirstName 等于 Thomas 而 LastName 等于 Carter 的所有记录？ (C)

A、SELECT * Persons WHERE FirstName LIKE 'Thomas' AND LastName LIKE 'Carter'

B、SELECT FirstName='Thomas', LastName='Carter' FROM Persons

C、SELECT * FROM Persons WHERE FirstName='Thomas' AND LastName='Carter'

D、SELECT * FROM Persons WHERE FirstName='Thomas' OR LastName='Carter'

3. 通过 SQL，您如何根据 "FirstName" 列降序地从 "Persons" 表返回所有记录？ (B)

A、SELECT * FROM Persons SORT 'FirstName' DESC

B、SELECT * FROM Persons ORDER BY FirstName DESC

C、SELECT * FROM Persons ORDER FirstName DESC

D、SELECT * FROM Persons SORT BY 'FirstName' DESC



4、您如何把 "Persons" 表中 "LastName" 列的 "Gates" 改为 "Wilson" ? (B)

A、MODIFY Persons SET LastName='Wilson' WHERE LastName='Gates'

B、UPDATE Persons SET LastName='Wilson' WHERE LastName='Gates'

C、UPDATE Persons SET LastName='Wilson' INTO LastName='Gates'

D、MODIFY Persons SET LastName='Wilson' INTO LastName='Gates'

5、通过 SQL，您如何返回 "Persons" 表中记录的数目? (C D)

A、SELECT COLUMNS(2) FROM Persons

B、SELECT COLUMNS(*) FROM Persons

C、SELECT COUNT(1) FROM Persons

D、SELECT COUNT(*) FROM Persons

3. 解答题

3.1 带条件的聚合统计

表table如下：

DDate shengfu

2015-05-09 胜

2015-05-09 胜

2015-05-09 负

2015-05-09 负

2015-05-10 胜

2015-05-10 负

2015-05-10 负

如果要生成下列结果，该如何写sql语句？





```
DDate      胜  负
2015-05-09  2   2
2015-05-10  1   2

select
    DDate,
    sum(case when shengfu = '胜' then 1 else 0 end) as '胜',
    sum(case when shengfu = '负' then 1 else 0 end) as '负'
from table
group by
    DDate
```

3.2 查询出有两条以上记录的id号

一个表table中的Id有多个记录，查询出有两条以上记录的id号（给出至少两种写法）

```
Select id, count(1) from table group by id having count(1) >=2;

select id from (
    Select id ,row_number() over(partition by id order by id) rn
    from table) t where t.rn=2;
```

3.3 请对题目2中写出的两条语句的效率进行说明

第1种写法效率比较高，一次检索就可以将最终所要的结果集全部获得；

3.4 累计统计

表table如下：

Year	Salary
------	--------





2000	1000
2001	2000
2002	3000
2003	4000

想得到如下形式的查询结果

Year	Salary
2000	1000
2001	3000
2002	6000
2003	10000

sql语句怎么写? (不得使用over函数)

```
create or replace temporary view table0(Year,Salary) as
values
(2000, 1000),
(2001, 2000),
(2002, 3000),
(2003, 4000);
```

使用over函数

```
select
    year,
    sum(salary) over( order by year asc) as salary
from table0;
```

不使用over函数

```
select stack(4,
    2000,sum(if(year<=2000,Salary,0)) ,
    2001,sum(if(year<=2001,Salary,0)) ,
    2002,sum(if(year<=2002,Salary,0)) ,
    2003,sum(if(year<=2003,Salary,0)) ) as (year,Salary)
from table0;
```

结果





	year	salary
1	2000	1000
2	2001	3000
3	2002	6000
4	2003	10000

为莘莘学子改变命运而讲课，为千万学生少走弯路而著书

3.5 查询每个玩家的最后一条战力值

表table如下:

Vopenid	ddatetime	power
小a	2017060506	100
小a	2017060608	112
小a	2017060611	118
小b	2017060606	188
小b	2017060606	188
小b	2017060506	99
小c	2017060708	199
小c	2017060606	177
小c	2017060506	99

使用Sql写出6月6日每个玩家的最后一条战力值?

```
select
    vopenid,
    power
from
(
    select
        vopenid,
        power,
        rank() over(partition by vopenid order by ddatetime desc) as r
    from table
    where
        substr(ddatetime, 1, 8) = '20170606'
)t1
where
    r = 1
```





3.6 like与范围的性能比较

简述一下2个SQL 是否有区别

- a) `select * from g_cardapply where g_applydate like '2011-01-01%';`
b) `select * from g_cardapply where g_applydate >= '2011-01-01 00:00:00' and g_applydate <= '2011-01-01 23:59:59';`

结果上没有区别，

效率上

- b 直接取到指定数据，a还要对每条数据的g_cardapply字段进行判断，b的效率会比a高

3.7 带条件的聚合统计

登录表playerlogin如下：

Vopenid	ddatetime
小a	2017060506
小a	2017060611
小b	2017060606
小b	2017060506
小c	2017060606
小c	2017060506

金币流水表moneyflow如下：

Vopenid	ddatetime	money	AddOrReduce（充值即增加，消费即减少）
小a	2017060506	500	增加
小a	2017060608	1000	减少
小a	2017060612	1000	减少
小b	2017060606	100	减少
小d	2017060106	1050	减少





1) 用sql写出6月6日，有过登录的人数，有过充值的人数，及充值金额（一条sql写出查询三个数值）

```
select
    count(distinct t1.vopenid) as login_num,
    count(distinct if(addOrReduce = '增加', t1.vopenid, null)) as add_num,
    sum(if(addOrReduce = '增加', money, 0)) as add_money
from
    (
        select
            vopenid,
        from playerlogin
        where
            substr(ddatetime, 1, 8) = '20170606'
    )t1
left join
    (
        select
            vopenid,
            money,
            addOrReduce
        from moneyflow
        where
            substr(ddatetime, 1, 8) = '20170606'
    )t2
on
    t1.vopenid = t2.vopenid
```

2) 求出游戏中所有的玩家数

```
select
    count(distinct vopenid) as player_num
from playerlogin
```



3.8 长度length+替换replace函数

表table字段A的值'11001101110000111'，如何通过sql返回'1'的个数？

```
create or replace temporary view table1 (a) as
values ('110011011'),
       ('1100110011'),
       ('11100110011'),
       ('1101');
--方式1
select a,
       length(replace(a,'0','')) as num
from table1;
--方式2
select a,
       (length(a) - length(regex_replace(a, '1', '')))
       / length('1') as num
from table1;
--方式3
SELECT a, size(regex_extract_all(a, '(1)')) as num from table1;
```

	a	num
1	110011011	6
2	1100110011	6
3	11100110011	7
4	1101	3

4. 问答题

1、写出分区表、索引的作用：分别的有点缺点。并且举例说明在什么场景下需要使用

分区表作用：在查询数据时，只从当前分区查询数据，可以提高数据查询的速度；

优点：改善查询性能，提高检索速度；增强可用性；便于维护；

缺点：已存在的表无法直接转化为分区表；分区表无法使用外键约束；

索引作用：创建索引可以提高系统的性能；

优点：可以加快数据的检索速度；通过创建唯一性索引，可以保证数据表中数据的唯一性；



缺点：创建索引和维护索引要耗费时间；索引会占用物理空间；

2、 开放性问题：

因仓库出库入库频繁，`t_warehouse_trade`只保存了最近一个月的操作记录，历史记录归档到文本`t_warehouse_trade.txt`里面，文本大小为100G。

需要从这个大文件中算出每种物品出库的总数量，提供处理思路。（不用给出代码，说明使用的工具、系统，以及处理原理、思路即可）

3、 写出left join 、 right join 、 full join 、 inner join 的区别。

left join：左连接，以左表为主，关联右表数据，查询结果为左表中的所有数据及右表中能和左表关联上的数据，过滤掉右表中没有关联上左表的数据；

right join：右连接，以右表为主，关联左表数据，查询结果为右表中的所有数据及左表中能和右表关联上的数据，过滤掉左表中没有关联上右表的数据；

full join：全连接，查询结果为两个表的并集，不会丢失数据；

inner join：内连接，查询结果为两个表有关联的数据，会过滤掉没有关联上的数据；

4、 增量表和全量表的含义解释？如何用增量表实现全量表？如何用全量表实现增量表？

增量表：新增的数据即本周期产生的数据；

全量表：历史所有的数据包含新增的数据；

用增量表实现全量表：

以天分区为例，先建一个与增量表一样的临时表，用于存放历史分区数据，再将增量表当天数据结合
如：



```
select
*
from table
where
date between '20210101' and '20211121'
```

将历史数据写入临时表table1

```
select
*
from
(
select
*
from table1

union

select
*
from table
where
date = '20211122'
)t1
```

将临时表数据和增量表当天数据写入到当天全量表里

之后只要昨天分区数据和今天分区数据合并，就是全量表数据

用全量表实现增量表：

以天分区为例，今日新增数据，可以用昨天数据跟今天数据进行关联，今天分区数据left join 昨天分区数据然后 取昨天关联条件为null的数据

如：

```
select
t1.*
```





```
from
(
    select
        *
    from table
    where
        date = '20211122'
)t1
left join
(
    select
        *
    from table
    where
        date = '20211121'
)t2
on
    t1.id = t2.id
where
    t2.id is null
```



腾讯视频游戏直播

1. 需求列表

需求1：活跃用户数、次日留存率

需求2：用户留存率、用户活跃度得分排序

需求3：row_number

需求4：having子句的使用

2. 第1题：活跃用户数、次日留存率

表：

tableA :

ds(日期),device,user_id,is_active

20200301,ios,0001,0

20200301,ios,0002,1

20200301,android,0003,1

20200302,ios,0001,0

20200302,ios,0002,0

20200302,android,0003,1

2.1 活跃用户数+grouping sets

表：

tableA :

ds(日期),device,user_id,is_active





20200301,ios,0001,0

20200301,ios,0002,1

20200301,android,0003,1

20200302,ios,0001,0

20200302,ios,0002,0

20200302,android,0003,1

结果:

ds,device,uv (活跃用户数)

device的取值有三个: ios, android, all

```
create or replace temporary view tableA
(ds/*(日期)*/,device,user_id,is_active) as values
('2020-03-01','ios',0001,0),
('2020-03-01','ios',0002,1),
('2020-03-01','ios',0004,1),
('2020-03-01','android',0003,1),
('2020-03-02','ios',0001,0),
('2020-03-02','ios',0002,0),
('2020-03-02','android',0003,1),
('2020-03-02','ios',0005,1),
('2020-03-02','ios',0004,1);

--输出1:
set spark.sql.shuffle.partitions=2;
--方案1-性能差
select ds,
       device ,
       if(grouping__id=1,'all',device) device2,
       count(distinct if(is_active = 1, user_id, null)) uv
from tableA
group by ds,device grouping sets ((ds,device),(ds));
--方案2-性能好，提前过滤数据，属于谓词下推
select ds,
       device ,
       if(grouping__id=1,'all',device) device2,
       count(distinct user_id) uv
from tableA
where is_active = 1
```





```
group by ds,device  
grouping sets ((ds,device),(ds));
```

2.2 次日留存率

20200301的ios设备用户活跃的次日留存率是多少？

```
--输出2  
--方案1，过程见下面的顺序编号  
with t1 as (  
    select user_id,  
        --3-一个用户如果在'2020-03-01'活跃，则cnt1>0  
        count(if(ds = '2020-03-01', 1, null)) cnt1,  
        --4-一个用户如果在'2020-03-02'活跃，则cnt2>0  
        count(if(ds = '2020-03-02', 1, null)) cnt2  
    from tableA  
    --1-预先全局过滤  
    where device = 'ios'  
        and is_active = 1  
        and ds in ('2020-03-01', '2020-03-02')  
    --2-按用户分组  
    group by user_id  
    --6-只筛选'2020-03-01'活跃的用户，他在'2020-03-02'是否活跃，看cnt2=0则不活跃，>0则活跃  
    having cnt1 > 0  
)  
select sum(cnt1) sum1,--'2020-03-01'的活跃数  
    count(if(cnt2 > 0, user_id, null)) sum2,----并且在次日依然活跃的用户数  
    count(if(cnt2 > 0, user_id, null)) / sum(cnt1) rate--次日留存率  
from t1;  
  
--方案2  
with t1 as (  
    select *,  
        --标记是否在'2020-03-01'活跃，0不活跃，非0就是活跃  
        count(if(ds='2020-03-01',1,null)) over(partition by user_id ) s  
    from tableA  
    where device='ios'  
        and is_active = 1  
        and ds in ('2020-03-01', '2020-03-02')  
) ,t2 as (  
    select *,  
        --标记是否在'2020-03-02'的次日活跃  
        if(ds=date_add('2020-03-01',1) ,1,0) as f
```





```
from t1 where s>0
)
select count(distinct user_id) sum1,--'2020-03-01'的活跃数
       sum(f) sum2,--在次日依然活跃的用户数
       sum(f)/count(distinct user_id) rate
from t2;
```

3. 第2题：用户留存率、用户活跃度得分排序

3.1 计算用户留存率

用户活跃表：

dws_user_active_t

ds bigint comment '日期',

device int comment '设备类型',

user_id bigint comment '用户id',

active_score double comment '活跃度'

计算留存率=（昨天活跃用户∩今天活跃用户）/昨天活跃用户

```
create or replace temporary view dws_user_active_t(
    ds          comment '日期',
    device      comment '设备类型',
    user_id     comment '用户id',
    active_score comment '活跃度'
) as values ('2020-03-01','ios','0001',0),
            ('2020-03-01','ios','0002',1),
            ('2020-03-01','ios','0004',1),
```





```
('2020-03-01','android','0003',1),
('2020-03-02','ios','0001',2),
('2020-03-02','ios','0002',2),
('2020-03-02','android','0003',2),
('2020-03-02','ios','0005',3) ;

--1.1
with t1 as (
    select user_id,
        --标记每个用户是否在'2020-03-01'活跃，0不活跃，非0就是活跃
        count_if(ds='2020-03-01' and active_score>0) as cnt1,
        --标记每个用户是否在'2020-03-02'活跃，0不活跃，非0就是活跃
        count_if(ds='2020-03-02' and active_score>0) as cnt2
    from dws_user_active_t
    where ds in ('2020-03-01','2020-03-02')
    group by user_id
) select count_if(cnt1>0 and cnt2>0)/count_if(cnt1>0) rate from t1;

--1.2
with t1 as (
    select user_id,
        --标记每个用户是否在'2020-03-01'活跃，0不存在，非0就是不存在
        count_if(ds='2020-03-01' and active_score>0) as cnt1,
        --标记每个用户是否既在'2020-03-01'活跃，又在'2020-03-02'活跃，2表示存在，非2就是不存在
        count_if((ds='2020-03-01' and active_score>0) or (ds='2020-03-02' and
active_score>0)) as cnt3
    from dws_user_active_t
    where ds in ('2020-03-01','2020-03-02')
    group by user_id
) select count_if(cnt3>=2)/count_if(cnt1>0) rate from t1;
```

3.2 根据不同设备，用户活跃度得分排序，输出排名，20200515当天

```
select *,
    row_number() over (partition by device order by active_score desc) rn
from dws_user_active_t where ds='2020-03-01';
```





4. 第3题 考察点：row_number

三个字段：课程、学生、成绩

输出：每个课程排名前三的学生。

```
select * from
(select *,
    row_number() over (partition by sc.cid order by sc.score desc) rn
from score sc
join student st on c.stuid=sc.stuid
join course c on c.cid=sc.cid ) t
where rn<=3;
```

5. 第4题 考察点：having子句的使用

用户玩游戏记录表 (user_play_games)

用户ID (user_id) 游戏ID (game_id)

a X
a Y
a Z
a S
a T
b X
b Y
b Z
c X
c Y
c T
d X
d Z
d S





创意游戏记录表 (creative_games)

游戏ID (game_id)

X

Y

Z

求玩过全部三款创意游戏的用户

输出: a、b

```
create or replace temporary view
  user_play_games(user_id comment '用户ID',game_id comment '游戏ID')
  as values
('a', 'X'),
('a', 'Y'),
('a', 'Z'),
('a', 'S'),
('a', 'T'),
('b', 'X'),
('b', 'Y'),
('b', 'Z'),
('c', 'X'),
('c', 'Y'),
('c', 'T'),
('d', 'X'),
('d', 'Z'),
('d', 'S');

drop table if exists creative_games;
create or replace temporary view creative_games
(game_id comment '游戏ID') comment '创意游戏记录表'
  as values
('X'),
('Y'),
('Z');

select a.user_id
  from user_play_games a
 join (select game_id, count(1) over () cnt from creative_games) b
    on a.game_id = b.game_id
 group by a.user_id,cnt having count(a.game_id)=cnt
```





黑马程序员
www.itheima.com

传智播客旗下
高端IT教育品牌

为莘莘学子改变命运而讲课，为千万学生少走弯路而著书



传智教育Python大数据开发





帆软[MySQL方向]

1. 需求列表

需求1：销售环比增长率

需求2：分组max-min-avg

需求3：销售业绩综合统计

需求4：LEFT JOIN、RIGHT JOIN、INNER JOIN、FULL JOIN的区别

2. 第1题：销售环比增长率

数据来源：参考题目所用表格&文件自建，说明所用数据库（限定使用MySQL/ORACLE/SQLSERVER），表名，字段名可自定义，数据内容请勿自行修改，

查询数据起始月份到当前月的销售环比，环比=（本月-上月）/上月，结果输出 年份 月份 销售金额 环比

TYEAR	TMONTH	SALE_MONEY
2019	10	1279
2019	11	2316
2019	12	2090
2020	01	1086
2020	02	2046
2020	03	0
2020	04	2959
2020	05	1314
2020	06	2751
2020	07	1492
2020	08	1414





2020	09	2895
2020	10	2999
2020	11	1982
2020	12	2793
2021	01	2156
2021	02	1733
2021	03	2184

```
create database sql_finereport;
use sql_finereport;
select *,
       (sale_money-last_sale_money)/last_sale_money as rate
from
  (select *,
       lag(sale_money) over (order by tyear,tmonth) as last_sale_money
   from table1) t;
```

3. 第2题 分组max-min-avg

数据来源：参考题目所用表格&文件自建，说明所用数据库（限定使用MYSQL/ORACLE/SQLSERVER），表名，字段名可自定义，数据内容请勿自行修改。

查询每个班级的最高分，最低分，平均分，以及大于平均分的人数，

结果输出 班级 最高分 最低分 平均分 大于平均分的人数

CLASSNO	STUDENTNO	GRADE
---------	-----------	-------





CLASS1	1001	86
CLASS1	1002	60
CLASS1	1003	85
CLASS1	1004	73
CLASS1	1005	95
CLASS1	1006	61
CLASS1	1007	77
CLASS1	1008	71
CLASS1	1009	61
CLASS1	1010	78
CLASS2	2001	81
CLASS2	2002	54
CLASS2	2003	57
CLASS2	2004	75
CLASS2	2005	98
CLASS2	2006	75
CLASS2	2007	76
CLASS2	2008	58
CLASS2	2009	73
CLASS2	2010	55
CLASS3	3001	42
CLASS3	3002	90
CLASS3	3003	81
CLASS3	3004	97
CLASS3	3005	68
CLASS3	3006	72
CLASS3	3007	81
CLASS3	3008	79
CLASS3	3009	87
CLASS3	3010	59





```
select classno,
       max(grade) max1,
       min(grade) min1,
       avg(grade) avg2,
       count(if(grade>avg1,grade,null)) cnt
from
  (select *,
       avg(grade) over(partition by classno) avg1
   from table2 ) t
group by classno;
```

4. 第3题 销售业绩综合统计

数据来源：参考题目所用表格&文件自建，说明所用数据库（限定使用MYSQL/ORACLE/SQLSERVER），表名，字段名可自定义，数据内容请勿自行修改

- 1) 查询2020年8月份每个客户每个产品的销售数量总和并根据销售数量总和降序排序，结果输出 年份 月份 客户ID 产品ID 销售数量
- 2) 查询每个产品2020年9月份在法国的销售金额总和，结果输出产品ID 产品名称 销售金额
- 3) 查询所有时间内，排名前3名的客户销售金额总和，结果输出 排名 客户名称 销售金额
- 4) 查询所有时间内，销售金额占比大于等于10%的产品，并按占比大小降序排序，结果输出产品名称 销售金额 占比
- 5) 查询所有时间内，所有产品销售金额占比，按占比大小降序排序，筛选累计占比在前80%的产品，结果输出 排名 产品名称 销售金额 占比 累计占比
- 6) 查询所有时间内，各个国家的销售情况，销售合计金额大于10000视为业绩合格，否则为不合格，结果输出 国家 销售金额 业绩情况
- 7) 查询中国、英国每个月份的销售情况，2020年8月份销售合计金额大于10000视为业绩合格，否则为不合格，2020年9月份销售合计金额大于12000视为业绩合格，否则为不合格，结果输出 月份 中国销售业绩 英国销售业绩





下表字段说明

订单ID(ORDERID)、

时间 (ZTIME) 、

产品ID(PRODUCTID)、

销售数量 (SALE_AMOUNT) 、

客户ID (CUSTOMID) 、

产品名称 (PRODUCTNAME) 、

价格 (PRICE) 、

客户名称(CUSTOMNAME)、

国家 (COUNTRY)

ORDERID	ZTIME	PRODUCTID	SALE_AMOUNT	CUSTOMID
O001	2020-09-10	P010	96	C008
O002	2020-08-29	P008	38	C007
O003	2020-08-10	P007	97	C008
O004	2020-09-27	P005	62	C006
O005	2020-08-17	P007	37	C009
O006	2020-09-06	P006	3	C005
O007	2020-08-30	P009	86	C007
O008	2020-09-04	P001	34	C007
O009	2020-09-09	P003	99	C004
O010	2020-09-06	P002	65	C010
O011	2020-08-08	P005	11	C002
O012	2020-09-20	P002	3	C008
O013	2020-08-15	P004	9	C004
O014	2020-08-28	P007	99	C010
O015	2020-08-23	P003	3	C005
O016	2020-08-08	P006	51	C008
O017	2020-09-04	P009	99	C002
O018	2020-08-12	P007	86	C003





O019	2020-09-22	P001	73	C005
O020	2020-08-03	P009	22	C006
O021	2020-08-22	P007	54	C006
O022	2020-09-29	P005	59	C005
O023	2020-08-15	P003	45	C006
O024	2020-09-12	P001	10	C004
O025	2020-08-23	P004	56	C008
O026	2020-09-17	P003	57	C004
O027	2020-08-23	P002	73	C003
O028	2020-09-22	P003	50	C008
O029	2020-09-22	P003	70	C007
O030	2020-08-13	P006	15	C002

PRODUCTID	PRODUCTNAME	PRICE
P001	产品A	29.18
P002	产品B	49.84
P003	产品C	41.68
P004	产品D	58.88
P005	产品E	49.37
P006	产品F	10.20
P007	产品G	22.87
P008	产品H	23.78
P009	产品I	50.28
P010	产品J	63.96

CUSTOMID	CUSTOMNAME	COUNTRY
C001	客户A	中国
C002	客户B	法国
C003	客户C	中国





C004	客户D	英国
C005	客户E	美国
C006	客户F	中国
C007	客户G	法国
C008	客户H	英国
C009	客户I	美国
C010	客户H	英国

```
drop table if exists orders ;
create table orders (
    ORDERID varchar(10) comment '订单ID',
    ZTIME varchar(10) comment '时间',
    PRODUCTID varchar(10) comment '产品ID',
    SALE_AMOUNT int comment '销售数量',
    CUSTOMID varchar(10) comment '客户ID' ) ;

insert into orders values
('O001', '2020-09-10', 'P010', 96, 'C008'),
('O002', '2020-08-29', 'P008', 38, 'C007'),
('O003', '2020-08-10', 'P007', 97, 'C008'),
('O004', '2020-09-27', 'P005', 62, 'C006'),
('O005', '2020-08-17', 'P007', 37, 'C009'),
('O006', '2020-09-06', 'P006', 3, 'C005'),
('O007', '2020-08-30', 'P009', 86, 'C007'),
('O008', '2020-09-04', 'P001', 34, 'C007'),
('O009', '2020-09-09', 'P003', 99, 'C004'),
('O010', '2020-09-06', 'P002', 65, 'C010'),
('O011', '2020-08-08', 'P005', 11, 'C002'),
('O012', '2020-09-20', 'P002', 3, 'C008'),
('O013', '2020-08-15', 'P004', 9, 'C004'),
('O014', '2020-08-28', 'P007', 99, 'C010'),
('O015', '2020-08-23', 'P003', 3, 'C005'),
('O016', '2020-08-08', 'P006', 51, 'C008'),
```





```
('O017', '2020-09-04', 'P009', 99, 'C002'),  
('O018', '2020-08-12', 'P007', 86, 'C003'),  
('O019', '2020-09-22', 'P001', 73, 'C005'),  
('O020', '2020-08-03', 'P009', 22, 'C006'),  
('O021', '2020-08-22', 'P007', 54, 'C006'),  
('O022', '2020-09-29', 'P005', 59, 'C005'),  
('O023', '2020-08-15', 'P003', 45, 'C006'),  
('O024', '2020-09-12', 'P001', 10, 'C004'),  
('O025', '2020-08-23', 'P004', 56, 'C008'),  
('O026', '2020-09-17', 'P003', 57, 'C004'),  
('O027', '2020-08-23', 'P002', 73, 'C003'),  
('O028', '2020-09-22', 'P003', 50, 'C008'),  
('O029', '2020-09-22', 'P003', 70, 'C007'),  
('O030', '2020-08-13', 'P006', 15, 'C002');
```

```
create table goods(  
    productid varchar(10) ,  
    productname varchar(10),  
    price double  
);
```

```
insert into goods values  
    ('P001', 'A', 29.18),  
    ('P002', 'B', 49.84),  
    ('P003', 'C', 41.68),  
    ('P004', 'D', 58.88),  
    ('P005', 'E', 49.37),  
    ('P006', 'F', 10.20),  
    ('P007', 'G', 22.87),  
    ('P008', 'H', 23.78),  
    ('P009', 'I', 50.28),  
    ('P010', 'J', 63.96);
```

```
drop table if exists customs;  
create table customs(  
    customid varchar(10),
```





```
        customname varchar(10)    ,
        country varchar(10)
    ) ;

insert into customs values
    ('C001', 'A', 'chn'),
    ('C002', 'B', 'frc'),
    ('C003', 'C', 'chn'),
    ('C004', 'D', 'eng'),
    ('C005', 'E', 'usa'),
    ('C006', 'F', 'chn'),
    ('C007', 'G', 'frc'),
    ('C008', 'H', 'eng'),
    ('C009', 'I', 'usa'),
    ('C010', 'H', 'eng');

/*1*/
select '2020' y,
       '08' mon,
       o.CUSTOMID,
       o.PRODUCTID,
       sum(SALE_AMOUNT) sum1
from orders o
where substr(o.ZTIME,1,7)='2020-08'
group by o.CUSTOMID, o.PRODUCTID
order by sum1 desc;

/*2*/
select o.PRODUCTID,g.PRODUCTNAME,
       sum(SALE_AMOUNT) sum1
from orders o,
     goods g,
     customs c
where o.CUSTOMID=c.customid
   and o.PRODUCTID= g.productid
   and substr(o.ZTIME,1,7)='2020-09'
   and c.country='frc'
group by o.PRODUCTID,g.PRODUCTNAME;
```





```
/*3*/
select *
from (select CUSTOMNAME,sum1, row_number() over (order by sum1 desc) as rn
      from (select o.CUSTOMID, CUSTOMNAME, sum(SALE_AMOUNT) as sum1
            from orders o,customs c where o.CUSTOMID=c.customid
            group by CUSTOMID, CUSTOMNAME) t) t
where t.rn <= 3;

/*4*/
select PRODUCTNAME,sum1,rate from
      (select *,sum1/sum(sum1) over () as rate from
            (select o.PRODUCTID,PRODUCTNAME,sum(SALE_AMOUNT) sum1
              from orders o,goods g where o.PRODUCTID=g.productid
              group by o.PRODUCTID,PRODUCTNAME) t)t
where rate>=0.1 order by rate desc;

/*5*/
select rn,PRODUCTNAME, sum1, rate2, rate1
from
      (select *,
            row_number() over (order by sum1 desc) rn,
            sum(sum1) over (order by sum1 desc)/sum(sum1) over () rate1,
            sum1/sum(sum1) over () as rate2
            from
                  (select o.PRODUCTID,PRODUCTNAME,
                        sum(SALE_AMOUNT) sum1
                  from orders o,goods g
                  where o.PRODUCTID=g.productid
                  group by o.PRODUCTID,PRODUCTNAME) t ) t where rate1<=0.8 ;

/*6*/
select c.country,sum(SALE_AMOUNT) sum1,
      case when sum(SALE_AMOUNT)>10000 then '合格'
            else '不合格'
            end flag
from orders o ,customs c where o.CUSTOMID=c.customid group by c.country;

/*7*/
```





```
select mon, chn_amt, eng_amt,
       case when mon='2020-08' and chn_amt>10000 then 'ok'
            when mon='2020-09' and chn_amt>12000 then 'ok'
            else 'not'
       end chn_state,
       case when mon='2020-08' and eng_amt>10000 then 'ok'
            when mon='2020-09' and eng_amt>12000 then 'ok'
            else 'not'
       end eng_state
from
  (select substr(o.ZTIME,1,7) mon,
        sum(case when country='chn' then o.SALE_AMOUNT
                else null
              end) chn_amt,
        sum(case when country='eng' then o.SALE_AMOUNT
                else null
              end) eng_amt
   from orders o , customs c
  where o.CUSTOMID=c.customid
        and c.country in ('eng','chn')
  group by substr(o.ZTIME,1,7) ) t;
```

5. 第4题 LEFT JOIN、RIGHT JOIN、INNER JOIN、FULL JOIN 的区别

数据来源：自建，说明所用数据库（限定使用MYSQL/ORACLE/SQLSERVER）

- 1) 说明LEFT JOIN、RIGHT JOIN、INNER JOIN、FULL JOIN的区别并自建表和数据做演示
- 2) 说明UNION和UNION ALL的区别并自建表和数据做演示
- 3) 说明 NOT IN和NOT EXISTS的区别并自建表和数据做演示 注意：若只回答区别没有演示不得分





黑马程序员
www.itheima.com

传智播客旗下
高端IT教育品牌

为莘莘学子改变命运而讲课，为千万学生少走弯路而著书

传智教育Python大数据开发



PayPal

1. 需求列表

需求1: Use Pyspark to generate plot line chart

需求2: Write a SQL query to get the second highest salary

需求3: Hive SQL 累积访问次数

2. 第1题: Use Pyspark to generate plot line chart

Given a dataset dataset.csv, generate plot line chart

Programming Requirement: load csv file to spark hive table, then plot it as line chart from the table.

Expected output:

1- print the dataframe right before plotting

2- the plot line chart

3. 第2题: Write a SQL query to get the second highest salary from the Employee table.

```
+-----+
| Id | Salary |
+-----+
| 1 | 100    |
| 2 | 200    |
| 3 | 300    |
+-----+
```





For example, given the above Employee table, the query should return 200 as the second highest salary. If there is no second highest salary, then the query should return null.

```
+-----+
| SecondHighestSalary |
+-----+
| 200                 |
+-----+
```

```
create or replace temporary view
  Employee(id,salary) as values
(1,100),
(2,200),
(3,300);

select salary as SecondHighestSalary from
(select *,row_number() over (order by salary desc) rn
from Employee) t
where rn=2;
```

4. 第3题: Hive SQL 累积访问次数

有如下的用户访问数据

userId date visitCount

01 2021/01/21 10

02 2021/01/23 8

03 2021/01/22 8

04 2021/01/20 3

01 2021/01/23 11

01 2021/02/21 8

02 2021/01/23 13

01 2021/02/22 4





要求使用SQL统计出每个用户的累积访问次数？输出

用户id	月份	total
01	2021/01	21
01	2021/02	12
02	2021/01	21
03	2021/01	8
04	2021/01	3

```
create or replace temporary view
login(userId, date, visitCount) as values
('01', '2021/01/21', 10),
('02', '2021/01/23', 8),
('03', '2021/01/22', 8),
('04', '2021/01/20', 3),
('01', '2021/01/23', 11),
('01', '2021/02/21', 8),
('02', '2021/01/23', 13),
('01', '2021/02/22', 4);

select userId,
       date_format(replace(date, '/', '-'), 'yyyy-MM') mon,
       sum(visitCount) total
from login
group by userId, date_format(replace(date, '/', '-'), 'yyyy-MM');
```





Opportunity

1. 需求列表

需求1：举例一下Hive中的数据类型？

需求2：最高成绩课程&每门课程大于80分的学生

需求3：连续三天登录人员

2. 第1题 举例一下Hive中的数据类型？

primitive基本类型int float double decimal string date boolean

复杂类型array struct map

3. 第2题 最高成绩课程&每门课程大于80分的学生

以下t表为学生的成绩表

a) 求出每个学生最高成绩是哪门课程

b) 用一条SQL语句查询出每门课都大于80分的学生姓名

Name course scores

张三 语文 81

张三 数学 75

李四 语文 76

李四 数学 90

王五 语文 81

王五 数学 100

王五 英语 90





```
-- 以下t表为学生的成绩表
create or replace temporary view
  score(name,course,scores) as values
('张三','语文',81),
('张三','数学',75),
('李四','语文',76),
('李四','数学',90),
('王五','语文',81),
('王五','数学',100),
('王五','英语',90);
-- 求出每个学生最高成绩是哪门课程
set spark.sql.shuffle.partitions=2;
select name,
       course
from
  (select name,
         course,
         row_number() over (partition by name order by scores desc) rn
  from score ) t
where rn=1;
-- 用一条SQL语句查询出每门课都大于80分的学生姓名
select name
from score group by name
having min(scores)>80;
```

4. 第3题 连续三天登录人员

3、以下为用户登陆游戏的日期，用一条SQL语句查询出连续三天登录的人员姓名

name	date
张三	2021-01-01





张三	2021-01-02
张三	2021-01-03
张三	2021-01-02
李四	2021-01-01
李四	2021-01-02
王五	2021-01-03
王五	2021-01-02
王五	2021-01-02

```
create or replace temporary view
game(name,date) as values
('张三','2021-01-01'),
('张三','2021-01-02'),
('张三','2021-01-03'),
('张三','2021-01-02'),
('李四','2021-01-01'),
('李四','2021-01-02'),
('王五','2021-01-03'),
('王五','2021-01-02'),
('王五','2021-01-02');

with t1 as ( select distinct name,date from game),
t2 as ( select *,
            row_number() over (partition by name order by date) rn
        from t1),
t3 as ( select *,date_sub(date,rn) date2 from t2 )
select distinct name from t3 group by name,date2 having count(1)>=3;
```



5. 图书借阅分析

本题用到下面三个关系表：

借书卡：card

cno	name	class
卡号	姓名	班级

图书：books

bno	bname	author	price	quantity
书号	书名	作者	单价	库存册数

借书记录：borrow

cno	bno	rdate
借书卡号	书号	应还书日期

备注：限定每人每种书只能借一本；库存册数随借书、还书而改变。

- 1、写出建立books表的全表语句
- 2、从books表中删除当前无人借阅的图书记录
- 3、查询当前同时借有“计算方法”和“组合数学”两本书的读者，输出其借书卡号，并按卡号升序排序输出
- 4、为card表增加1列deptname，字段注释为部门名称
- 5、通过借阅和库存数据设计一个指标评估某书的实际价值
- 6、统计每本书的实际价值并按实际价值降序输出书名

```
--
create table card(cno string,name string,class string);
create table borrow(cno string,bno string,rdate string);
--1
create table if not exists books(
    bno string comment '',
```





```
    bname string comment '',
    author string comment '',
    price double comment '',
    quantity double comment ''
) comment '';

--2
insert overwrite table books
select a.*
from books a
--b表是正在借的书号
left join (select distinct bno from borrow where rdate>=`current_date`()) b
    on a.bno = b.bno
--a表内挑选有b表在借书号的，那其他未借的就被排除了
where b.bno is not null;

--3
select b.cno
from borrow b
join books c on b.bno=c.bno
group by b.cno
having count(if(c.bname in('计算方法','组合数学'),1,null))=2
order by b.cno;

--4
alter table card add columns (deptname string comment '部门名称') ;

--5
--一种书实际价值=这个书总数*这个书单价
--一种书总数=这个书库存数+这个书借阅在途数

--6
select a.bno,
    (a.quantity+nvl(b.cnt,0))*a.price as value
from books a
left join
(select bno,count(1) cnt from borrow where rdate > `current_date` group by bno) b
on a.bno=b.bno
order by value desc;
```





广州网迅

1. 需求列表

需求1：统计每个渠道每天的“充值”、“扣费”交易类型的交易金额

需求2：每天最早和最迟发生交易的客户信息以及时间

需求3：每天交易金额前 10 的产品信息

需求4：每天交易金额前 10 的客户的注册时长

交易信息表(TRADE_T)，字段如下表：

交易日期 TRADE_DATE	交易时间 TRADE_TIME	客户编码 USER_ID	交易类型 TRADE_TYPE	产品编码 PRODUCT_CODE	费用(分) FEE	渠道编码 CHNL_CODE
20210501	035001	4401002	充值	0000001	10000	51b2k5e
20210501	140453	4402361	扣费	0000002	500	51b17v9
20210529	103210	4402651	抵扣	0000003	800	51b2k5n
.....						
20210701	182222	4405982	充值	0000001	30000	51b2k5o

渠道信息表(CHNL_T)，字段如下表：

渠道编码 CHNL_CODE	渠道名称 CHNL_NAME
51b2k5e	10010 联通手机网上营业厅
51b17v9	广州万家乐通讯
51b2k5n	茂名人人欢通讯
.....	
51b2k5o	广州新时空营业厅

客户信息表(USER_T)

客户编码 USER_ID	注册日期 REG_DATE
4401002	20180808
4402361	20190123





4402651	20160622
.....	
4405982	20210218

请通过编写 HiveSQL 完成以下需求，其中TRADE_T 数量巨大，请考虑SQL 的执行效率：

2. 第1题：统计每个渠道每天的充值、扣费交易类型的交易金额

需求 1：提取 2021 年 5 月，每个渠道每天的“充值”、“扣费”交易类型的交易金额。输出字段：日期、渠道名称、该渠道当天充值总金额、该渠道当天扣费总金额。

```
set spark.sql.shuffle.partitions=4;
show databases ;
create schema if not exists exams;
use exams;

drop table if exists TRADE T;
create table if not exists TRADE_T (
    TRADE_DATE string,
    TRADE TIME string,
    USER_ID string,
    TRADE_TYPE string,
    PRODUCE CODE string,
    FEE int,
    CHNL_CODE string
)ROW FORMAT DELIMITED FIELDS TERMINATED BY '\t';
set hive.stats.column.autogather=false;
insert into table TRADE_T VALUES ('20210501','035001','4401002','充值',
', '0000001',10000,'51b2k5e');
insert into table TRADE_T VALUES ('20210501','140453','4402361','扣费',
', '0000002',500,'51b17v9');
insert into table TRADE_T VALUES ('20210529','103210','4402651','抵扣',
', '0000003',800,'51b2k5n');
insert into table TRADE_T VALUES ('20210701','18222','4405982','充值',
', '0000001',30000,'51b2k5o');

create table if not exists CHNL T(
    CHNL_CODE string,
    CHNL_NAME string
)ROW FORMAT DELIMITED FIELDS TERMINATED BY '\t';
insert into CHNL_T VALUES ('51b2k5e','10010 联通手机网上营业厅');
insert into CHNL_T VALUES ('51b17v9','广州万家乐通讯');
insert into CHNL_T VALUES ('51b2k5n','茂名人人欢通讯');
```





```
insert into CHNL_T VALUES ('51b2k5o','广州新时空营业厅');

create table if not exists USER_T(
    USER_ID string,
    REG_DATE string
) ROW FORMAT DELIMITED FIELDS TERMINATED BY '\t';
insert into USER_T values ('4401002','20180808');
insert into USER_T values ('4402361','20190123');
insert into USER_T values ('4402651','20160622');
insert into USER_T values ('4405982','20210218');
```

```
select a.TRADE_DATE,
       b.CHNL_NAME,
       amt1,
       amt2
from
(select TRADE_DATE,
        CHNL_CODE,
        sum(case
              when TRADE_TYPE="充值" then FEE else 0 end) as amt1,
        sum(case
              when TRADE_TYPE="扣费" then FEE else 0 end) as amt2
from TRADE_T where TRADE_DATE between 20210501 and 20210531
group by TRADE DATE,CHNL CODE) a
join CHNL_T b on a.CHNL_CODE=b.CHNL_CODE;
```

3. 第2题：每天最早和最迟发生交易的客户信息以及时间

需求 2：提取 2021 年 6 月每天第一个（最早）发生交易的客户信息以及时间，每天最后一个（最迟）发生交易的客户信息以及时间。输出字段：日期，最早交易时间，最早交易客户的编码，最后一个交易时间，最后交易客户的编码。

```
select TRADE_DATE,
       max(if(rn1=1,TRADE_TIME,null)) as x1,
       max(if(rn1=1,USER_ID,null)) as x2,
       max(if(rn2=1,TRADE_TIME,null)) as x3,
       max(if(rn2=1,USER_ID,null)) as x4
from
(select TRADE_DATE,TRADE_TIME,USER_ID,
        row number() over (partition by TRADE DATE order by trade time ) rn1,
        row_number() over (partition by TRADE_DATE order by trade_time desc )
rn2
from TRADE T where TRADE DATE between 20210601 and 20210630) t
where rn1=1 or rn2=1
```





```
group by TRADE_DATE;
```

4. 第3题：每天交易金额前 10 的产品信息

需求 3：提取 2021 年 7 月每天交易金额前 10 的产品信息。输出字段：日期，产品编码，交易金额，排名。

```
select TRADE_DATE,PRODUCE_CODE,fee,rn
from
(select TRADE_DATE,PRODUCE_CODE,fee,
    row_number() over (partition by TRADE_DATE order by fee desc ) rn
from TRADE T where TRADE DATE between 20210701 and 20210731) t
where rn<=10;
```

5. 第4题：每天交易金额前 10 的客户的注册时长

需求 4：提取 2021 年 7 月充值金额前 10 的客户截止到 7 月底的注册时长。输出字段：年月，客户编码，充值金额，注册时长，排名。

```
select date_format(TRADE_DATE,'YYYY-MM') ym,
    b.USER ID,
    fee,
    --last_day(TRADE_DATE) 表示 7 月底
    datediff(last_day(TRADE DATE) ,REG DATE) days,
    rn
from
(select *
from
    (select date_format(to_date(TRADE_DATE,'yyyymmdd'),'yyyy-MM-dd') TRADE_DATE,
        TRADE DATE,
        USER ID,fee,
        row_number() over (order by fee desc ) rn
    from TRADE T where TRADE DATE between 20210701 and 20210731) a
where rn<=10) b
join USER_T c on b.USER_ID=c.USER_ID;
```





字节跳动

1. 需求列表

需求1：各个角色当日消耗元宝的付费免费比例

需求2：次日留存率

需求3：各服务器各角色各任务的累积活跃度

需求4：【高难度】获得活跃度宝箱时候所完成任务数量分布

本次笔试可以上网查阅资料辅助答题，但是不可以替代他人作答。

2. 第1题：各个角色当日消耗元宝的付费免费比例

1、某个游戏中的元宝分为，付费元宝和免费元宝。玩家购买商城道具时候，可以使用付费元宝也可以使用免费元宝。请使用Hive SQL语句计算出2021-01-01至2021-01-07期间各个角色当日消耗元宝的付费免费比例（付费免费比 = 付费元宝消耗量 / 免费元宝消耗量）。

现有表结构如下：

desc	dm_paid_buy;
#dm_paid_buy	角色使用付费元宝购买商城道具时候记录一条

time	bigint	#购买的时间戳
server_id	string	#服务器 ID
role_id	int	#角色 ID
cost	int	#购买对应道具消耗的付费元宝数量
item_id	int	#购买对应道具的 id
amount	int	#购买对应道具的数量
p_date	string	#登录日期,yyyy-MM-dd

desc	dm_free_buy;
#dm_free_buy	角色使用免费元宝购买商城道具时候记录一条





time	bigint	#购买的时间戳
server_id	string	#服务器 ID
role_id	int	#角色 ID
cost	int	#购买对应道具消耗的免费元宝数量
item_id	int	#购买对应道具的 id
amount	int	#购买对应道具的数量
p_date	string	#登录日期,yyyy-MM-dd

示例:

结果输出

p_date	server_id	role_id	付费免费比
2021-01-01	123	10098	0
2021-01-01	120	10098	0.4
2021-01-02	123	10098	0.2

```
create table dm paid buy
(
    `time`      bigint comment '#购买的时间戳',
    server id string comment '#服务器 ID',
    role_id    int comment '#角色 ID',
    cost       int comment '#购买对应道具消耗的付费元宝数量',
    item id    int comment '#购买对应道具的 id',
    amount     int comment '#购买对应道具的数量',
    p_date     string comment '#登录日期, yyyy-MM-dd'
) comment '角色使用付费元宝购买商城道具时候记录一条';

create table dm_free_buy
(
    `time`      bigint comment '#购买的时间戳',
    server_id string comment '#服务器 ID',
    role id    int comment '#角色 ID',
    cost       int comment '#购买对应道具消耗的免费元宝数量',
    item_id    int comment '#购买对应道具的 id',
    amount     int comment '#购买对应道具的数量',
    p_date     string comment '#登录日期, yyyy-MM-dd'
) comment '角色使用免费元宝购买商城道具时候记录一条';

select a.p date, a.server id, a.role id,
       round(a.cost / b.cost,1) as rate
from (select p_date, server_id, role_id, sum(cost) cost
      from dm paid buy
      where p_date >= '2021-01-01')
```



```
and p_date <= '2021-01-07'  
group by p_date, server_id, role_id) a  
join (select p_date, server_id, role_id, sum(cost) cost  
from dm_free_buy  
where p_date >= '2021-01-01'  
and p_date <= '2021-01-07'  
group by p_date, server_id, role_id) b  
on a.p_date=b.p_date  
and a.server_id=b.server_id  
and a.role_id=b.role_id;
```

3. 第2题：次日留存率

2、请用一个Hive SQL语句计算出2021-01-01至2021-01-07期间所有新增账号的次留。

(tips：次留是指次日留存率，即当日新增的全部账号中第二天仍然有登录的比例。)

现有表结构如下（dm_active_purchase_user_di只记录当日有登录的账号）：

desc dm_active_purchase_user_di;

register_date	string	#新增日期, yyyy-MM-dd
account_id	string	#账号ID
day_pay	string	#账号当天的充值额度
p_date	string	#登录日期, yyyy-MM-dd

示例：

结果输出

p_date	新增账号数量	次留
2021-01-01	109098	0.67
2021-01-02	123498	0.70
2021-01-03	130090	0.69

2021-01-01 当日新增账号数 次留





2021-01-02 当日新增账号数 次留

.....

2021-01-07 当日新增账号数 次留

```
create table dm_active_purchase_user_di(  
    register_date string comment '#新增日期, yyyy-MM-dd',  
    account_id    string comment '#账号 ID',  
    day_pay       string comment '#账号当天的充值额度',  
    p_date        string comment '#登录日期, yyyy-MM-dd'  
);  
  
insert overwrite table dm_active_purchase_user_di values  
(  
'2020-01-01','1',100,'2020-01-01'),  
(  
'2020-01-01','1',200,'2020-01-02'),  
(  
'2020-01-01','9',300,'2020-01-01'),  
(  
'2020-01-02','2',200,'2020-01-02'),  
(  
'2020-01-02','2',300,'2020-01-03');  
  
set spark.sql.shuffle.partitions=4;  
--方案 1, 效率不高, 加载 2 次表, 多次 shuffle  
select register_date, count(distinct a.account_id) as cnt,  
       count(distinct b.account_id) / count(distinct a.account_id) as rate  
from (select distinct register_date, account_id from dm_active_purchase_user_di) a  
left join (select distinct account_id, p_date from dm_active_purchase_user_di) b  
on a.account_id = b.account_id and date_add(a.register_date, 1) = b.p_date  
group by a.register_date;  
--方案 2 效率高, 只加载一次表。  
with t1 as (  
    select distinct register_date, account_id, p_date  
    from dm_active_purchase_user_di  
) ,  
    t2 as (  
        select *,  
               if(datediff(p_date, register_date)=1, 1, 0) as flag  
        from t1  
    )  
select register_date,  
       count(distinct account_id) cnt,  
       sum(flag)/count(distinct account_id) rate  
from t2 group by register_date;
```



3、一款MMORPG游戏有一个每日任务设计，完成不同任务可以获得不同的活跃度奖励。当日活跃度累计超过100，就可以获得一个活跃宝箱（获得宝箱后玩家仍然会继续参加任务获得活跃度，但不会有额外奖励）。活跃度会每日清零，活跃任务也是每日刷新。现有表结构如下

表dm_activity_di（当角色每完成一个任务时候，记录一条获取活跃度的日志）：

server_id	int	#服务器
role_id	int	#角色id
task_id	int	#本次完成的任务id
activity_add	int	#本次角色获取的活跃度
p_date	string	#日志触发的日期, yyyy-MM-dd
time	bigint	#该日志的触发时间戳

4. 第3题：各服务器各角色各任务的累积活跃度

①统计一下2021-01-01当天，各个服务器的各个角色在完成各个任务时累积已获得的活跃度。

示例：

一个角色在一天内全部的活跃度获取日志

server_id	role_id	task_id	activity_add	p_date	time
10001	12345	12	20	2021-01-01	1609467817
10001	12345	13	30	2021-01-01	1609475017
10001	12345	1	10	2021-01-01	1609475982
10001	12345	3	60	2021-01-01	1609478617
10001	12345	6	30	2021-01-01	1609479519

结果输出

server_id	role_id	task_id	activity_add	p_date	time	累积获得的活跃度
10001	12345	12	20	2021-01-01	1609467817	20
10001	12345	13	30	2021-01-01	1609475017	50
10001	12345	1	10	2021-01-01	1609475982	60
10001	12345	3	60	2021-01-01	1609478617	120
10001	12345	6	30	2021-01-01	1609479519	150





```
create table dm_activity_di
(
    server_id    int comment '#服务器',
    role_id      int comment '#角色 id',
    task_id      int comment '#本次完成的任务 id',
    activity_add  int comment '#本次角色获取的活跃度',
    p_date       string comment '#日志触发的日期, yyyy-MM-dd',
    time         bigint comment '#该日志的触发时间戳'
) comment '当角色每完成一个任务时候，记录一条获取活跃度的日志';

insert overwrite table dm_activity_di values
(10001, 12345, 12, 20, '2021-01-01', 1609467817),
(10001, 12345, 13, 30, '2021-01-01', 1609475017),
(10001, 12345, 1, 10, '2021-01-01', 1609475982),
(10001, 12345, 3, 60, '2021-01-01', 1609478617),
(10001, 12345, 6, 30, '2021-01-01', 1609479519);

select *,
       sum(activity_add) over (partition by p_date, server_id, role_id order
by `time`) sum1
from dm_activity_di;
```

5. 第4题：【高难度】获得活跃度宝箱时候所完成任务数量分布

②统计一下2021-01-01当天，角色获得活跃度宝箱时候所完成任务数量分布。

示例：

结果输出

p_date	获得宝箱时候的任务数量	角色数
10001	5	1231234
10001	6	12312
10001	7	123213

```
--写法 1：详细步骤
with t1 as (
```




```
select *,
        sum(activity_add) over (partition by p_date, server_id,role_id order by
`time` ) sum1
from dm_activity_di
) ,
t2 as (
    select *,
        --得到当前活跃度时，闯了几关
        count(1)over (partition by p_date,role_id order by sum1) cnt
    from t1
),
t3 as (
    select * from t2 where sum1>100
),
t4 as (
    select *,
        row_number() over (partition by p_date,role_id order by sum1 ) rn
    from t3
),
t5 as (
    select * from t4 where rn=1
)
select p_date,cnt,
        count(role id) as num roles
from t5 group by p_date,cnt;
--写法1: 合并简化
with t1 as (
    select *,
        sum(activity_add) over (partition by p_date, server_id,role_id order by
`time` ) sum1
    from dm_activity_di
) ,
t2 as (
    select *,
        --得到当前活跃度时，闯了几关,
        count(1)over (partition by p_date,role id order by sum1) cnt,
        row_number() over (partition by p_date,role_id order by
if(sum1>100,sum1,999) ) as rn1
    from t1
)
select p_date,cnt,
        count(role id) as num roles
from t2 where rn1=1
group by p_date,cnt;
```





平安产险

1. 需求列表

需求1：查出所有学科分数都大于80分的学生姓名

需求2：每门学科都高于该学科平均分的学生姓名

需求3：转置/行转列

需求4：求公司每年费用总支出，并按年份降序排列

需求5：求公司每年费用总支出，并按年份降序，按部门支出升序排列

2. 第1题：查出所有学科分数都大于80分的学生姓名

1，现有学生成绩表如下：

姓名	学科	分数
张三	语文	81
张三	数学	75
李四	语文	76
李四	数学	90
王五	语文	81
王五	数学	100

1) 用一条SQL查出所有学科分数都大于80分的学生姓名。

```
--第一问第1题
select `姓名`
from table0 group by `姓名`
having min(`分数`)>80;
```

3. 第2题：每门学科都高于该学科平均分的学生姓名

2) 用一条SQL查出每门学科都高于该学科平均分的学生姓名。

```
--第一问第2题
```



```
select distinct `姓名`  
from  
    (select *,  
        min(`分数`) over (partition by `姓名`) min1,  
        avg(`分数`) over (partition by `学科`) avg1  
    from table0) t1  
where min1>avg1;
```

4. 第3题：转置/行转列

3) 用一条SQL写出如下的查询结果：

姓名 语文 数学

张三 81 75

李四 76 90

王五 81 100

```
--第一问第3题  
select `姓名`,  
    sum(if(`学科`='语文',`分数`,0)) as `语文`,  
    sum(if(`学科`='数学',`分数`,0)) as `数学`  
from table0 group by `姓名`;
```

2、利用HivesQL语句，创建如下两张表：

创建员工基本信息表（EmployeeInfo），字段包括（员工ID，员工姓名，员工身份证号，性别，年龄，所属部门，岗位，入职公司时间，离职公司时间），分区字段为入职公司时间，其行分隔符为‘\n’，字段分隔符为‘\t’。其中所属部门包括行政部、财务部、研发部、教学部，其对应岗位包括行政经理、行政专员、财务经理、财务专员、研发工程师、测试工程师、实施工程师、讲师、助教、班王任等，时间类型值如：2018-05-10 11:00:00创建员工收入表（IncomeInfo），字段包括（员工ID，员工姓名，收入金额，收入所属月份，收入类型，收入薪水的时间），分区字段为发放薪水的时间，其中收入类型包括薪资、奖金、公司福利、罚款四种情况；时间类型值如：2018-05-10 11:00:00。



5. 第4题：求公司每年费用总支出，并按年份降序排列

```
--15 题建表 1
create table employeeInfo(
    emp_id string,
    imp_name string,
    card_id string,
    sex string,
    age int,
    dept string,
    job string,
    hire_time string,
    leave_time string
) partitioned by (hire_date string) comment '员工基本信息表'
row format delimited fields terminated by '\t';

--15 题建表 2
create table incomeInfo(
    emp_id string,
    imp_name string,
    income double,
    month1 string,
    type1 string,
    time1 string
) partitioned by (income_date string) comment '员工收入表'
row format delimited fields terminated by '\t';
```

3、用HOL 实现，求公司每年的员工费用总支出各是多少，并按年份降序排列？

```
--16
select year(income_date) year1,
       sum(income) sum income
from incomeInfo
group by year(income_date)
order by year(income_date) desc;
```

6. 第5题：求公司每年费用总支出，并按年份降序，按部门支出升序排列

4、用HOL实现，求各部门每年的员工费用总支出各是多少，并按年份降序，按部门的支出升序排列？





```
select a.dept,  
       year1,  
       sum(sum_income) sum_income  
from employeeInfo a  
left join (  
    select year(income_date) year1,  
           emp_id,  
           sum(income) sum_income  
    from incomeInfo  
    group by year(income_date), emp_id  
) b on a.emp_id=b.emp_id  
group by a.dept, year1  
order by year1 desc, sum_income;
```

vivo

1. 需求列表

需求1：爆炸函数计算常驻人数top10的城市，男女分布情况

需求2：full join计算对局数，订单数

2. 第1题：爆炸函数计算常驻人数top10的城市，男女分布情况

用户常驻城市信息表addr_info: uid, addr_list

数据示例：

"001", ["深圳", "东莞", "广州"]

"002", ["深圳", "佛山", "广州"]

用户基本信息表user: uid, name, gender, birth

问题：常驻人数top10的城市，男女比例的人数分布情况，要求结果如下：

城市名称，排名，男性人数，女性人数

"北京", 1, 20008, 25005

"深圳", 2, 23408, 30005





```
--面试题 1
use test_db;
create table addr_info(
    --string, int, float, double, boolean 等都是基本类型
    uid string,
    --数组 array, map, struct 都是复杂类型
    addr_list array<string>
);
set hive.exec.mode.local.auto=false;
insert into addr_info
select '001' , array('sz','dg','gz') union all
select '002' , array('sz','fs','gz') ;

select * from addr_info;
create table user (
    uid string,
    name string,
    gender string,
    birth string
);
insert into user
select '001','zs','male','1995-01-01' union all
select '002','ls','female','1996-01-01';

with t1 as (
    select uid,addr from addr_info
    lateral view explode(addr_list) view1 as addr
),
    t2 as (
        select t1.addr,
            --统计每个城市的人数，方便后续排序
            count(t1.uid) cnt,
            count(if(gender='male',t1.uid,null)) cnt_male, --男性人数
            count(if(gender='female',t1.uid,null)) cnt_female--女性人数
        from t1 join user u on t1.uid=u.uid
        group by t1.addr
    )
select addr,
    row_number() over (order by cnt desc) rn,
    cnt_male,
    cnt_female
from t2 ;
```





3. 第2题：用full join计算对局数，订单数

两个表，每个表10亿的数据，王者荣耀

用户id。对局id

table_a (uid, cookie_id) , 其中uid、cookie_id 为联合主键；

用户id。订单id

table_b (uid, order_id) , 其中uid、order_id 为联合主键；

结果。用户。对局数，订单数

求 result (uid, cookie_cnt, order_cnt) , 性能越高越好，使用hql。

```
-- 面试题 2
--2 表 join 前减少数据量，并用 full join 更贴切。
select coalesce(a.uid,b.uid) as uid,
       a.cookie_cnt,
       b.order_cnt
from
(select uid,count(cookie_id) as cookie_cnt from table_a group by uid) a
full outer join
(select uid,count(order id) as order cnt from table b group by uid) b
on a.uid=b.uid;
```

交通银行

1. 需求列表

需求1: left join/ coalesce/ round的用法

需求2: 统计9月份开户且账户余额不为0的有效客户数

需求3: 索引优化

需求4: 转置/行转列

需求5: 拉链表算法



需求6：如何使用拉链表

需求7：Linux命令find/tar

需求8：统计理财产品的客户数，余额汇总，并按照产品客户数降序排列

需求9：统计全行持有产品"05108100"的客户列表中持有其他产品的客户数，并按照产品客户数降序排列

需求10：统计全行所有产品余额总计超过1000万的前10家分行，并按照产品余额总计降序排列

2. 第1题：left join/ coalesce/ round的用法

数据模型如下：

表T1的数据结构：

字段英文名	字段中文名	类型	主键标志	注释
Rec_no	记录号	CHAR(3)	Y	
Ci_no	客户号	CHAR(6)	N	
Cust_Type	客户类型	CHAR(2)	N	
Cre_dt	开户日期	Date	N	
Cus_sts	客户状态	Char(1)	N	Y-正常 N-无效

表T1的数据

Rec_no	ci_no	cust_type	cre_dt	cus_sts
123	111111	01	2010-11-15	Y
234	222222	02	2011-09-01	N
345	333333	02	2012-01-09	Y
456	444444	01	2012-09-08	Y

表T2的数据结构：

字段英文名	字段中文名	类型	主键标志	注释
Ci_no	客户号	CHAR(6)	Y	
AC_no	客户账号	CHAR(9)	Y	
Bal	账号余额	DECIMAL(15, 2)	N	

表T2的数据：

Ci_no char(6)	Ac_no char(9)	Bal decimal(15, 2)
222222	123456789	1000.28
333333	123454321	5000





问题1 请写出

```
select t1.rec_no,  
       t1.ci_no,  
       coalesce(t2.ac_no,'888888888') as ac_no,  
       round(coalesce(t2.bal,0.00),1) as bal
```

from t1

left join t2

on t1.ci_no=t2.ci_no;

的结果，填于下表中。（8分）

Rec_no	ci_no	Ac_no	Bal

实际运行答案：

Rec_no	ci_no	Ac_no	Bal
123	111111	888888888	0
234	222222	123456789	1000.3
345	333333	123454321	5000
456	444444	888888888	0

3. 第2题：统计9月份开户且账户余额不为0的有效客户数

问题2：请编写sql统计在9月份开户且账户余额不为0的有效客户数。（8分）

```
select count(t1.ci_no) as cnt  
from (select * from t1 where month(create_dt)=9 and cust_status='Y') t1  
join  
(select ci_no from t2 group by ci_no having sum(bal)>0) t2  
on t1.ci_no=t2.ci_no;
```

4. 第3题：索引优化

问题3：如果在上一题中，遇到性能问题（当数据量非常大时，尤其是T2表的数据量远超T1表时），请从index入手，分析如何可以提高查询统计的性能；在关联表的顺序上，





请问上一题的sql写法是否合理，是否需要调整顺序，给出理由：（9分）

答

- ① 如果是mysql 数据库，可以在关联字段上 ci no 上创建索引，以提高关联查询效率：
`create index idx_t1_cino on t1(ci no);`
t2 表上要创建主键索引，这样可以提高查询性能。
- ② 在关联表的顺序上，
可以调整将小表 join 大表关联顺序来调整顺序，查看出哪张是大表哪张是小表。

5. 第4题：转置/行转列

三.考察数据展现（1小题，合计8分）表TT的数据模型如下：

字段英文名	字段中文名	类型	主键标志	注释
Year	年度	Int	N	
Quarter	季度	Int	N	
Amt	金额	Int	N	

目前的数据如下：

YEAR	QUARTER	AMT
2011	1	110
2011	2	120
2011	3	130
2011	4	140
2012	4	240
2012	3	230
2012	2	220
2012	1	210

编写SQL得到如下结果

YEAR	Q1	Q2	Q3	Q4
2011	110	120	130	140
2012	210	220	230	240

```
--行转列，固定套路一般是 group by+sum/count(case when)
create or replace temporary view TT(year,quarter,amt) as values
(2011,1,110),
(2011,2,120),
(2011,3,130),
(2011,4,140),
(2012,4,240),
```




```
(2012,3,230),
(2012,2,220),
(2012,1,210);
select * from tt;
select year,
       sum(if(quarter=1,amt,0)) as q1,
       sum(if(quarter=2,amt,0)) as q2,
       sum(if(quarter=3,amt,0)) as q3,
       sum(if(quarter=4,amt,0)) as q4
  from tt group by year
 order by year;

--pivot
select * from tt
pivot(sum(amt) for quarter in (1,2,3,4)) order by year;
```

6. 第5题：拉链表算法

四.拉链算法理解（2小题，合计20分）

拉链算法，顾名思义，有着连接、衔接的意思.拉链算法是目前数据仓库领域比较好的算法之一.通用非常广.记录数据量很大且为全量实体记录历史的操作。

T3表

字段英文名	字段中文名	类型	主键标志	注释
Ci_no	客户号	CHAR(6)	N	
AC_no	客户账号	CHAR(9)	N	
Bal	账号余额	DECIMAL(15, 2)	N	
St_dt	拉链开始日期	Date	N	
End_dt	拉链结束日期	Date	N	

假设在最初（2012-01-05），有以下数据：

Ac_no	Bal	st_dt	End_dt
123456789	100	2012-01-05	2999-12-31
123454321	200	2012-01-05	2999-12-31

当在(2012-01-06)，账户情况发生了变化：

- (1)AC_no为123456789的余额(Bal)变成500
- (2)新增了账号888812345，其余额(Bal)为1000。





问题1: 请写出T3表在2012-01-06批量后的数据情况:

Ac_no	Bal	st_dt	End_dt

```
--拉链表
create or replace temporary view t3(ac_no,bal,st_dt,end_dt)
as values
(123456789,100,'2012-01-05','2999-12-31'),
(123454321,200,'2012-01-05','2999-12-31');
select * from t3;
create or replace temporary view t3_2(ac_no,bal)
as values
(123456789,500),
(888812345,1000);

select * from t3_2;

--5 号的全量拉链数据，并将 update 过的数据关键
drop table if exists t3_new;
create table t3_new as
select t3.ac_no,
      t3.bal,
      t3.st_dt,
      if(t3_2.ac_no is not null and
t3.end_dt='2999-12-31','2012-01-06',t3.end_dt) as end_dt
from t3 left join t3_2 on t3.ac_no=t3_2.ac_no
union all
--6 号的新增的全部数据，都开链
select ac_no,
      bal,
      '2012-01-06' as st_dt,
      '2999-12-31' as end_dt
from t3_2;
```

实际运行结果: select * from t3_new

ac_no	bal	st_dt	end_dt
123456789	100	2012-01-05	2012-01-06
123454321	200	2012-01-05	2999-12-31
123456789	500	2012-01-06	2999-12-31



888812345	1000	2012-01-06	2999-12-31
-----------	------	------------	------------

7. 第6题：如何使用链表表

问题2：假设2012-05-10是一个普通的会计日期，请使用sql列出当前余额大于1000的记录。（8分）

```
select * from t3 new where st dt<='2012-05-10' and end dt>='2012-05-10'
and bal>1000;
```

8. 第7题：Linux命令find/tar

五.SHELL程序编写（1小题，合计12分）

AIX（unix）下，有一存放数据的目录，数据以日期为文件夹进行存放，如：

20120101

20120102

20120103

...

请编写shell程序，对该目录下的所有日期里面的，以"dat"结尾的文件进行压缩。（请务必把重点使用的方法写明）

```
find ./ -name '*.dat' | xargs tar zcf ./test3.tar.gz
```

9. 第8题：统计理财产品的客户数，余额汇总，并按照产品客户数降序排列

六.案例分析：客户产品交叉销售(3小题，合计25分)数据模型如下：

T4：（客户持有产品汇总表）

字段英文名	字段中文名	类型	主键标志	注释
Rec_No	序列号	Char(9)	Y	
Cust_Id	客户编号	CHAR(13)	Y	
Product_Id	产品编号	CHAR(8)	Y	
Amt	产品余额	Decimal(15.2)	N	
Txn_Dt	交易日期	DATE	N	
org_Id	分行编号	CHAR(6)	N	





问题1: 编写一段sQL，统计2012年9月深圳分行（310111）各产品的客户数以及余额汇总，并按照产品客户数降序排列（8分）

产品编号	客户数	余额汇总
XXXX	123456	88888888
XXXX	10000	99999999

```
drop table if exists t4;
create table t4(rec no char(9),cust id char(13),product id char(8),amt
decimal(15,2),txn_dt string,org_id char(6));
insert overwrite table t4 values
(1,'zs','05108100',12300000,'2012-09-22','310111'),
(2,'zs','05108100',12300088,'2012-09-23','310111'),
(3,'zs','e 租宝',12300000,'2012-09-13','310111'),
(4,'zs','e 租宝',12300077,'2012-09-14','310111'),
(4,'zs','余额宝',17400077,'2012-09-15','310111'),
(5,'ls','05108100',12300000,'2012-09-21','310111'),
(6,'ls','05108100',12300099,'2012-09-26','310111'),
(7,'ls','余额宝',12300000,'2012-09-14','310111'),
(8,'ls','余额宝',12300700,'2012-09-16','310111'),
(9,'ww','余额宝 x',12300000,'2012-09-15','310111'),
(10,'ww','余额宝 x',12304000,'2012-09-17','310111'),
(11,'zl','余额宝',12300000,'2012-09-16','310111'),
(12,'zl','余额宝',12300040,'2012-09-18','310111'),
(13,'qq','e 租宝',12300000,'2012-09-17','310111'),
(14,'qq','e 租宝',12306000,'2012-09-18','310111'),
(15,'aa','e 租宝',12306000,'2012-09-18','广州分行'),
(16,'bb','e 租宝',12306000,'2012-09-18','东莞分行'),
(17,'cc','e 租宝',12306000,'2012-09-18','珠海分行'),
(18,'dd','e 租宝',12306000,'2012-09-18','澳门分行'),
(19,'ee','e 租宝',12306000,'2012-09-18','梅州分行'),
(20,'ff','e 租宝',12306000,'2012-09-18','汕头分行'),
(21,'gg','e 租宝',12306000,'2012-09-18','惠州分行'),
(22,'hh','e 租宝',12306000,'2012-09-18','潮州分行'),
(23,'ii','e 租宝',12306000,'2012-09-18','湛江分行'),
(24,'jj','e 租宝',12306000,'2012-09-18','江门分行'),
(25,'kk','e 租宝',12306000,'2012-09-18','韶关分行');
select * from t4;
with b as (
  select *
  from (select *,
        row_number() over (partition by cust_id,product_id order by
txn_dt desc) rn
        from t4
        where org_id = '310111'
        and date_format(txn_dt, 'yyyy-MM') = '2012-09') a
```





```
where a.rn = 1
)
select product_id,
       count(distinct cust_id) as cnt,
       sum(amt)                as amt_all
from b
group by product_id
order by cnt desc;
```

10. 第9题：统计全行持有产品"05108100"的客户列表中持有其他产品的客户数，并按照产品客户数降序排列

问题2：编写一段SQL，找到2012年9月份全行

持有产品"05108100"的客户列表中持有其他产品的客户数，

并按照产品客户数降序排列（9分）

产品编号	客户数
XXXX	123456
XXXX	100000

```
with a as (
--1-去重，只保留最后时间的一条
select *
from (select *,
       row number() over (partition by cust_id ,product_id order by txn_dt desc) rn
      from t4
      where date_format(txn_dt, 'yyyy-MM') = '2012-09') a
where a.rn = 1
),
b as (
  select *
  from (select *,
        --2-用窗口函数标记持有'05108100'的记录
        count_if(product_id = '05108100') over (partition by cust_id) cnt
        from a)
  --3-只要标记 cnt>0，就表示持有'05108100'，窗口函数的好处是，其他产品行数也会保留
  where cnt > 0
  --4-'05108100'的使命已经完成，去掉'05108100'这种行，留下其他产品行
  and product_id != '05108100'
)
--5-按照产品分组，聚合的 cust_id 就是步骤 3 中的那些客户列表，也是题目说的客户列表
select product_id, count(cust_id) cnt2 from b group by product_id order by cnt2 desc;
```





11. 第10题：统计全行所有产品余额总计超过1000万的前10家分行，并按照产品余额总计降序排列

问题3：编写一段SQL，找到2012年9月份全行所有产品余额总计超过1000万的前10家分行，并按照产品余额总计降序排列（8分）

分行编号	产品编号	余额总计
XXXX	XXXX	12345600
XXXX	XXXX	10000000

```
with b as (  
    select *  
    from (select *,  
            row_number() over (partition by cust_id ,product_id order by  
txn_dt desc) rn  
        from t4  
        where date format(txn dt, 'yyyy-MM') = '2012-09') a  
    where a.rn = 1  
)  
select org id,  
       product_id,  
       sum(amt) as amt_total  
from b  
group by org_id,product_id  
having amt_total>10000000  
order by amt total desc  
limit 10;
```





附录

1. 微众银行

1. 核额流水表		
字段名	字段意义	字段类型
ds	日期分区，样例格式为 20200101，每个分区有全量流水	string
sno	每个 ds 内主键，流水号	string
uid	用户 id	string
is_risk_apply	是否核额申请（核额漏斗第一步）取值 0 和 1	bigint
is_pass_rule	是否通过规则（核额漏斗第二步）取值 0 和 1	bigint
is_obtain_qutoa	是否授信成功（核额漏斗第三步）取值 0 和 1	bigint
quota	授信金额	decimal(30,6)
update_time	更新时间 样例格式 为 2020-11-14 08:12:12	string





2. 借据表

字段名	字段意义	字段类型
ds	日期分区，样例格式为 20200101 每个分区有全量借据号	string
duebill_id	借据号（每个日期分区内的主键）	string
uid	用户 id	string
prod_type	产品名称 仅 3 个枚举值 XX 贷 YY 贷 ZZ 贷	string
putout_date	发放日期 样例格式为 2020-10-10 00:10:30	bigint
putout_amt	发放金额	decimal(30,6)
balance	借据余额	decimal(30,6)
is_buliang	状态-是否不良 取值 0 和 1	bigint
overduedays	逾期天数	bigint

3. 模型输出表

字段名	字段意义	字段类型
ds	日期分区，样例格式为 20200101 增量表 部分流水记录可能有更新	string
sno	流水号，主键	string
create_time	创建日期 样例格式为 2020-10-10 00:10:30 与 sno 唯一绑定，不会变更	string
uid	用户 id	string
content	Json 格式 key 值名称为 V01~V06， value 值取值为 0 和 1，	string
create_time	更新日期 样例格式为 2020-10-10 00:10:30	string





笔试题目

说明：SQL 语法请使用 HiveSQL/SparkSQL

1. 基于附录 1《核额流水表》和附录 2《借据表》统计下述指标，请提供统计 SQL

指标	当日新增	昨日新增	历史累计
申请户数			
规则通过户数			
核额成功户数			
授信金额			
平均核额			
发放金额			
户均发放金额			

2. 基于附录 2《借据表》统计下述指标，请提供统计 SQL

产品类型	在贷客户数	在贷余额	不良余额	余额不良率	不良客户数	客户不良率
XX 贷						
YY 贷						
ZZ 贷						
汇总						

3. 基于附录 2《借据表》统计下述指标，请提供统计 SQL

产品类型	户数	余额	逾期率/不良率
逾期 1-30 天			
逾期 30-90 天			
逾期 90 天以上			
逾期合计			
不良合计			





4. 基于附录3《模型输出表》统计下述指标，请提供统计 SQL（备注：value 值为 1 时即命中）

统计日期	统计指标	命中户数	命中率
20201010/累 计	V01		
	V02		
	V03		
	V04		
	V05		
	V06		

5. 基于附录2《借据表》统计下述指标，请提供 Vintage 统计 SQL（备注：mobX 指的是发

基于借据表统计指标，请提供Vintage统计SQL（mobX指的是发放后第X月末的不良余额/发放月金额）

发放月份	发放金 额	MOB1	MOB2	MOB3	MOB4	MOB5	MOB6	MOB7	MOB8	MOB9	MOB10	MOB11	MOB12
2019-10													
2019-11													
2019-12													
2020-01													
2020-02													
2020-03													
2020-04													
2020-05													
2020-06													
2020-07													
2020-08													
2020-09													
2020-10													

2. 华泰证券

1. 用一条 SQL 语句 查询出每门课都大于 80 分的学生姓名

name	kecheng	fenshu
张三	语文	81
张三	数学	75
李四	语文	76
李四	数学	90
王五	语文	81
王五	数学	100
王五	英语	90





2.面试题：表转置

year	month	amount
1991	1	1.1
1991	2	1.2
1991	3	1.3
1991	4	1.4
1992	1	2.1
1992	2	2.2
1992	3	2.3
1992	4	2.4

查成这样一个结果

year	m1	m2	m3	m4
1991	1.1	1.2	1.3	1.4
1992	2.1	2.2	2.3	2.4

3.有表结构及数据如下，请完成相应查询

cid	caption
1	三年二班
2	一年三班
3	三年一班

sid	sname	gender	class_id
1	张三	女	1
2	李四	女	2
3	王五	男	2

tid	tname
1	马云
2	马化腾
3	俞敏洪

cid	cname	teacher_id
1	生物	1
2	体育	1
3	物理	2

sid	student_id	course_id	number
1	1	1	58
2	1	2	68
3	2	2	89

3.1. 查询课程编号“2”的成绩比课程编号“1”课程低的所有同学的学号、姓名。

答: _____

3.2. 查询没有学全所有课的同学的学号、姓名;

答: _____





4.基金产品交易流水表 t_trade, 描述如下:

```
create table t_trade
(init_date integer,    --日期
client_id varchar(20), --客户号
fund_code  varchar(6), --基金产品代码
business_flag integer, --1 卖出, 2 买入
business_balance numeric(38,10) --交易金额
```

);
求每日买入成交金额前十名和卖出成交金额的前十名的基金产品。

5.客户购买理财 hs_asset.stock, 描述如下:

```
create table hs_asset.stock
(init_date integer,    --日期
client_id varchar(20), --客户号
stock_code varchar(6), --股票代码
market_value numeric(38,10) --持仓市值
```

);
定义: $\text{市值增长率} = (\text{当日市值} - \text{上日市值}) / \text{上日市值}$ 。
求每日每个客户每只股票的市值增长率。

17:18

4G



ITWBFW20210039501.docx



3.编写一个脚步, 脚步代码可用python或pyspark或scala。(40分)

需求: cust_pft 是客户 (cust id) 每天 (date) 的资产净值 (pft), 现在需要获得每个客户近1年的最大回撤:

最大回撤定义: 在该客户的净值曲线中, 当出现最大的净值的时点记为m1, 这之后出现的净值比m1那天净值相差最大的净值记为m2, 最大回撤就是(m2-m1)/m1.(注意是出现最大净值之后的最小净值, 两者的差)





3. 招商信诺人寿

监管团队数据库面试题
时间约 45 分钟

I. 基础题

- 简单描述 Table / Extent / Segment / Block 之间的关系。
表 表组 数据 块
- 描述 Tablespace 和 Datafile 之间的关系。
- 描述日志的作用。
记录用户访问 浏览 等各项信息
- 请列举几种表连接的内部算法(非 Inner, Left, Right Join)。
笛卡尔积
- SQL 调整最关注的是什么?
执行顺序 Join 小表在前减少笛卡尔积
- 描述对索引的认识(索引的结构、对 DML 影响、为什么提高了查询性能)。
分页
- 绑定变量是什么? 绑定变量有什么优缺点?
- 比较 TRUNCATE 和 DELETE 命令。
**DELETE: 删除表数据, 不删除表的结构, 只是删除元数据
TRUNCATE: 删除数据不进入回收站, 无法找回, 快速清空**
- FACT Table 上需要建立何种索引?
行索引
- 什么是 SCD? 请列出 SCD 的种类与区别。
拉链表 缓慢渐变拉链表

II. SQL 试题

- 有一张表 T(F1, F2, F3, F4), 要根据字段 F2 排序后取第 8—16 条记录显示, 请写出 SQL。

- 写出下面 SQL 语句的执行顺序:

```
select a.column1, COUNT(*) AS CountValue, SUM(b.column3) AS SumValue
from TableA a inner join TableB b on a.column1 = b.column1 and a.column2 = 1
where b.column3 = 10 group by a.column1 having COUNT(*) > 10
order by a.column1
```

1. from 2. inner join 3. where 4. group by 5. having 6. order by 7. order by 8. select





3. 成绩表 Score: SNO、SCALSS、CHINESE、ENGLISH、ARITH 分别代表学生号、学生所属班级、语文成绩、英语成绩、数学成绩。

数据如下:

SNO	SCALSS	CHINESE	ENGLISH	ARITH
1	1	83	57	73
2	1	58	76	90
3	3	89	84	96
4	2	79	86	82
5	1	58	81	93
6	1	56	90	59
7	3	77	66	77
8	2	63	56	55
9	1	91	78	91
10	3	95	81	50

根据以班级分组，按照语文成绩、英语成绩、数学成绩进行排名。

SNO	SCALSS	CHINESE	ENGLISH	ARITH	CHINESE_RANK	ENGLISH_RANK	ARITH_RANK
5	1	58	81	93	3	2	1
9	1	91	78	91	1	3	2
2	1	58	76	90	3	4	3
1	1	83	57	73	2	5	4
6	1	56	90	59	5	1	5
4	2	79	86	82	1	1	1
8	2	63	56	55	2	2	2
3	3	89	84	96	2	1	1
7	3	77	66	77	3	3	2
10	3	95	81	50	1	2	3

4. 根据下图 1 的数据编写 SQL，实现图 2 中的结果。

ID	NAME
1	/
2	A
3	B
4	C
5	/
6	D
7	E
8	/
9	F
10	G
11	H

ID	NAME	GROUP_ID
2	A	1
3	B	1
4	C	1
6	D	2
7	E	2
9	F	3
10	G	3
11	H	3





5.根据下图中的数据，请编写 SQL 找出申请日期到受理日期间超过 10 个工作日（包含 10 个工作日）的 ID 有哪些？

图1

日期	是否工作日
2020/6/1	Y
2020/6/2	Y
2020/6/3	Y
2020/6/4	Y
2020/6/5	Y
2020/6/6	N
2020/6/7	N
2020/6/8	Y
2020/6/9	Y
2020/6/10	Y
2020/6/11	Y
2020/6/12	Y
2020/6/13	N
2020/6/14	N
2020/6/15	Y
2020/6/16	Y
2020/6/17	Y
2020/6/18	Y
2020/6/19	Y
2020/6/20	N
2020/6/21	N
2020/6/22	Y
2020/6/23	Y
2020/6/24	Y
2020/6/25	N
2020/6/26	N
2020/6/27	N
2020/6/28	Y
2020/6/29	Y
2020/6/30	Y

图2

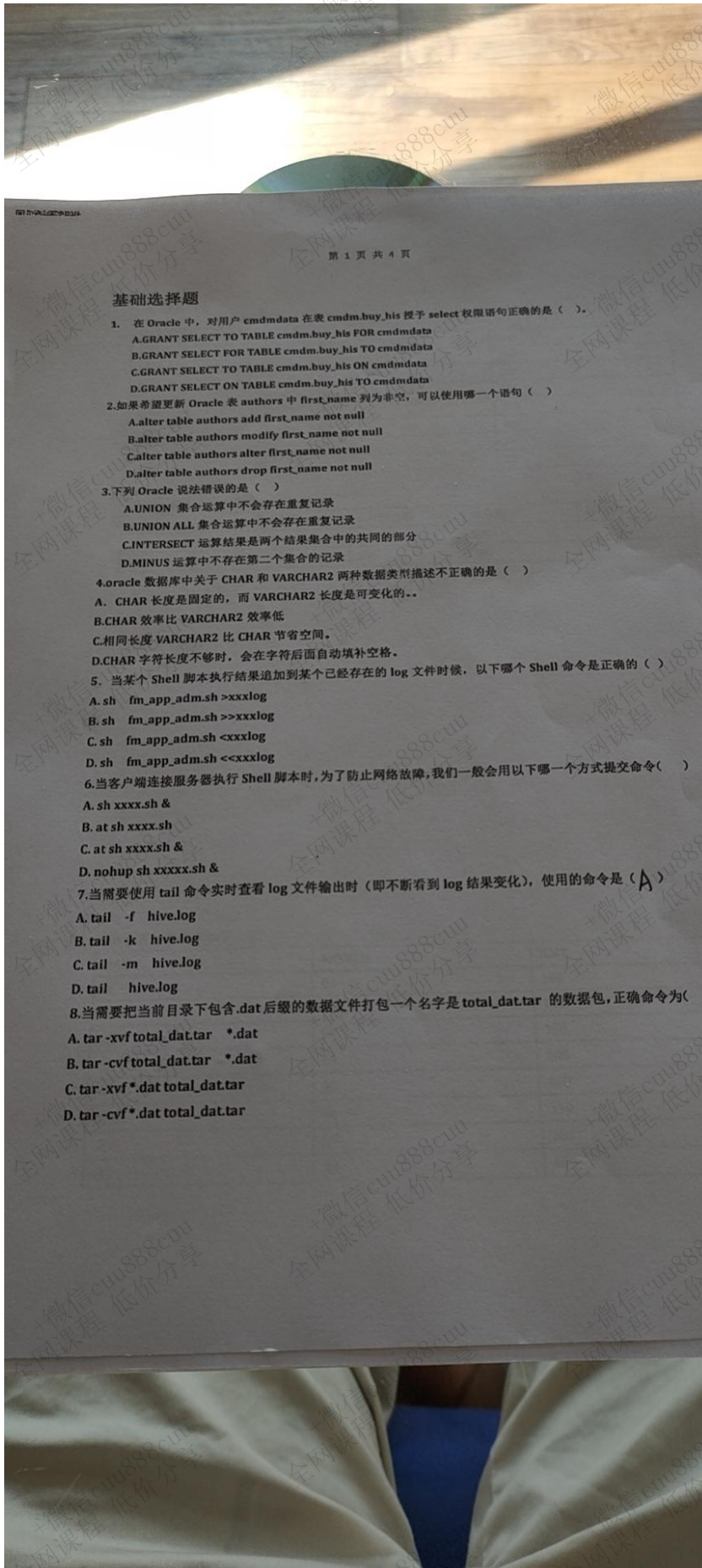
ID	申请日期	受理日期
S001	2020/6/2	2020/6/8
S002	2020/6/3	2020/6/26
S003	2020/6/5	2020/6/15
S004	2020/6/15	2020/6/30
S005	2020/6/18	2020/6/27





4. 交通银行





第 1 页 共 4 页

基础选择题

1. 在 Oracle 中，对用户 cmdmdata 在表 cmdm.buy_his 授予 select 权限语句正确的是（ ）。
A. GRANT SELECT TO TABLE cmdm.buy_his FOR cmdmdata
B. GRANT SELECT FOR TABLE cmdm.buy_his TO cmdmdata
C. GRANT SELECT TO TABLE cmdm.buy_his ON cmdmdata
D. GRANT SELECT ON TABLE cmdm.buy_his TO cmdmdata
2. 如果希望更新 Oracle 表 authors 中 first_name 列为非空，可以使用哪一个语句（ ）。
A. alter table authors add first_name not null
B. alter table authors modify first_name not null
C. alter table authors alter first_name not null
D. alter table authors drop first_name not null
3. 下列 Oracle 说法错误的是（ ）。
A. UNION 集合运算中不会存在重复记录
B. UNION ALL 集合运算中不会存在重复记录
C. INTERSECT 运算结果是两个结果集中的共同的部分
D. MINUS 运算中不存在第二个集合的记录
4. oracle 数据库中关于 CHAR 和 VARCHAR2 两种数据类型描述不正确的是（ ）。
A. CHAR 长度是固定的，而 VARCHAR2 长度是可变化的。
B. CHAR 效率比 VARCHAR2 效率低
C. 相同长度 VARCHAR2 比 CHAR 节省空间。
D. CHAR 字符长度不够时，会在字符后面自动填充空格。
5. 当某个 Shell 脚本执行结果追加到某个已经存在的 log 文件时候，以下哪个 Shell 命令是正确的（ ）。
A. sh fm_app_adm.sh >xxxlog
B. sh fm_app_adm.sh >>xxxlog
C. sh fm_app_adm.sh <xxxlog
D. sh fm_app_adm.sh <<xxxlog
6. 当客户端连接服务器执行 Shell 脚本时，为了防止网络故障，我们一般会用以下哪一个方式提交命令（ ）。
A. sh xxxx.sh &
B. at sh xxxx.sh
C. at sh xxxx.sh &
D. nohup sh xxxxx.sh &
7. 当需要使用 tail 命令实时查看 log 文件输出时（即不断看到 log 结果变化），使用的命令是（ A ）
A. tail -f hive.log
B. tail -k hive.log
C. tail -m hive.log
D. tail hive.log
8. 当需要把当前目录下包含 .dat 后缀的数据文件打包一个名字是 total_dat.tar 的数据包，正确命令为（ ）。
A. tar -xvf total_dat.tar *.dat
B. tar -cvf total_dat.tar *.dat
C. tar -xvf *.dat total_dat.tar
D. tar -cvf *.dat total_dat.tar





简答题目

1: SALES 表的表数据如下:

year	month	amt
2021	04	100
2021	05	200
2020	04	400
2020	05	150
2019	04	300
2019	05	
2020	04	50

问题: 请写出可以得到以下的结果 SQL

year	4 月份金额	5 月份金额	总金额	环比(%)
2020	150	150	600	
2021	100	200	300	

2:

Emp 表的表数据如下:

NAME	MONTH	AMT
张三	01	100
李四	02	120
王五	03	150
赵六	04	500
张三	05	400
李四	06	350
王五	07	180
赵六	08	400

问题: 请写出可以得到以下的结果 SQL

NAME	总金额	排名	占比
赵六	900	1	40.91%
张三	500	2	22.73%
李四	470	3	21.36%
王五	330	4	15.00%





3:

EMP 表的表数据如下,其中 CUS_NO 为主键

CUS_NO	CUS_NAME	ORG_CODE	BALANCE
1	张三	01330999999	100000
2	李四	01331999999	200000
3	王五	01332999999	300000
4	赵六	01331999999	400000
5	胜七	01331999999	500000

EMP_ORG 表的表数据如下,其中 CUS_NO+ORG_CODE 为主键

CUS_NO	ORG_CODE	ORG_NAME	HIER_NO
1	01330999999	深圳分行	G0000001
1	01331999999	广州分行	G0000002
1	01332999999	湛江分行	G0000003

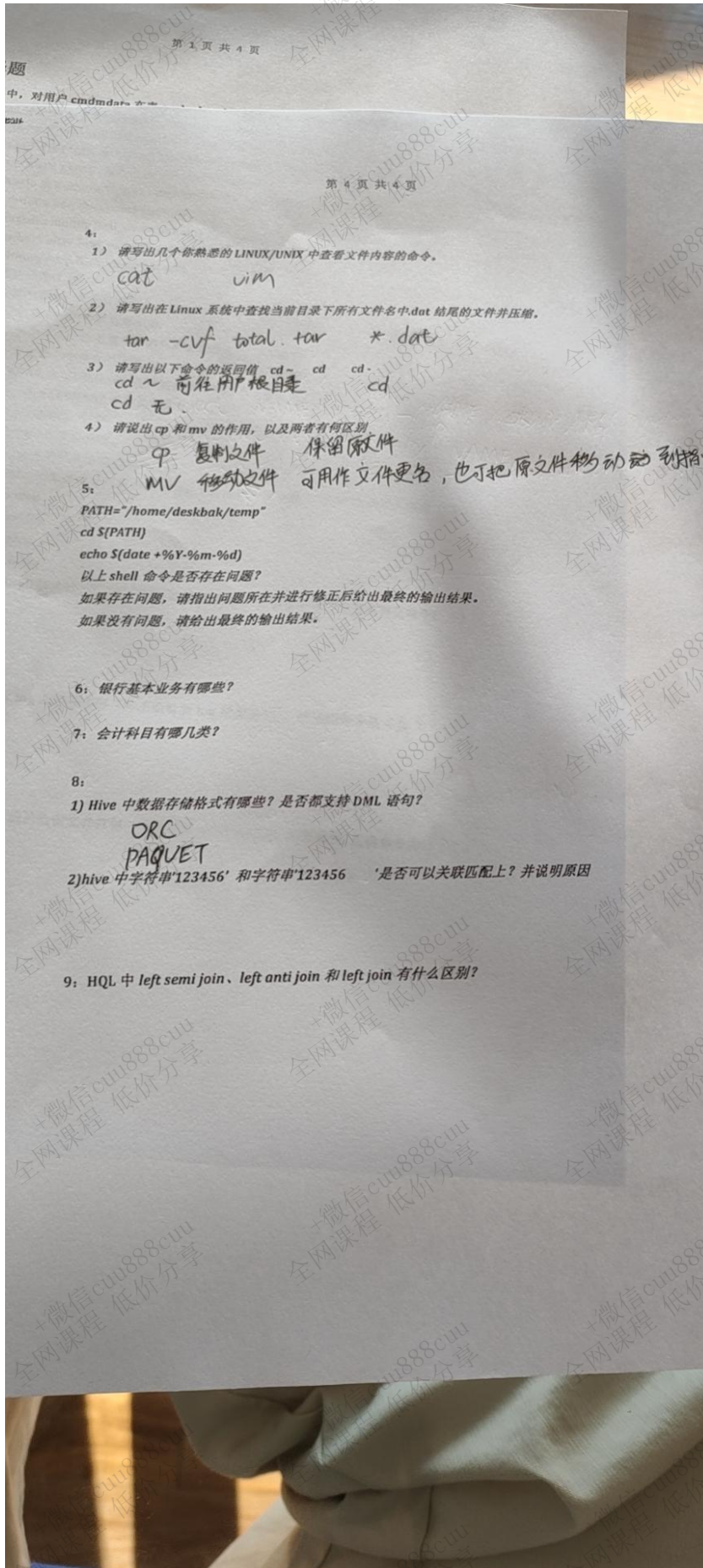
问题1:

```
SELECT      CUS_NO
            ,A1.CUS_NAME
            ,COALESCE(A.ORG_CODE,B.ORG_CODE) AS ORG_CODE
            ,B.ORG_NAME
            ,A.BALANCE
            ,B.HIER_NO
FROM      EMP A
LEFT JOIN EMP_ORG B
        ON A.CUS_NO=B.CUS_NO
```

以上 SQL 是否存在问题? 如果存在问题, 请指出问题, 并修改正确。

问题2: 问题1 中修改正确后的 SQL 可以得到几条数据?







5. 百丽

数仓笔试题（共三道题）

【笔试时间为 30 分钟，做完后请告知前台】

一、请写出将 /home/work/20191020.csv 数据上传到 hive 表 belle_test.dm_belle_csv_data（内部表）的脚本

二、请将如下 sql 修改为 Hive 的窗口分析函数来实现（不使用 union all）

```
select
invoice_type,
delivery_type,
sum(quantity) quantity,
sum(pay_total_amt) pay_total_amt
from bi_analysis.lark_ec_sale_order_parq where pay_time='2019-10-20'
group by invoice_type,delivery_type
union all
select
invoice_type,
delivery_type,
sum(quantity) quantity,
sum(pay_total_amt) pay_total_amt
from bi_analysis.lark_ec_sale_order_parq where pay_time='2019-10-20'
group by invoice_type
```





三、数据表如下

Date	username	user_age	brand	sales_number
2019/10/1	依娜	28	百丽	12
2019/10/1	依娜	28	天美意	30
2019/10/2	丽美	26	百丽	6
2019/10/2	筱雪	20	他她	9
2019/10/2	丽美	26	思加图	1
2019/10/3	筱雪	20	拔佳	2
2019/10/3	诗凝	23	百思图	5

数据字典：

Date: 日期

Username: 用户名称

User_age: 用户年龄

Brand: 购买品牌

Sale_number: 购买件数

问题：

- 1、统计：用户总量、用户平均年龄、用户平均购买件数
- 2、统计：每 5 岁一个年龄分段，统计每个分段用户总量、用户平均购买件数
- 3、统计：每个用户最喜欢购买的品牌
- 4、统计：每个品牌购买件数大于等于 3 的用户总量，如果某个用户有一个品牌购买件数小于 3 就不算，例如：用户丽美，因为思加图品牌购买数小于 3，所以不应该出现在结果中。

6. 跨越物流





7. 腾讯外包(微信相关)

8. 小米电商

SQL 题目:

要求: 编写 SQL 能运行, 数据正确且符合规范, 如遇到自定义函数或不记得的函数可以用 XX 代替

1. 已知有如下两个表:

表 sale: 字段如下

```
Create table sale_order(  
Order_id bigint comment '订单 ID',  
User_id bigint comment '用户 ID',  
Order_status int,  
Create_time string,  
Last_update_time string,  
Product_id bigint,  
Product_num bigint
```

用户注册表:

```
Create table user_info(  
user_id bigint comment '用户 ID, 唯一主键',  
sex string,  
age int  
);
```

问题: 用一条 SQL 生成完整的用户画像表, 包含如下字段:

User_id, sex, age, d7order_num, d14_order_num, 后面两个字段分别为近 7 天定单数量, 近 14 天定单数量。

2. 有如下两张结构完全相同的表, sale_order(结构同第一题), sale_order_inc

其中 sale_order 是截止到昨天凌晨的订单全量, sale_order_inc 是今天凌晨根据最后修改记录拉取的昨天有更新或新增的订单记录。

问题:

现需要获取一个最新的订单全量, 所有订单均是最新状态。





3. 基于以上 sale_order 表计算每个用户的订单量，并计算该订单量的中位数。

42. 订单表, t_order, 字段, user_id, order_id, ctime(10 位时间戳), city_id, sale_num, sku_id(商品)

问题: 20201201 至今每日订单量 top10 的城市及其订单量(订单量对 order_id 去重)(在线写)

9. 鄂州机场

10. 腾讯游戏数据开发A卷

11. 腾讯视频游戏直播

12. 帆软[MySQL方向]

数据来源：参考题目所用表格&文件自建，说明所用数据库（限定使用MYSQL/ORACLE/SQLSERVER），表名，字段名可自定义，数据内容请勿自行修改
查询数据起始月份到当前月的销售环比，环比=（本月-上月）/上月，结果输出 年份 月份 销售金额 环比

数据来源：参考题目所用表格&文件自建，说明所用数据库（限定使用MYSQL/ORACLE/SQLSERVER），表名，字段名可自定义，数据内容请勿自行修改
查询每个班级的最高分，最低分，平均分，以及大于平均分的人数，结果输出 班级 最高分 最低分 平均分 大于平均分的人数

数据来源：参考题目所用表格&文件自建，说明所用数据库（限定使用MYSQL/ORACLE/SQLSERVER），表名，字段名可自定义，数据内容请勿自行修改

- 1) 查询2020年8月份每个客户每个产品的销售数量总和并根据销售数量总和降序排序，结果输出年份 月份 客户ID 产品ID 销售数量
- 2) 查询每个产品2020年9月份在法国的销售金额总和，结果输出产品ID 产品名称 销售金额
- 3) 查询所有时间内，排名前3名的客户销售金额总和，结果输出排名 客户名称 销售金额
- 4) 查询所有时间内，销售金额占比大于等于10%的产品，并按占比大小降序排序，结果输出 产品名称 销售金额 占比
- 5) 查询所有时间内，所有产品销售金额占比，按占比大小降序排序，筛选累计占比在前80%的产品，结果输出 排名 产品名称 销售金额 占比 累计占比
- 6) 查询所有时间内，各个国家的销售情况，销售合计金额大于10000视为业绩合格，否则为不合格，结果输出 国家 销售金额 业绩情况
- 7) 查询中国、英国每个月份的销售情况，2020年8月份销售合计金额大于10000视为业绩合格，否则为不合格，2020年9月份销售合计金额大于12000视为业绩合格，否则为不合格，结果输出 月份 中国销售业绩 英国销售业绩





数据来源：自建，说明所用数据库（限定使用MYSQL/ORACLE/SQLSERVER）

1)说明 LEFT JOIN、RIGHT JOIN、INNER JOIN、FULL JOIN 的区别并自建表和数据做演示

2)说明 UNION 和 UNION ALL 的区别并自建表和数据做演示

3)说明 NOT IN 和 NOT EXISTS的区别并自建表和数据做演示

注意：若只回答区别没有演示不得分

13. PayPal

14. Oppo

1. 列举一下 Hive 中的数据类型？

int, bigint, string, varchar, text

2. 以下 t 表为学生的成绩表

a) 求出每个学生最高成绩是哪门课程

b) 用一条 SQL 语句查询出每门课都大于 80 分的学生姓名

name course scores

张三 语文 81

张三 数学 75

李四 语文 76

李四 数学 90

王五 语文 81

王五 数学 100

王五 英语 90

3. 以下为用户登陆游戏的日期，用一条 SQL 语句查询出连续三天登录的人员姓名

name date

张三 2021-01-01

张三 2021-01-02

张三 2021-01-03

张三 2021-01-02

李四 2021-01-01

李四 2021-01-02

王五 2021-01-03

王五 2021-01-02

王五 2021-01-02





本题用到下面三个关系表：
借书卡：card

card	name	class
卡号	姓名	班级

图书：books

books	name	author	price	quantity
书号	书名	作者	单价	库存册数

借书记录：borrow

borrow	card	books	date
借书卡号	书号	应还书日期	

备注：限定每人每种书只能借一本；库存册数随借书、还书而改变。

- 1、写出建立 books 表的 HQL 语句
- 2、从 books 表中删除当前无人借阅的图书记录
- 3、查询当前同时借有"计算方法"和"组合数学"两本书的读者，输出其借书卡号，并按卡号升序排序输出
- 4、为 card 表增加 1 列 deptname，字段注释为部门名称
- 5、通过借阅和库存数据设计一个指标评估某书的实际价值
- 6、统计每本书的实际价值并按实际价值降序输出书名





15. 广州网讯

16. 字节跳动

17. 平安产险

14. 现有学生成绩表如下：

姓名	学科	分数
张三	语文	81
张三	数学	75
李四	语文	76
李四	数学	90
王五	语文	81
王五	数学	100

1) 用一条 SQL 查出所有学科分数都大于 80 分的学生姓名。

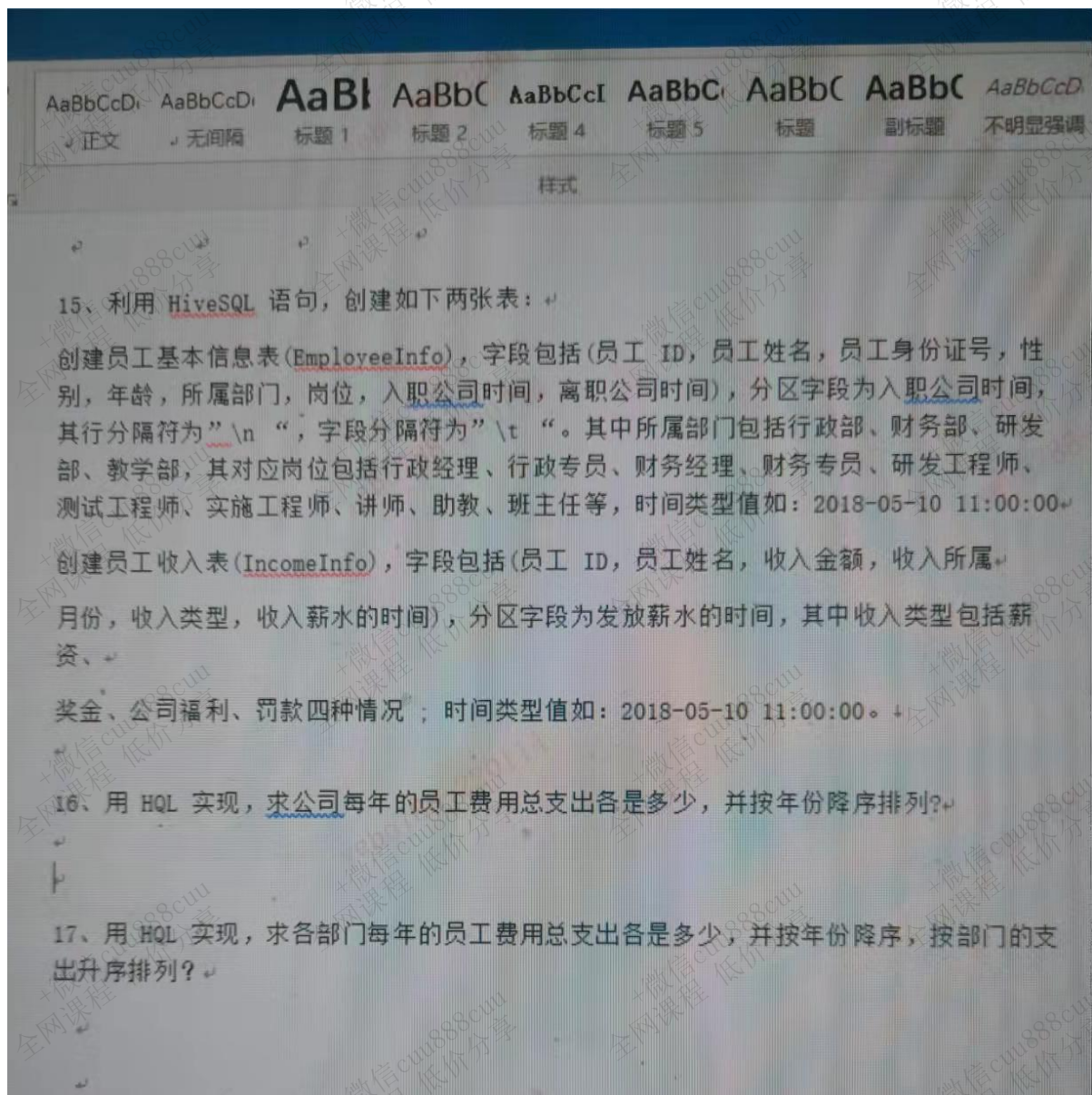
2) 用一条 SQL 查出每门学科都高于该学科平均分的学生姓名。

3) 用一条 SQL 写出如下的查询结果：

姓名	语文	数学
张三	81	75
李四	76	90
王五	81	100

15. 利用 HiveSQL 语句，创建如下两张表：







18. vivo

```
1 用户常驻城市信息表
2 addr_info: uid, addr_list
3 数据示例:
4     "001", [ "深圳,东莞,广州" ]
5     "002", [ "深圳,佛山,广州" ]
6 用户基本信息表
7 user: uid, name, gender, birth
8
9 问题: 常驻人数top10的城市, 男女比例的人数分布情况,
10 要求结果如下:
11     城市名称, 排名, 男性人数, 女性人数
12     "北京", 1, 20008, 25005
13     "深圳", 2, 23408, 30005
14
15 hql
```

```
1 两个表, 每个表10亿的数据,
2 王者荣耀
3
4     用户id. 对局id
5     table_a (uid, cookie_id), 其中uid、cookie_id 为联合主键;
6
7     用户id. 订单id
8     table_b (uid, order_id), 其中uid、order_id 为联合主键;
9
10    结果. 用户. 对局数, 订单数
11 求 result(uid, cookie_cnt, order_cnt),
12
13 性能越高越好, 使用hql.
```





19. 交通银行

二. 考察表关联 (3 小题, 合计 25 分)

数据模型如下:

表 T1 的数据结构:

字段英文名	字段中文名	类型	主键标志	注释
Rec_no	记录号	CHAR(3)	Y	
Ci_no	客户号	CHAR(6)	N	
Cust_Type	客户类型	CHAR(2)	N	
Cre_dt	开户日期	Date	N	
Cus_sts	客户状态	Char(1)	N	Y—正常 N—无效

表 T1 的数据:

Rec_no	Ci_no	Cust_type	Cre_dt	Cus_sts
123	111111	01	2010-11-15	Y
234	222222	02	2011-09-01	N
345	333333	02	2012-01-09	Y
456	444444	01	2012-09-08	Y

表 T2 的数据结构:

字段英文名	字段中文名	类型	主键标志	注释
Ci_no	客户号	CHAR(6)	Y	
Ac_no	客户账号	CHAR(9)	Y	
Bal	账号余额	DECIMAL(15,2)	N	

表 T2 的数据:

Ci_no	Ac_no	Bal
char(6)	char(9)	decimal(15,2)
222222	123456789	1000.28
333333	123454321	5000.00

问题 1. 请写出

```
select t1.rec_no, t1.ci_no, coalesce(t2.ac_no, '888888888') as  
ac_no, round(coalesce(t2.bal, 0.00), 1) as bal from T1 left join T2 on T1.ci_no=T2.ci_no
```



的结果，填于下表中。（8分）

REC_NO	CI_NO	AC_NO	BAL
123	11111	123456789	1000.3
234	22222	123454321	5000.0
345	33333	888888888	0.0
456	44444	888888888	0.0
...	...	...	...

问题2：请编写sql统计在9月份开户且账户余额不为0的有效客户数。（8分）

问题3：如果在上一题中，遇到性能问题（当数据量非常大时，尤其是T2表的数据量远超T1表时），请从index入手，分析如何提高查询统计的性能；在关联表的顺序上，请问上一题的sql写法是否合理，是否需要调整顺序，给出理由：（9分）

三. 考察数据展现（1小题，合计8分）

表T1的数据模型如下：

字段英文名	字段中文名	类型	主键标志	注释
Year	年度	Int	N	
Quarter	季度	Int	N	
Amt	金额	Int	N	

目前的数据如下：

YEAR	QUARTER	AMT
2011	1	110
2011	2	120
2011	3	130
2011	4	140
2012	4	240
2012	3	230
2012	2	220
2012	1	210



2011	110	120	130	140
2012	210	220	230	240

四. 拉链算法理解 (2 小题, 合计 20 分)

拉链算法, 顾名思义, 有着连接、衔接的意思. 拉链算法是目前数据仓库领域比较好的算法之一. 通用非常广. 记录数据量很大且为全量实体记录历史的操作.

T3:

字段英文名	字段中文名	类型	主键标志	注释
Ac_no	账号	CHAR(9)	N	
Bal	账号余额	Decimal(15,2)	N	
St_dt	拉链开始日期	Date	N	
End_dt	拉链结束日期	Date	N	

假设在最初(2012-01-05), 有以下数据:

Ac_no	Bal	St_dt	End_dt
123456789	100	2012-01-05	2999-12-31
123454321	200	2012-01-05	2999-12-31

当在 (2012-01-06), 账户情况发生了变化:

- (1) Ac_no 为 123456789 的余额 (Bal) 变成 500
- (2) 新增了账号 888812345, 其余额 (Bal) 为 1000.

问题 1: 请写出 T3 表在 2012-01-06 批量后的数据情况: (12 分)

Ac_no	Bal	St_dt	End_dt

问题 2: 假设 2012-05-10 是一个普通的会计日期, 请使用 sql 列出当前余额大于 1000 的记录. (8 分)





五. SHELL 程序编写（1 小题，合计 12 分）

AIX（unix）下，有一存放数据的目录，数据以日期为文件夹进行存放，如：

20120101

20120102

20120103

...

请编写 shell 程序，对该目录下的所有日期里面的，以“.dat”结尾的文件进行压缩。（请务必把重点使用的方法写明）

六. 案例分析：客户产品交叉销售（3 小题，合计 25 分）

数据模型如下：

T4: (客户持有产品汇总表)

字段英文名	字段中文名	类型	主键标志	注释
Rec_No	序列号	Char(9)	Y	
Cust_Id	客户编号	CHAR(13)	Y	
Product_Id	产品编号	CHAR(8)	Y	
Amt	产品余额	Decimal(15,2)	N	
Txn_Dt	交易日期	DATE	N	
Org_Id	分行编号	CHAR(6)	N	

问题 1：编写一段 SQL，统计 2012 年 9 月深圳分行（310111）各产品的客户数以及余额汇总，并按照产品客户数降序排列（8 分）

产品编号	客户数	余额汇总
Xxxxxx	123456	88888888
Xxxxxx	100000	99999999
..	..	..

问题 2：编写一段 SQL，找到 2012 年 9 月份全行持有产品“05108100”的客户列表中持有其他产品的客户数，并按照产品客户数降序排列（9 分）

产品编号	客户数
------	-----





Xxxxxx	123456
Xxxxxx	100000
..	..

问题 3：编写一段 SQL，找到 2012 年 9 月份全行所有产品余额总计超过 1000 万的前 10 家分行，并按照产品余额总计降序排列（8 分）

分行编号	产品编号	余额总计
Xxxxxx	Xxxxxx	12345600
Xxxxxx	Xxxxxx	10000000
..	..	..

